



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Análisis Geoespacial y Estimación de Precios Inmobiliarios mediante Redes Neuronales de Grafos

Realizado por
Virgilio Oliva Alonso

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
Manuel Jesús Jiménez Navarro
Andreea Madalina Oprescu Popescu

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de julio, curso 2024/25

Agradecimientos

Aprovecho esta sección para expresar el agradecimiento que siento por todas las personas que han formado parte de mi trayectoria universitaria y que han contribuido en mayor o menor medida a mi desarrollo personal y académico.

En primer lugar, me gustaría expresar mi más sincero aprecio a Manuel Jesús Jiménez Navarro y Andreea Madalina Oprescu Popescu, que además de haber sido unos guías excelentes para encaminar este trabajo por la mejor vía posible, he tenido la suerte de ser su alumno en algunas materias de la carrera y poder aprender la docencia que imparten.

Asimismo, quiero mostrarles gratitud a mis compañeros Cynthia Castaño, Macarena Pereira, Ramón Vergara y José Manuel Miret por brindarme no solo su amistad, sino también un ambiente favorable para el aprendizaje.

También quiero dedicarle este trabajo a mis amistades, quienes han puesto siempre la mano en el fuego por mí y me acompañan en el sendero por perseguir mis sueños y metas. Especialmente me gustaría destacar a mi amiga Mercedes, por brindarme su energía diariamente.

Por último pero no por ello menos importante, quiero dedicarle unas palabras a mi familia, en especial, a mis padres y mi hermano Marcelo, quienes con cariño han puesto toda su energía y esfuerzo para que pudiese llegar al final de esta etapa universitaria.

Resumen

Este proyecto engloba la investigación y el desarrollo para encontrar el mejor modelo de predicción de precios de la vivienda mediante el uso de Redes Neuronales de Grafos.

En primer lugar, se elaboró una serie de planes para estructurar el trabajo a realizar, que permitiría organizar los objetivos, tareas y tiempos de ejecución. A continuación, se llevó a cabo una etapa de investigación marcada por un proceso de descubrimiento del conocimiento, donde se integraron datos de la vivienda interesantes para el contexto del problema, se procesaron y se transformaron para ser representados como un grafo con el que entrenar distintos modelos, con el objetivo de encontrar el mejor, dadas unas métricas de evaluación características para la tarea de predicción.

Como resultado de la extracción del conocimiento, se diseña y se implementa una aplicación funcional que integra un formulario para el ingreso de datos de las viviendas, que devuelve como resultado el precio predicho a través del modelo entrenado, además de mostrar la integración de la vivienda con los datos del grafo, ofreciendo así una herramienta interactiva y útil para evaluar el valor potencial de un inmueble.

Palabras clave: vivienda, extracción del conocimiento, grafos de convolución, predicción.

Abstract

This project encompasses both research and development in the pursuit of identifying the most effective model for housing price prediction using Graph Neural Networks.

In the first place, a structured work plan was defined in order to organize the objectives, tasks, and timeline of execution. Following this, a research phase was carried out, driven by a Knowledge Discovery process. During this phase, relevant housing data were collected, processed, and transformed into a graph representation. This graph was used to train various models with the goal of identifying the most accurate one based on specific evaluation metrics for prediction.

As a result of the knowledge extraction process, a functional application was designed and implemented. It features a form for inputting housing data and provides a predicted price using the trained model. Additionally, the application shows how the property integrates into the graph data structure, offering an interactive and practical tool to evaluate the potential value of a home.

Keywords: housing, knowledge discovery, graph convolution, prediction.

Índice general

1	Introducción	1
1.1.	Contexto	1
1.2.	Motivación	1
1.3.	Objetivos	2
1.4.	Estructura del documento	2
2	Planificación	3
2.1.	Equipo de proyecto	3
2.1.1.	Roles	3
2.1.2.	Integrantes del equipo	4
2.2.	Gestión de los Interesados	5
2.2.1.	Registro de interesados	6
2.2.2.	Clasificación de interesados	7
2.2.3.	Usuarios del sistema	8
2.3.	Gestión del Alcance	8
2.3.1.	Recopilación de requisitos	9
2.3.2.	Reglas de Negocio	14
2.3.3.	Enunciado del Alcance	15
2.3.4.	EDT	16
2.3.5.	Diccionario de la EDT	16
2.4.	Gestión del Tiempo	30
2.4.1.	Hitos del proyecto	30
2.4.2.	Cronograma	30
2.4.3.	Control del cronograma	32
2.5.	Gestión de los Costos	32
2.5.1.	Planificación de los Costos	32
2.5.2.	Desglose de costos	33
2.5.3.	Presupuesto planificado	35
2.5.4.	Presupuesto real	39
2.6.	Gestión de la Calidad	44
2.6.1.	Herramientas	44
2.6.2.	Procesos y técnicas de calidad	44
2.6.3.	Métricas de calidad	45
2.6.4.	Control de calidad	46
2.7.	Gestión de los Riesgos	46
2.7.1.	Registro de Riesgos	47
2.7.2.	Planes de Contingencia	48
2.7.3.	Monitoreo y control de riesgos	49
2.8.	Gestión de la Comunicación	49
2.8.1.	Estrategia de comunicación	49
2.9.	Metodología	49
2.9.1.	Flujo de trabajo	50

2.9.2.	Tecnologías	50
2.9.3.	Gestión de tareas	52
2.9.4.	Patrón de Issues	53
2.9.5.	Estrategia de ramificado	54
2.9.6.	Política de commits	54
2.9.7.	Patrones y procedimientos de revisión	55
2.9.8.	Documentación del sistema	56
2.10.	Replanificación	57
2.10.1.	Hitos	57
2.10.2.	Presupuesto	58
3	Marco teórico	60
3.1.	Introducción	60
3.2.	Fundamentos de Grafos	60
3.3.	Redes Neuronales de Grafos	63
3.4.	Inteligencia Artificial en el Mercado Inmobiliario	69
4	Ejecución	71
4.1.	Introducción	71
4.2.	Knowledge Discovery in Databases	71
4.2.1.	Selección de datos	72
4.2.2.	Preprocesamiento de datos	81
4.2.3.	Transformación de datos	93
4.2.4.	Minería de datos	104
4.2.5.	Evaluación e interpretación	110
4.2.6.	Resultados	119
4.2.7.	Uso del conocimiento	127
4.3.	Aplicación	127
4.3.1.	Introducción	127
4.3.2.	Arquitectura	128
4.3.3.	Manual de instalación	132
4.3.4.	Funciones principales	136
4.3.5.	Pruebas de funcionalidad	138
5	Cierre	143
5.1.	Propuestas de mejora	143
5.2.	Lecciones aprendidas	145
5.3.	Conclusiones	146
	Bibliografía	147
A	Anexos	150
A.1.	Actas de reuniones	150
A.2.	Informe de Clockify	153

Índice de figuras

2.1. Matriz Interés / Poder de los interesados	8
2.2. EDT del proyecto organizada por fases	16
2.3. Cronograma - Paquete planificación	31
2.4. Cronograma - Paquetes de estudio previo, configuración y primera iteración	31
2.5. Cronograma - Paquetes de la segunda y la tercera iteración	31
2.6. Cronograma - Paquetes memoria y manuales	32
2.7. Cronograma - Paquete presentación	32
2.8. Rango de calificación de Codacy	46
2.9. Estructura del tablero Kanban en GitHub	53
2.10. Formato del commit	55
3.1. Representación de aristas dirigidas y no dirigidas	61
3.2. Representación gráfica de un grafo	61
3.3. Tareas de GNN	65
3.4. Esquema del entrenamiento	68
4.1. Fases del proceso KDD	72
4.2. Enmascarado de coordenadas en Valencia	81
4.3. Generación de aristas por vecindad primer caso	96
4.4. Generación de aristas por vecindad segundo caso	96
4.5. Esquema del modelo MTV para Django	128
4.6. Diagrama UML de carpetas de la aplicación	131
4.7. Modelo relacional de la aplicación	132
4.8. Pasos en GitHub para la instalación	134
4.9. Pasos en Visual Studio Code para la instalación	134
4.10. Interfaz de los resultados - Mapa	138

Índice de tablas

2.1. Detalles integrante del Equipo de Proyecto - Virgilio	5
2.2. Detalles integrante del Equipo de Proyecto - Manuel Jesús	5
2.3. Detalles integrante del Equipo de Proyecto - Andreea	5
2.4. Detalles del Interesado - Virgilio	6
2.5. Detalles del Interesado - Manuel Jesús	6
2.6. Detalles del Interesado - Andreea	7
2.7. Detalles del Grupo de Interesados - Mesa Tribunal	7
2.8. Requisito de Información - RI-001	9
2.9. Requisito de Información - RI-002	9
2.10. Requisito de Información - RI-003	10
2.11. Requisito de Información - RI-004	10
2.12. Requisito de Información - RI-005	10
2.13. Requisito Funcional - RF-006	11
2.14. Requisito Funcional - RF-007	11
2.15. Requisito Funcional - RF-008	11
2.16. Requisito Funcional - RF-009	11
2.17. Requisito Funcional - RF-010	12
2.18. Requisito Funcional - RF-011	12
2.19. Requisito Funcional - RF-012	12
2.20. Requisito Funcional - RF-013	12
2.21. Requisito No Funcional - RNF-014	13
2.22. Requisito No Funcional - RNF-015	13
2.23. Requisito No Funcional - RNF-016	13
2.24. Requisito No Funcional - RNF-017	13
2.25. Requisito No Funcional - RNF-018	14
2.26. Regla de Negocio - RN-019	14
2.27. Regla de Negocio - RN-020	14
2.28. Regla de Negocio - RN-021	15
2.29. Regla de Negocio - RN-022	15
2.30. Desglose Paquete de Trabajo - Equipo de proyecto	17
2.31. Desglose Paquete de Trabajo - Gestión de Interesados	18
2.32. Desglose Paquete de Trabajo - Gestión del Alcance	19
2.33. Desglose Paquete de Trabajo - Gestión del Tiempo	20
2.34. Desglose Paquete de Trabajo - Gestión de los Costos	21
2.35. Desglose Paquete de Trabajo - Gestión de la Calidad	21
2.36. Desglose Paquete de Trabajo - Gestión de Riesgos	22
2.37. Desglose Paquete de Trabajo - Metodología	22
2.38. Desglose Paquete de Trabajo - Estudio previo	23
2.39. Desglose Paquete de Trabajo - Configuración del entorno	24
2.40. Desglose Paquete de Planificación - Iteración 1	25
2.41. Desglose Paquete de Planificación - Iteración 2	26
2.42. Desglose Paquete de Planificación - Iteración 3	27

2.43. Desglose Paquete de Trabajo - Memoria	28
2.44. Desglose Paquete de Planificación - Manuales	29
2.45. Desglose Paquete de Trabajo - Presentación	29
2.46. Lista de hitos del Trabajo de Fin de Grado	30
2.47. Lista de costes directos del proyecto	34
2.48. Lista de costes indirectos del proyecto	34
2.49. Lista de costes materiales del proyecto	35
2.50. Desglose detallado de tareas del proyecto	38
2.51. Desglose de costes indirectos del proyecto	38
2.52. Desglose costes materiales del proyecto	39
2.53. Presupuesto planificado del proyecto	39
2.54. Desglose detallado de tareas del proyecto	43
2.55. Desglose de costes indirectos reales del proyecto	43
2.56. Desglose costes materiales reales del proyecto	43
2.57. Presupuesto real del proyecto	43
2.58. Listado de riesgos del proyecto	48
2.59. Listado de planes de contingencia para los riesgos	48
2.60. Lista de hitos replanificados del Trabajo de Fin de Grado	57
2.61. Desglose replanificación de costes indirectos del proyecto	58
2.62. Desglose replanificación de costes materiales del proyecto	58
2.63. Presupuesto replanificado del proyecto	59
4.1. Clasificación de atributos de la vivienda	84
4.2. Clasificación de atributos de los barrios	85
4.3. Media de las métricas del modelo de convolución homogéneo	120
4.4. Evaluación del mejor grafo homogéneo	123
4.5. Media de las métricas del modelo de convolución heterogéneo	124
4.6. Evaluación del mejor grafo heterogéneo	126
4.7. Tecnologías y herramientas requeridas	133
A.2. Registro de reunión del 18 de septiembre de 2024	150
A.4. Registro de reunión del 21 de noviembre de 2024	151
A.6. Registro de reunión del 30 de enero de 2025	151
A.8. Registro de reunión del 27 de marzo de 2025	152
A.10.Registro de reunión del 15 de mayo de 2025	152
A.12.Registro de reunión del 27 de mayo de 2025	153
A.14.Registro de reunión del 11 de junio de 2025	153

Índice de extractos de código

2.1. Ejemplo de función documentada	56
4.1. Función de lectura del CSV	85
4.2. Función de escritura del CSV	86
4.3. Función de procesado de barrios	86
4.4. Función de preprocesamiento	88
4.5. Función de creación de los nodos	93
4.6. Función de creación de aristas por vecindad	94
4.7. Función de creación de aristas por similitud	97
4.8. Función de creación de aristas por barrio	99
4.9. Función para la creación de un grafo	100
4.10. Función para la representación de un grafo	102
4.11. Clase del modelo GCN homogéneo	105
4.12. Función de entrenamiento para GCN homogéneo	105
4.13. Clase del modelo GCN heterogéneo	107
4.14. Función de entrenamiento para GCN heterogéneo	108
4.15. Evaluación por pares en GCN homogéneo	112
4.16. Evaluación por pares en GCN heterogéneo	115
4.17. Pruebas unitarias de la aplicación	138
4.18. Pruebas de integración de la aplicación	140

1. Introducción

En este primer capítulo de la memoria, se tiene como objetivo explicar de forma clara cómo se abordará la realización del Trabajo de Fin de Grado (TFG). Es por ello que se tratarán las siguientes secciones:

- Contexto del problema
- Motivación del proyecto
- Objetivos planteados
- Estructura del documento

1.1. Contexto

Realizar una estimación precisa del precio de la vivienda resulta ser un desafío complejo debido a que hay multitud de factores que influyen en el precio, como pueden ser la ubicación o el las comodidades del inmueble.

En la actualidad, los métodos tradicionales de valoración suelen basarse en modelos estadísticos, los cuales pueden no capturar de forma eficiente algunos tipos de interdependencia entre las viviendas.

Para abordar este problema, el presente Trabajo de Fin de Grado propone realizar estimaciones de los domicilios en la ciudad de Valencia mediante el uso de redes neuronales de grafos (GNN), una técnica avanzada de aprendizaje profundo que permite modelar relaciones espaciales o estructurales dentro del conjunto de datos.

1.2. Motivación

La motivación principal por la cual se ha elegido este Trabajo de Fin de Grado, surge del interés por la investigación y la experimentación de datos de la vivienda, haciendo hincapié en el mundo de redes neuronales de grafos, hasta entonces un entorno desconocido para el estudiante.

De igual manera, viene de querer planificar y desarrollar un proyecto software de alto grado de complejidad, siguiendo las buenas prácticas explicadas en el grado para así afianzar todo el aprendizaje adquirido.

Es por esto que la realización del presente trabajo será una gran forma de despedir la etapa universitaria por todo lo alto, dejando un buen punto de partida para el mundo laboral.

1.3. Objetivos

Existen diversos objetivos a lo largo del proyecto, pero los más significativos son los siguientes:

- **OBJ-001 - Crear un software de experimentación:** el software a desarrollar durante la etapa de ejecución, servirá como una plataforma para el tratamiento de datos de la vivienda, entrenar los modelos de aprendizaje y evaluar su rendimiento. Deberá ser modular y escalable a grandes volúmenes de datos.
- **OBJ-002 - Analizar los resultados:** se aplicarán métricas a los modelos de entrenamiento para generar resultados del rendimiento de los mismos, comparando todos los algoritmos y formas de transformar los datos para finalmente escoger el que mejor rinda para las consultas de precio.
- **OBJ-003 - Diseñar una interfaz web:** se desarrollará una aplicación web interactiva para que los distintos usuarios interesados en el proyecto tengan acceso a la experimentación realizada, además de poder probar el modelo finalmente seleccionado y predecir el precio de un domicilio introduciendo una serie de parámetros.
- **OBJ-004 - Redactar la memoria:** se documentará todo lo alusivo al proyecto, de tal forma que quede reflejada toda la investigación y trabajo realizado a lo largo del ciclo de vida del mismo.

1.4. Estructura del documento

Para que la lectura de la memoria siga una cohesión y sea lo más legible posible, se ha decidido estructurarla de la siguiente forma:

- **Planificación:** contiene todos los planes elaborados para guiar al proyecto durante su desarrollo hacia el éxito.
- **Marco teórico:** capítulo de la memoria que tiene intención de poner al lector en el contexto teórico del problema, sirviendo este de apoyo al siguiente.
- **Ejecución:** capítulo dedicado a explicar en profundidad el contexto del problema y todos los procesos seguidos para resolver el mismo, incluyendo las fases de investigación e implementación.
- **Cierre:** etapa del documento final dedicada a hacer reflexión de todo el trabajo, contemplando lecciones aprendidas, conclusiones y propuestas de mejora.
- **Anexos:** se trata del conjunto de documentos alusivos al trabajo, como pueden ser actas de reunión o informes de retrospectiva, entre otros.

2. Planificación

La planificación es una fase primordial en el desarrollo de cualquier software y más para un proyecto de este calibre. Esto lleva a que antes de dar comienzo a implementar una solución al contexto del problema, deberán asentarse unas bases necesarias para llevar a cabo una etapa de ejecución lo más favorable posible.

Es por ello que este capítulo se dedica a plasmar la planificación seguida durante el desarrollo del proyecto, siguiendo algunas de las buenas prácticas establecidas por PMBOK [1].

2.1. Equipo de proyecto

Previo a realizar los Planes de Gestión de este proyecto resulta necesario conocer quiénes son los altos responsables de que este se ponga en marcha. Por ello en este apartado se dará a conocer al equipo de trabajo, detallando asimismo datos de los integrantes de este y los distintos roles con los que se trabaja.

2.1.1. Roles

Siguiendo las pautas de la guía, se reconocen los siguientes perfiles dentro del Equipo de Proyecto:

- **Director del proyecto:** representa al alto cargo dentro de la estructura jerárquica del equipo de proyecto y es el encargado de liderar y coordinar al equipo para asegurar el cumplimiento de los objetivos, además de establecer las comunicaciones con los interesados.
- **Personal de dirección del proyecto:** compuesto por aquellos miembros del equipo que están encargados de realizar actividades para la dirección de proyectos tales como elaborar el cronograma, preparar presupuestos o gestionar riesgos, entre otros.
- **Personal del proyecto:** son aquellos miembros del equipo de proyecto encargados de llevar a cabo el trabajo de crear los entregables que se realizarán. Dentro de este perfil, se distinguen diversos roles:
 - **Analista:** personal encargado de recopilar las demandas de los interesados para posteriormente transformar estas necesidades en requisitos que propongan soluciones a sus peticiones.
 - **Especialista de calidad:** responsable del aseguramiento de la calidad de los productos finales y entregables del proyecto.

- **Documentador técnico:** figura encargada de redactar aquella documentación del proyecto de forma estructurada y clara al público objetivo, tales como la memoria del trabajo o los manuales de usuario.
- **Desarrollador:** son aquellos miembros encargados de implementar un software que de solución a los requisitos del proyecto. Se identifican varios subroles bajo este perfil:
 - **Ingeniero de Datos:** responsable de transformar los datos con los que se trabaja en estructuras limpias y procesadas, con el principal objetivo de que sean bien aceptados por los modelos que se entrenarán con los mismos.
 - **Científico de Datos:** se encarga de experimentar con los datos estableciendo distintas relaciones entre estos para ver de qué forma interactúan mejor. Evalúan por medio de métricas a los modelos de predicción y seleccionan aquellos que devuelvan mejores resultados.
 - **Desarrollador Backend:** personal encargado de implementar toda la lógica para cumplir con las necesidades del sistema software.
 - **Desarrollador Frontend:** rol encargado de elaborar una interfaz conectada a la lógica del proyecto para permitir a los usuarios interactuar con la funcionalidad de la aplicación.
- **Tester:** gestionan la elaboración de pruebas sobre el software implementado para asegurar el cumplimiento de la funcionalidad y dotar de calidad al proyecto.
- **Representantes del Cliente:** son aquellos miembros de la organización del cliente que aceptarán los entregables recibidos durante el ciclo de vida del proyecto, siendo responsables directos de la aceptabilidad de los resultados del mismo.

2.1.2. Integrantes del equipo

Una vez comprendida la estructura y roles que se llevarán a cabo en el equipo de proyecto se puede dar paso a conocer a los integrantes que formarán parte del mismo, así como algunos de sus datos más relevantes recogidos en los siguientes cuadros:

Ficha integrante	
Apellidos, Nombre	Oliva Alonso, Virgilio
Roles	Director y personal del proyecto (analista, especialista de calidad, documentador técnico, desarrollador y tester)
Organización	Escuela Técnica Superior de Ingeniería Informática

Tabla 2.1: Detalles integrante del Equipo de Proyecto - Virgilio

Ficha integrante	
Apellidos, Nombre	Jiménez Navarro, Manuel Jesús
Roles	Representante del Cliente
Organización	Departamento de Lenguajes y Sistemas Informáticos

Tabla 2.2: Detalles integrante del Equipo de Proyecto - Manuel Jesús

Ficha integrante	
Apellidos, Nombre	Opreescu Popescu, Andreea
Roles	Representante del Cliente
Organización	Departamento de Tecnología Electrónica

Tabla 2.3: Detalles integrante del Equipo de Proyecto - Andreea

2.2. Gestión de los Interesados

Realizar un Plan para la Gestión de los Interesados resulta crucial en la etapa de planificación del proyecto, ya que servirá de apoyo a la hora de conocer las necesidades de los interesados y contactar con ellos.

Es por ello que esta sección incluye todo proceso necesario para la identificación de aquellas personas, grupos u organizaciones que puedan tener impacto en el desarrollo del proyecto, además de analizar a estos individuos y entidades para la posterior clasificación de los mismos.

2.2.1. Registro de interesados

Se han identificado cuatro interesados, entre los que se encuentra un grupo. Los detalles de los mismos han sido recogidos a través de las siguientes tablas:

Ficha interesado	
ID	STK-001
Apellidos, Nombre	Oliva Alonso, Virgilio
Cargo	Estudiante de Ingeniería de Software
Organización	Escuela Técnica Superior de Ingeniería Informática
Intereses	Sacar una calificación alta en el Trabajo de Fin de Grado
Nivel de influencia	Alta

Tabla 2.4: Detalles del Interesado - Virgilio

Ficha interesado	
ID	STK-002
Apellidos, Nombre	Jiménez Navarro, Manuel Jesús
Cargo	Tutor del Trabajo de Fin de Grado
Organización	Departamento de Lenguajes y Sistemas Informáticos
Intereses	Guiar en la elaboración del Trabajo de Fin de Grado al estudiante y que este llegue a realizar un Mínimo Producto Viable (MPV)
Nivel de influencia	Alta

Tabla 2.5: Detalles del Interesado - Manuel Jesús

Ficha interesado	
ID	STK-003
Apellidos, Nombre	Oprescu Popescu, Andreea Madalina
Cargo	Cotutora del Trabajo de Fin de Grado
Organización	Departamento de Tecnología Electrónica
Intereses	Guiar en la elaboración del Trabajo de Fin de Grado al estudiante y que este llegue a realizar un Mínimo Producto Viable (MPV)
Nivel de influencia	Alta

Tabla 2.6: Detalles del Interesado - Andreea

Ficha grupo interesados	
ID	STK-004
Nombre del grupo	Mesa tribunal
Número de integrantes	2 miembros
Organización	Escuela Técnica Superior de Ingeniería Informática
Intereses	Recibir un proyecto software y una memoria sobre el trabajo que sea de su agrado
Nivel de influencia	Alta

Tabla 2.7: Detalles del Grupo de Interesados - Mesa Tribunal

2.2.2. Clasificación de interesados

Tras identificar y analizar a los interesados del proyecto a desarrollar es posible introducir una clasificación de los mismos por medio de una matriz de interés/poder. La matriz se muestra en la figura 2.1 y se compone de cuatro cuadrantes bien diferenciados:

- **Monitorear:** indica para aquellos usuarios que se encuentren en esta celda que para el proyecto poseen bajo poder e interés.
- **Mantener informado:** que un interesado se encuentre en este cuadrante indica que tiene bajo poder pero gran interés en el proyecto.
- **Mantener satisfecho:** un interesado bajo este cuadrante quiere indicar que tiene alto poder para el proyecto pero bajo interés en el.
- **Gestionar atentamente:** aquellos interesados que se encuentren en este cuadrante serán los que más poder e interés aporten al proyecto.

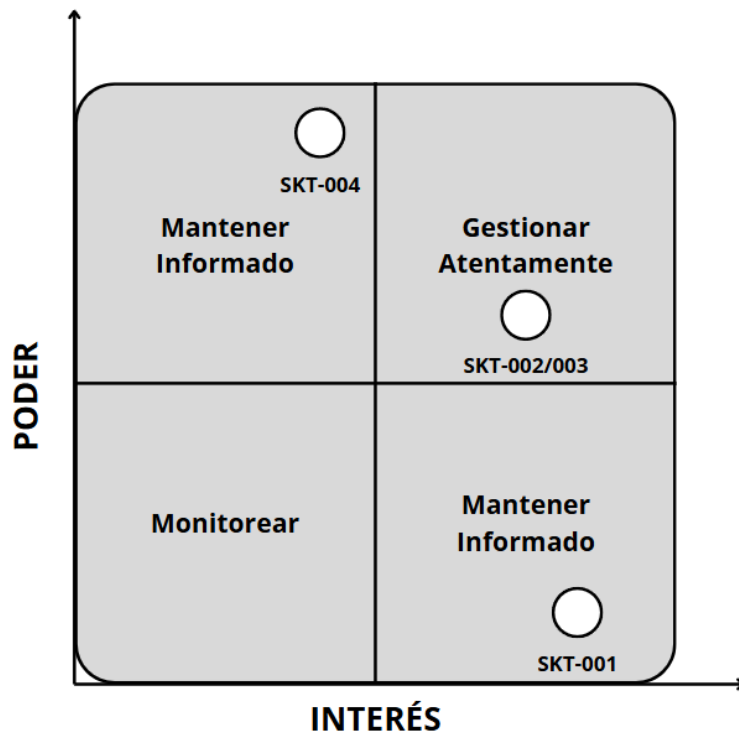


Figura 2.1: Matriz Interés / Poder de los interesados

2.2.3. Usuarios del sistema

Otro de los factores clave para el sistema a desarrollar es identificar a los distintos tipos de usuario que habrán dentro del software, de cara a la elaboración del consecuente Plan de Gestión del Alcance. Se detectan los siguientes:

- **Administrador:** usuarios con total acceso en la aplicación que tendrán como función principal la gestión de los usuarios y del propio sistema.
- **Usuarios finales:** son aquellos que acceden al sistema con cualquier fin relativo a las funcionalidades de la aplicación. Estos podrían ser investigadores, agentes inmobiliarios, personas en situación de compra-venta de vivienda, etc.

2.3. Gestión del Alcance

En este apartado se incluyen todas las prácticas seguidas necesarias para enfocar de forma primordial la definición y el control de qué se incluye y qué queda descartado en el alcance del proyecto.

Esto incluye la recopilación de los requisitos del proyecto, la creación de la Estructura de Desglose de Trabajo (EDT) con su consecuente diccionario, qué contiene y cómo se cumple el alcance.

2.3.1. Recopilación de requisitos

Registrar y analizar las distintas necesidades de los interesados ha llevado a la obtención de los siguientes requisitos para el desarrollo del proyecto, los cuales han sido recogidos poniendo en práctica las Historias de Usuario.

Requisitos de Información

Los requisitos de información documentan toda necesidad asociada a datos con los que se trabaja en el proyecto e información que el sistema deberá procesar, almacenar o generar. Para el desarrollo del software se han identificado los requisitos descritos en las siguientes tablas:

RI-001: Datos vivienda	
Descripción	COMO Administrador del sistema QUIERO que los datos de las viviendas estén compuestos principalmente por el precio, comodidades de la vivienda, orientaciones, posición geográfica, su estado y antigüedad y el barrio al que pertenecen PARA trabajar con estos durante las predicciones.
Prioridad	Alta

Tabla 2.8: Requisito de Información - RI-001

RI-002: Datos barrios	
Descripción	COMO Administrador del sistema QUIERO que los datos de los barrios se almacenen con un nombre y un identificador PARA que se relacionen correctamente con las viviendas cuando estas se almacenen.
Prioridad	Alta

Tabla 2.9: Requisito de Información - RI-002

RI-003: Datos metro	
Descripción	COMO Administrador del sistema QUIERO que los datos de las bocas de metro de Valencia se almacenen con un nombre y sus coordenadas de latitud y longitud PARA conocer sus posiciones exactar y calcular distancias.
Prioridad	Alta

Tabla 2.10: Requisito de Información - RI-003

RI-004: Datos entrenamiento	
Descripción	COMO Administrador del sistema QUIERO que se almacenen los pesos y los hiperparámetros óptimos del entrenamiento PARA que los modelos no requieran de entrenarse de nuevo.
Prioridad	Alta

Tabla 2.11: Requisito de Información - RI-004

RI-005: Datos experimentación	
Descripción	COMO Investigador QUIERO que se registren los resultados de los modelos durante la experimentación, como puede ser, el modelo usado y sus métricas de evaluación PARA ver cómo de buenos son estos para el contexto del problema.
Prioridad	Alta

Tabla 2.12: Requisito de Información - RI-005

Requisitos Funcionales

Los requisitos funcionales están pensados para especificar qué deberá hacer el sistema para cumplir con las necesidades registradas en el proyecto. Se han reconocido los siguientes:

RF-006: Leer y almacenar datos	
Descripción	COMO Administrador QUIERO que el sistema reciba un fichero de los datos definidos en los requisitos información y sea capaz de leer y almacenarlos PARA hacer las consultas directamente desde la base de datos.
Prioridad	Alta

Tabla 2.13: Requisito Funcional - RF-006

RF-007: Preparar los datos	
Descripción	COMO Investigador QUIERO que el sistema sea capaz de preparar los datos y procesarlos PARA que sean manejables por los modelos de entrenamiento y predicción.
Prioridad	Alta

Tabla 2.14: Requisito Funcional - RF-007

RF-008: Generar grafos	
Descripción	COMO Investigador QUIERO que el sistema sea capaz de transformar los datos procesados en un objeto de tipo grafo PARA que puedan ser usados por los modelos de redes neuronales.
Prioridad	Alta

Tabla 2.15: Requisito Funcional - RF-008

RF-009: Visualizar grafos	
Descripción	COMO Investigador QUIERO que el sistema permita visualizar los grafos generados PARA ver de forma gráfica como se establecen las relaciones entre las viviendas según las aristas que se definan.
Prioridad	Alta

Tabla 2.16: Requisito Funcional - RF-009

RF-010: Entrenar algoritmos	
Descripción	COMO Investigador QUIERO modelos entrenados mediante grafos de viviendas PARA ver como trabajan.
Prioridad	Alta

Tabla 2.17: Requisito Funcional - RF-010

RF-011: Generar métricas	
Descripción	COMO Investigador QUIERO que tras entrenar un algoritmo, se genere como resultado las métricas de medición PARA comprobar su funcionamiento en el contexto del problema.
Prioridad	Alta

Tabla 2.18: Requisito Funcional - RF-011

RF-012: Predecir vivienda	
Descripción	COMO Usuario final QUIERO predecir el precio de una vivienda de Valencia registrando los datos asociados a la misma PARA conocer el precio que establecería el modelo.
Prioridad	Alta

Tabla 2.19: Requisito Funcional - RF-012

RF-013: Observar resultados	
Descripción	COMO Investigador QUIERO que tras predecir una vivienda, se puedan observar detalles de cómo se integra esta durante el entrenamiento PARA conocer con qué otros inmuebles ha tomado relación y cómo trabaja.
Prioridad	Alta

Tabla 2.20: Requisito Funcional - RF-013

Requisitos No Funcionales

Los requisitos no funcionales del proyecto definen especificaciones de cómo deberá comportarse el sistema a desarrollar. Se reconocen los siguientes:

RNF-014: Representar idioma	
Descripción	COMO Usuario QUIERO que el sistema a desarrollar se muestre en español PARA que sea accesible al público objetivo.
Prioridad	Alta

Tabla 2.21: Requisito No Funcional - RNF-014

RNF-015: Refactorizar código	
Descripción	COMO Administrador QUIERO que el código del sistema desarrollado esté refactorizado PARA que sea mantenible en el tiempo.
Prioridad	Media

Tabla 2.22: Requisito No Funcional - RNF-015

RNF-016: Realizar consultas rápidas	
Descripción	COMO Usuario QUIERO que el sistema tenga tiempos de respuesta rápidos al interactuar con la interfaz PARA realizar consultas más ágiles.
Prioridad	Media

Tabla 2.23: Requisito No Funcional - RNF-016

RNF-017: Realizar interfaz intuitiva	
Descripción	COMO Usuario QUIERO que la interfaz sea lo más intuitiva posible PARA que sea fácil de manejar y agradable al uso.
Prioridad	Media

Tabla 2.24: Requisito No Funcional - RNF-017

RNF-018: Realizar pruebas de funcionalidad	
Descripción	COMO Administrador QUIERO que toda funcionalidad implementada en el sistema tenga pruebas sobre las mismas PARA comprobar el correcto funcionamiento del mismo.
Prioridad	Baja

Tabla 2.25: Requisito No Funcional - RNF-018

2.3.2. Reglas de Negocio

Las reglas de negocio definen las directrices de cómo deberá actuar el sistema dentro de un contexto definido por la organización. Para el sistema a desarrollar, se reconocen los siguientes:

RN-019: Validar atributos	
Descripción	COMO Administrador QUIERO que los atributos clave de la vivienda, como pueden ser la ubicación de la vivienda, comodidades o datos del inmueble sean obligatorios PARA realizar predicciones correctas.
Prioridad	Alta

Tabla 2.26: Regla de Negocio - RN-019

RN-020: Restringir precios	
Descripción	COMO Administrador QUIERO que se establezca un límite mínimo y máximo del precio de la vivienda en la predicción PARA evitar mostrar precios muy dispares en caso de que se prediga mal.
Prioridad	Media

Tabla 2.27: Regla de Negocio - RN-020

RN-021: Restringir barrios	
Descripción	COMO Administrador QUIERO que se limite la predicción en barrios donde se posean pocos datos para PARA evitar una mala predicción.
Prioridad	Media

Tabla 2.28: Regla de Negocio - RN-021

RN-022: Restringir viviendas	
Descripción	COMO Administrador QUIERO que se limiten valores para algunas características de la vivienda PARA evitar predecir inmuebles que no tengan una coherencia, como por ejemplo, viviendas que poseen ascensor en un edificio de una sola planta.
Prioridad	Media

Tabla 2.29: Regla de Negocio - RN-022

2.3.3. Enunciado del Alcance

El presente Trabajo de Fin de Grado se desarrolla con la finalidad de obtener un sistema de predicción de precios de viviendas en Valencia, representando los datos como grafos, con el principal propósito de ofrecer a los usuarios estimaciones basadas en relaciones entre los datos históricos de la ciudad.

Para que el proyecto finalice con éxito, deberá cumplir los siguientes criterios de aceptación:

- El proyecto de software deberá tener implementado al menos, todo lo que se ha definido como Mínimo Producto Viable (MPV), detallado en los siguientes puntos:
 - Se deberá haber realizado un estudio de los datos y los modelos usados, probando distintas formas de generar el grafo y entrenarlo.
 - Una vez seleccionado el mejor par modelo-grafo para el contexto del problema, se deberá desarrollar una aplicación web que permita a un usuario registrar una nueva vivienda para consultar su precio mediante una predicción.
 - Tras realizar una predicción, el sistema deberá revelar información de cómo se obtiene el resultado, mostrando cómo se integra la vivienda en el grafo y cómo se comporta con otras.

- Toda la investigación e implementación realizada deberá quedar reflejado en una memoria, donde se justifique en cada etapa todo lo llevado a cabo.

2.3.4. EDT

La EDT planteada para el proyecto está estructurada por fases y establece la siguiente jerarquía:

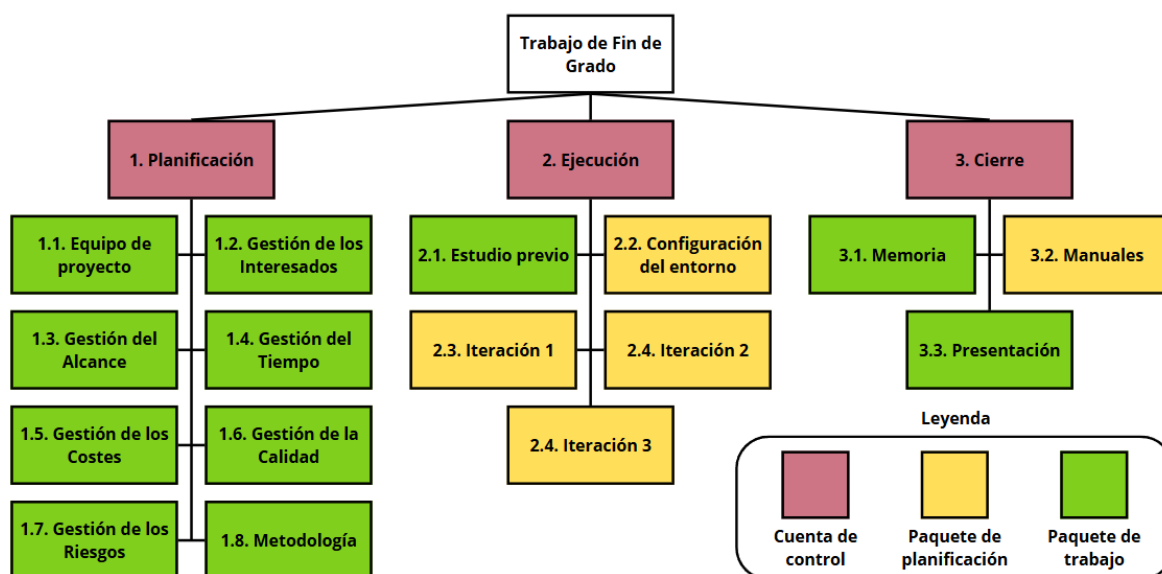


Figura 2.2: EDT del proyecto organizada por fases

2.3.5. Diccionario de la EDT

El siguiente apartado se dedica a desglosar las actividades asociadas a los paquetes de trabajo y de planificación de la EDT, cuyos detalles se definen en las siguientes tablas:

Paquete de trabajo - Equipo de proyecto	
ID	1.1.
Cuenta de control	1
Descripción	Se identifican a los miembros del equipo de proyecto
Entregables	Documentación relativa al equipo de proyecto
Actividades	■ 1.1.1. Definir los roles del equipo ■ 1.1.2. Identificar a los participantes del equipo de proyecto
Criterios de aceptación	■ Todos los roles del proyecto estarán descritos e identificados ■ Los miembros del equipo tendrán asignados los roles que cumplen en el proyecto

Tabla 2.30: Desglose Paquete de Trabajo - Equipo de proyecto

Paquete de trabajo - Gestión de Interesados	
ID	1.2.
Cuenta de control	1
Descripción	Documentación relativa a planificar y registrar a los interesados del proyecto
Entregables	Plan de Gestión de Interesados
Actividades	■ 1.2.1. Registrar a los interesados del proyecto ■ 1.2.2. Clasificar a los interesados según su poder/interés ■ 1.2.3. Definir los roles del sistema a desarrollar
Criterios de aceptación	■ Los principales interesados estarán registrados y se recogerán detalles de los mismos ■ El sistema tendrá ya definidos los roles principales

Tabla 2.31: Desglose Paquete de Trabajo - Gestión de Interesados

Paquete de trabajo - Gestión del Alcance	
ID	1.3.
Cuenta de control	1
Descripción	Documentación relativa a planificar el alcance junto a los requisitos y la EDT del proyecto
Entregables	Plan de Gestión del Alcance
Actividades	■ 1.3.1. Registrar y clasificar los requisitos del proyecto ■ 1.3.2. Definir el enunciado del alcance ■ 1.3.3. Crear la EDT del proyecto y su diccionario ■ 1.3.4. Validar el alcance
Criterios de aceptación	■ El proyecto tendrá asentado una línea base del alcance ■ La EDT y su diccionario desglosan y describen correctamente la jerarquía y los paquetes del proyecto

Tabla 2.32: Desglose Paquete de Trabajo - Gestión del Alcance

Paquete de trabajo - Gestión del Tiempo	
ID	1.4.
Cuenta de control	1
Descripción	Documentación relativa a planificar el tiempo junto al desglose de tareas y creación del cronograma
Entregables	Plan de Gestión del Tiempo
Actividades	■ 1.4.1. Identificar, secuenciar y establecer duración a las tareas del proyecto ■ 1.4.2. Identificar hitos del proyecto ■ 1.4.3. Crear el cronograma ■ 1.4.4. Planificar el control y seguimiento del cronograma
Criterios de aceptación	■ Se registrarán las tareas necesarias para cumplir al menos con lo que se enuncia en el alcance del proyecto ■ El cronograma se cumple dentro de los hitos establecidos

Tabla 2.33: Desglose Paquete de Trabajo - Gestión del Tiempo

Paquete de trabajo - Gestión de los Costos	
ID	1.5.
Cuenta de control	1
Descripción	Documentación relativa a planificar los costes del proyecto, creando un presupuesto
Entregables	Plan de Gestión de los Costes
Actividades	■ 1.5.1. Identificar los costes ■ 1.5.2. Establecer los costes ■ 1.5.3. Crear un presupuesto ■ 1.5.4. Planificar el control y seguimiento de los costes
Criterios de aceptación	■ Se tendrá un presupuesto con el que comparar para ver los costes planificados frente a los costes reales al finalizar el proyecto

Tabla 2.34: Desglose Paquete de Trabajo - Gestión de los Costos

Paquete de trabajo - Gestión de la Calidad	
ID	1.6.
Cuenta de control	1
Descripción	Documentación relativa a planificar la calidad del proyecto
Entregables	Plan de Gestión de la Calidad
Actividades	■ 1.6.1. Identificar herramientas de calidad ■ 1.6.2. Establecer técnicas de calidad ■ 1.6.3. Planificar aseguramiento de la calidad
Criterios de aceptación	■ Se deberá tener una constancia mínima de las bases de la calidad del proyecto

Tabla 2.35: Desglose Paquete de Trabajo - Gestión de la Calidad

Paquete de trabajo - Gestión de Riesgos	
ID	1.7.
Cuenta de control	1
Descripción	Documentación relativa a planificar los riesgos del proyecto
Entregables	Plan de Gestión de Riesgos
Actividades	■ 1.7.1. Identificar riesgos del proyecto ■ 1.7.2. Establecer planes de contingencia ■ 1.7.3. Planificar control y seguimiento de los riesgos
Criterios de aceptación	■ Deberán quedar registrados al menos todos los riesgos potenciales del proyecto con sus respectivos planes de contingencia

Tabla 2.36: Desglose Paquete de Trabajo - Gestión de Riesgos

Paquete de trabajo - Metodología	
ID	1.8.
Cuenta de control	1
Descripción	Documentación relativa a la metodología que se seguirá en el desarrollo
Entregables	Metodología del proyecto
Actividades	■ 1.8.1. Establecer metodologías de desarrollo ■ 1.8.2. Registrar tecnologías a usar ■ 1.8.3. Crear Plan de Gestión de la Configuración
Criterios de aceptación	■ Se deberá conocer la metodología a aplicar durante el desarrollo, así como las tecnologías principales que se utilizan

Tabla 2.37: Desglose Paquete de Trabajo - Metodología

Paquete de trabajo - Estudio previo	
ID	2.1.
Cuenta de control	2
Descripción	Aprendizaje y manejo con las herramientas para realizar el desarrollo software del proyecto
Entregables	Cursos completos
Actividades	■ 2.1.1. Realizar cursos introductorios de Python ■ 2.1.2. Leer artículo de introducción a GNN ■ 2.1.3. Realizar ejemplo práctico GNN ■ 2.1.4. Revisar tecnologías a usar ■ 2.1.5. Revisar datos a usar
Criterios de aceptación	■ Se tendrán que haber completado los cursos impuestos, además de adquirir los conocimientos mínimos de las tecnologías para comenzar a implementar código

Tabla 2.38: Desglose Paquete de Trabajo - Estudio previo

Paquete de planificación - Configuración del entorno	
ID	2.2.
Cuenta de control	2
Descripción	Engloba todo lo relacionado con la instalación y configuración del entorno de trabajo del proyecto
Entregables	Entorno de trabajo y GitHub
Actividades	■ 2.2.1. Instalar las tecnologías a usar en el proyecto ■ 2.2.2. Configurar el entorno de desarrollo ■ 2.2.3. Preparar el repositorio en GitHub [2] así como el Projects
Criterios de aceptación	■ Se deberá tener un entorno de desarrollo con todas las tecnologías y herramientas necesarias para comenzar la implementación del código, así como un repositorio en GitHub organizado y listo para almacenar el código

Tabla 2.39: Desglose Paquete de Trabajo - Configuración del entorno

Paquete de planificación - Iteración 1	
ID	2.3.
Cuenta de control	2
Descripción	Recoge toda actividad a realizar durante la primera iteración del proyecto
Entregables	Iteración funcional del software
Actividades	■ 2.3.1. Procesar los datos de la primera iteración ■ 2.3.2. Entrenar modelos de la primera iteración ■ 2.3.3. Validar modelos de la primera iteración ■ 2.3.4. Generar resultados de la primera iteración ■ 2.3.5. Revisar resultados de la primera iteración
Criterios de aceptación	■ Se deberá haber realizado una experimentación completa siguiendo lo que se pide en el enunciado del alcance, esto es, transformar los datos en grafo, entrenar a los modelos con el mismo y establecer métricas de medición para probar su funcionamiento

Tabla 2.40: Desglose Paquete de Planificación - Iteración 1

Paquete de planificación - Iteración 2	
ID	2.4.
Cuenta de control	2
Descripción	Recoge toda actividad a realizar durante la segunda iteración del proyecto
Entregables	Iteración funcional del software
Actividades	■ 2.4.1. Procesar los datos de la segunda iteración ■ 2.4.2. Entrenar modelos de la segunda iteración ■ 2.4.3. Validar modelos de la segunda iteración ■ 2.4.4. Generar resultados de la segunda iteración ■ 2.4.5. Revisar resultados de la segunda iteración
Criterios de aceptación	■ Se deberá haber realizado una experimentación completa siguiendo lo que se pide en el enunciado del alcance, esto es, transformar los datos en grafo, entrenar a los modelos con el mismo y establecer métricas de medición para probar su funcionamiento

Tabla 2.41: Desglose Paquete de Planificación - Iteración 2

Paquete de planificación - Iteración 3	
ID	2.5.
Cuenta de control	2
Descripción	Recoge toda actividad a realizar durante la tercera iteración del proyecto
Entregables	Aplicación software
Actividades	■ 2.5.1. Registrar y almacenar datos ■ 2.5.2. Consultar precio de una vivienda ■ 2.5.3. Visualizar resultados ■ 2.5.4. Realizar una interfaz intuitiva
Criterios de aceptación	■ Se deberá tener una interfaz del software funcional e intuitiva que incluya, al menos, todo lo especificado en el enunciado del alcance como Mínimo Producto Viable

Tabla 2.42: Desglose Paquete de Planificación - Iteración 3

Paquete de trabajo - Memoria	
ID	3.1.
Cuenta de control	3
Descripción	Engloba tareas relacionadas con la redacción de la memoria del Trabajo de Fin de Grado
Entregables	Memoria del proyecto
Actividades	■ 3.1.1. Redactar capítulo I: Introducción ■ 3.1.2. Redactar capítulo II: Planificación ■ 3.1.3. Redactar capítulo III: Ejecución ■ 3.1.4. Redactar capítulo IV: Cierre ■ 3.1.5. Redactar capítulo V: Anexos
Criterios de aceptación	■ La memoria contendrá todo el contenido especificado en el enunciado del alcance ■ El contenido deberá estar bien estructurado, invitando a una lectura clara y cohesiva ■ No deberá superar las 200 páginas

Tabla 2.43: Desglose Paquete de Trabajo - Memoria

Paquete de planificación - Manuales	
ID	3.2.
Cuenta de control	3
Descripción	Engloba tareas relacionadas con la redacción de manuales de usuario que irán contenidos en la memoria
Entregables	Manuales de usuario
Actividades	■ 3.2.1. Redactar manuales ■ 3.2.2. Revisar manuales
Criterios de aceptación	■ La memoria tendrá contendrá todos los manuales elaborados ■ Los manuales deberán ser claros y estar bien estructurados para que puedan guiar al usuario a finalizarlos con éxito

Tabla 2.44: Desglose Paquete de Planificación - Manuales

Paquete de trabajo - Presentación	
ID	3.3.
Cuenta de control	3
Descripción	Engloba tareas relacionadas con la presentación final del proyecto frente a la mesa tribunal
Entregables	Diapositivas del Trabajo de Fin de Grado
Actividades	■ 3.3.1. Elaborar diapositivas ■ 3.3.2. Ensayar la presentación ■ 3.3.3. Realizar la presentación
Criterios de aceptación	■ La presentación deberá incluir una demostración del software implementado para el proyecto ■ La presentación deberá satisfacer a la mesa tribunal presente durante la defensa

Tabla 2.45: Desglose Paquete de Trabajo - Presentación

2.4. Gestión del Tiempo

Planificar la Gestión del Tiempo resulta crucial para la elaboración del Trabajo de Fin de Grado, ya que esta etapa se centra en definir, estimar y secuenciar las actividades necesarias para completar el proyecto en el tiempo establecido, así como la elaboración del consecuente cronograma y una lista de hitos.

2.4.1. Hitos del proyecto

A continuación, se exponen los hitos más importantes del proyecto, así como algunos detalles relativos a los mismos, en el cuadro [2.46](#):

ID	Nombre	Fecha inicio	Fecha finalización
HT-001	Planificación	27/01/2025	23/02/2025
HT-002	Estudio previo	24/02/2025	02/03/2025
HT-003	Configuración del entorno	03/03/2025	09/03/2025
HT-004	Iteración 1	10/03/2025	23/03/2025
HT-005	Iteración 2	24/03/2025	06/04/2025
HT-006	Iteración 3	07/04/2025	20/04/2025
HT-007	Memoria y manuales	21/04/2025	18/05/2025
HT-008	Presentación	19/05/2025	13/06/2025

Tabla 2.46: Lista de hitos del Trabajo de Fin de Grado

2.4.2. Cronograma

En los siguientes cuadros se refleja la secuenciación de las tareas del proyecto, representadas por mediante un diagrama de Gantt elaborado con la herramienta de Microsoft Excel [\[3\]](#):

		Semana 1	Semana 2	Semana 3	Semana 4
Equipo de proyecto	1.1.1.				
	1.1.2.				
Gestión de Interesados	1.2.1.				
	1.2.2.				
	1.2.3.				
Gestión del Alcance	1.3.1.				
	1.3.2.				
	1.3.3.				
Gestión del Tiempo	1.4.1.				
	1.4.2.				
	1.4.3.				
	1.4.4.				
Gestión de los Costos	1.5.1.				
	1.5.2.				
	1.5.3.				
	1.5.4.				
Gestión de la Calidad	1.6.1.				
	1.6.2.				
	1.6.3.				
Gestión de los Riesgos	1.7.1.				
	1.7.2.				
	1.7.3.				
Metodología	1.8.1.				
	1.8.2.				
	1.8.3.				

Figura 2.3: Cronograma - Paquete planificación

		Semana 5	Semana 6	Semana 7	Semana 8
Estudio previo	2.1.1.				
	2.1.2.				
	2.1.3.				
	2.1.4.				
	2.1.5.				
Configuración del entorno	2.2.1.				
	2.2.2.				
	2.2.3.				
Iteración 1	2.3.1.				
	2.3.2.				
	2.3.3.				
	2.3.4.				
	2.3.5.				

Figura 2.4: Cronograma - Paquetes de estudio previo, configuración y primera iteración

		Semana 9	Semana 10	Semana 11	Semana 12
Iteración 2	2.4.1.				
	2.4.2.				
	2.4.3.				
	2.4.4.				
	2.4.5.				
Iteración 3	2.5.1.				
	2.5.2.				
	2.5.3.				
	2.5.4.				

Figura 2.5: Cronograma - Paquetes de la segunda y la tercera iteración

		Semana 13	Semana 14	Semana 15	Semana 16
Memoria	3.1.1.				
	3.1.2.				
	3.1.3.				
	3.1.4.				
	3.1.5.				
	3.1.6.				
Manuales	3.2.1.				
	3.2.2.				

Figura 2.6: Cronograma - Paquetes memoria y manuales

		Semana 17	Semana 18	Semana 19	Semana 20
Presentación	3.3.1.				
	3.3.2.				
	3.3.3.				

Figura 2.7: Cronograma - Paquete presentación

2.4.3. Control del cronograma

Para mantener el proyecto controlado en términos de tiempos, durante el desarrollo del mismo se realizan seguimientos semanales del cronograma, dedicando una columna del mismo para marcar los tiempos de esfuerzo reales que se tardan en realizar una tarea. Esto se controla mediante la herramienta de Clockify [4], especificando el nombre de la tarea y la cuenta de control a la que pertenece.

Además, se añadirá una columna adicional que indicará en qué porcentaje se ha completado una tarea en específico. Así no solo se tendrá control del flujo de las tareas en el cronograma, sino que también se podrá visualizar cómo se completan las tareas individualmente.

Es de real importancia indicar estos dos parámetros en el cronograma, ya que resultarán útiles para la Gestión de los Costos y la Gestión de los Riesgos en tareas como presupuestos o la activación de riesgos relacionados con el tiempo.

2.5. Gestión de los Costos

Esta etapa se centra en gestionar todo lo relacionado con temas de costes del proyecto, identificando y clasificando los mismos, además de asentar la línea base y registrar los costes reales a medida que avanza el proyecto.

2.5.1. Planificación de los Costos

Antes de desglosar los costes del proyecto, es fundamental establecer y explicar algunos parámetros usados para realizar toda esta gestión.

- **Unidades de medida:** se registran las siguientes:
 - **Medidas de tiempo:** se mide en horas de trabajo realizadas por personal individual. Las horas serán pagadas completas, es decir, si un miembro del equipo trabajase dos horas y media, el redondeo sería ascendente y por tanto se contaría como tres horas de trabajo.
 - **Medidas de cantidad:** se mide en número de material usado o consumo, dependiendo del coste a representar.
- **Nivel de precisión:** los costes en el proyecto no serán redondeados y además serán representados con dos decimales.

Además, cualquier coste planificado del proyecto será recogido usando técnicas de estimación análoga, contemplando valores y medidas de proyectos similares. Para este caso, se han tomado como referencia otros Trabajos de Fin de Grado similares realizados por antiguos alumnos de la Universidad de Sevilla, así como proyectos previos relacionados en los cuales participaba el Jefe de Proyecto.

2.5.2. Desglose de costos

La sección recoge todos los costes identificados en el proyecto, registrados y clasificados en tres tipos mediante tablas.

Costes directos

Dentro de los costes directos, se incluyen aquellos que han sido asignados explícitamente al proyecto, como pueden ser los distintos sueldos de los miembros del equipo según el rol que ocupen.

Para obtener información sobre los sueldos según el puesto que ocupasen en el proyecto, se ha consultado a la plataforma Talent [\[5\]](#), página web que incluye una funcionalidad por la cual se pueden obtener los sueldos medios a través de ofertas activas de empleo para ese puesto.

Es en este punto cuando se puede dar paso a mostrar los detalles de los costes directos, por medio del cuadro [2.47](#):

Nombre coste	Leyenda	Coste estimado
Jefe de proyecto	JP	34,36 €/h
Personal de dirección	DIR	26,67 €/h
Analista	ANA	12,54 €/h
Especialista de calidad	EQ	16,67 €/h
Documentador técnico	DT	13,21 €/h
Ingeniero de Datos	ID	15,38 €/h
Científico de Datos	CD	17,62 €/h
Desarrollador Backend	DB	17,18 €/h
Desarrollador Frontend	DF	17,95 €/h
Tester	TES	14,10 €/h

Tabla 2.47: Lista de costes directos del proyecto

Costes indirectos

Se refiere a aquellos costes del proyecto que no pueden ser asignados directamente al mismo, como son el consumo energético del equipo de trabajo o la tarifa de Internet.

Para calcular el gasto energético que produce el proyecto de la forma más precisa posible, se han tenido en cuenta los siguientes factores:

- **Precio de la luz en España:** con datos obtenidos a través de Selectra [6], página que ofrece una sección de consulta del precio de la luz por horas del día, es posible realizar un cálculo más exacto del consumo energético. A través de estos datos, se realiza una media de las horas del día en las que se estaría realizando alguna tarea del trabajo, excluyendo así el intervalo nocturno entre las 0:00 h y 6:00 h.
- **Consumo del equipo:** según un artículo de Repsol [7], el consumo medio de un ordenador portátil ronda los 2 kWh.

Conocidas las condiciones del consumo energético del proyecto, se puede dar paso a la introducción de los costes indirectos del mismo, desglosados en detalle en el cuadro 2.48:

Nombre coste	Coste estimado
Tarifa Internet	29,99 €/mes
Consumo energético	0,34 € Kw/h

Tabla 2.48: Lista de costes indirectos del proyecto

Costes materiales

Se recogen aquellos costes pertenecientes a bienes físicos utilizados para el desarrollo del proyecto, desglosados en el cuadro 2.49:

Nombre coste	Coste estimado
Licencia Overleaf	8,00 €/mes
Impresión memoria	0,05€/página
Encuadernación memoria	4,00€
Guía del PMBOK	221,82€
Ordenador portátil	1250,00€

Tabla 2.49: Lista de costes materiales del proyecto

2.5.3. Presupuesto planificado

Conocidos y registrados todos los costes identificados en el proyecto, se puede dar paso a preparar un presupuesto del mismo.

A continuación, se desglosan los costes por tareas, así como costes materiales, membresías, consumo energético y reservas con el fin de preparar el presupuesto final del proyecto mostrado, en detalle en los cuadros 2.50, 2.51 y 2.52:

Desglose de tareas				
EDT	Nombre tarea	Roles implicados	Esfuerzo	Coste
1.1.1.	Definir roles del equipo	■ JP	1 h	34,36 €
1.1.2.	Identificar participantes del proyecto	■ JP	1 h	34,36 €
1.2.1.	Registrar interesados	■ DIR	2 h	53,34 €
1.2.2.	Clasificar interesados	■ DIR	1 h	26,67 €
1.2.3.	Definir roles del sistema	■ DIR	1 h	26,67 €
1.3.1.	Registrar y clasificar requisitos	■ ANA	3 h	37,62 €
1.3.2.	Definir enunciado del Alcance	■ ANA	1 h	12,54 €
1.3.3.	Crear la EDT/WBS	■ ANA	3 h	37,62 €
1.4.1.	Identificar hitos	■ DIR	1 h	26,67 €
1.4.2.	Identificar, secuenciar y establecer duración de las tareas	■ DIR	2 h	53,34 €
1.4.3.	Crear cronograma	■ DIR	8 h	213,36 €
1.4.4.	Planificar control y seguimiento del cronograma	■ DIR	1 h	26,67 €
1.5.1.	Identificar costes	■ DIR	2 h	53,34 €
1.5.2.	Establecer costes	■ DIR	2 h	53,34 €
1.5.3.	Crear presupuesto	■ DIR	3 h	80,01 €

EDT	Nombre tarea	Roles implicados	Esfuerzo planificado	Coste planificado
1.5.4.	Planificar control y seguimiento costes	▪ DIR	2 h	53,34 €
1.6.1.	Identificar herramientas	▪ EQ	2 h	33,34 €
1.6.2.	Establecer técnicas	▪ EQ	2 h	33,34 €
1.6.3.	Planificar aseguramiento de la Calidad	▪ EQ	1 h	16,67 €
1.7.1.	Identificar riesgos	▪ DIR	2 h	53,34 €
1.7.2.	Establecer planes de contingencia	▪ DIR	2 h	53,34 €
1.7.3.	Planificar control y seguimiento de riesgos	▪ DIR	1 h	26,67 €
1.8.1.	Establecer metodologías de desarrollo	▪ DIR	1 h	26,67 €
1.8.2.	Registrar tecnologías	▪ DIR	2 h	53,34 €
1.8.3.	Crear Plan de Gestión de la Configuración	▪ DIR	5 h	133,35 €
2.1.1.	Realizar cursos introductorios Python	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	5 h	411,15 €
2.1.2.	Leer artículo introducción GNN	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	5 h	411,15 €
2.1.3.	Realizar ejemplo práctico GNN	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	10 h	822,30 €
2.1.4.	Revisar tecnologías a usar	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	2,5 h	164,46 €
2.1.5.	Revisar datos a usar	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	2,5 h	164,46 €

EDT	Nombre tarea	Roles implicados	Esfuerzo planificado	Coste planificado
2.2.1.	Instalar tecnologías	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	8 h	657,84 €
2.2.2.	Configurar entorno de desarrollo	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	8 h	657,84 €
2.2.3.	Preparar repositorios de GitLab y el Projects	▪ DIR	4 h	106,68 €
2.3.1.	Procesar datos	▪ ID ▪ CD	15 h	495,00 €
2.3.2.	Entrenar modelos	▪ ID ▪ CD	10 h	330,00 €
2.3.3.	Validar modelos	▪ ID ▪ CD ▪ TES	4 h	188,40 €
2.3.4.	Generar resultados	▪ CD ▪ DB	3 h	93,84 €
2.3.5.	Revisar resultados	▪ ID ▪ CD ▪ DB ▪ TES	2 h	128,56 €
2.4.1.	Procesar datos	▪ ID ▪ CD	15 h	495,00 €
2.4.2.	Entrenar modelos	▪ ID ▪ CD	10 h	330,00 €
2.4.3.	Validar modelos	▪ ID ▪ CD ▪ TES	4 h	188,40 €
2.4.4.	Generar resultados	▪ CD ▪ DB	3 h	93,84 €
2.4.5.	Revisar resultados	▪ ID ▪ CD ▪ DB ▪ TES	2 h	128,56 €

EDT	Nombre tarea	Roles implicados	Esfuerzo planificado	Coste planificado
2.5.1.	Registrar y almacenar usuarios	■ DB ■ TES	5 h	156,40 €
2.5.2.	Consultar precio de la vivienda	■ DB ■ TES	10 h	312,80 €
2.5.3.	Visualizar resultados	■ DB ■ TES	10 h	312,80 €
2.5.4.	Realizar interfaz	■ DF ■ TES	5 h	160,25 €
3.1.1.	Redactar capítulo Introducción	■ DT	2 h	26,42 €
3.1.2.	Redactar capítulo Planificación	■ DT	15 h	198,15 €
3.1.3.	Redactar capítulo Ejecución	■ DT	15 h	198,15 €
3.1.4.	Redactar capítulo Cierre	■ DT	15 h	198,15 €
3.1.5.	Redactar capítulo Anexos	■ DT	15 h	198,15 €
3.1.6.	Revisión memoria	■ EQ	8 h	100,32 €
3.2.1.	Redactar manuales	■ DT	20 h	264,20 €
3.2.2.	Revisar manuales	■ EQ	7 h	87,78 €
3.3.1.	Elaborar diapositivas	■ JP	5 h	171,80 €
3.3.2.	Ensayar presentación	■ JP	3 h	103,08 €
3.3.3.	Realizar presentación	■ JP	1 h	34,36 €
TOTAL			300 h	9.643,60 €

Tabla 2.50: Desglose detallado de tareas del proyecto

Costes indirectos			
Nombre coste	Precio consumo	Cantidad consumida	Cst. P
Tarifa Internet	29,99 € / mes	6 meses	179,94 €
Consumo energético	0,34 € Kw/h	300 h.	102 €
TOTAL			281,94 €

Tabla 2.51: Desglose de costes indirectos del proyecto

Costes materiales			
Nombre coste	Precio	Cantidad usada	Cst. P
Licencia Overleaf Pro	8 € / mes	6 meses	48,00 €
Guía del PMBOK	221,82 €	-	221,82 €
Ordenador portátil	1.250,00 €	-	1.250,00 €
Página memoria	0,05 €	200	10,00 €
Encuadernado memoria	4,00 €	1	4,00 €
TOTAL			1.533,82 €

Tabla 2.52: Desglose costes materiales del proyecto

Tras calcular el valor total de los costes según el tipo, se puede obtener un presupuesto final del proyecto, reflejado en el cuadro 2.53. A este se le añade un fondo de 2000,00€ de reserva, dedicado a planes de contingencia en caso de que se activen riesgos durante el proceso de desarrollo del proyecto.

Presupuesto planificado	
Grupo de costes	Coste total
Costes directos	9.643,60 €
Costes indirectos	281,94 €
Costes materiales	1.533,82 €
Contingencia	2.000,00 €
TOTAL	13.459,36 €

Tabla 2.53: Presupuesto planificado del proyecto

2.5.4. Presupuesto real

A continuación se exponen las tablas de los costes reales del proyecto por medio de los cuadros 2.54, 2.55 y 2.56:

Desglose de tareas				
EDT	Nombre tarea	Roles implicados	Esfuerzo	Coste
1.1.1.	Definir roles del equipo	■ JP	1 h	34,36 €
1.1.2.	Identificar participantes del proyecto	■ JP	2 h	68,72 €
1.2.1.	Registrar interesados	■ DIR	2 h	53,34 €
1.2.2.	Clasificar interesados	■ DIR	2 h	53,34 €
1.2.3.	Definir roles del sistema	■ DIR	2 h	80,01 €
1.3.1.	Registrar y clasificar requisitos	■ ANA	6 h	75,24 €
1.3.2.	Definir enunciado del Alcance	■ ANA	2 h	25,08 €

EDT	Nombre tarea	Roles implicados	Esfuerzo planificado	Coste planificado
1.3.3.	Crear la EDT/WBS	■ ANA	1 h	12,54 €
1.4.1.	Identificar hitos	■ DIR	2 h	53,34 €
1.4.2.	Identificar, secuenciar y establecer duración de las tareas	■ DIR	3 h	80,01 €
1.4.3.	Crear cronograma	■ DIR	15 h	400,05 €
1.4.4.	Planificar control y seguimiento del cronograma	■ DIR	1 h	26,67 €
1.5.1.	Identificar costes	■ DIR	8 h	213,36 €
1.5.2.	Establecer costes	■ DIR	2 h	53,34 €
1.5.3.	Crear presupuesto	■ DIR	4 h	106,68 €
1.5.4.	Planificar control y seguimiento costes	■ DIR	1 h	26,67 €
1.6.1.	Identificar herramientas	■ EQ	2 h	33,34 €
1.6.2.	Establecer técnicas	■ EQ	4 h	66,68 €
1.6.3.	Planificar aseguramiento de la Calidad	■ EQ	3 h	50,01 €
1.7.1.	Identificar riesgos	■ DIR	2 h	53,34 €
1.7.2.	Establecer planes de contingencia	■ DIR	2 h	53,34 €
1.7.3.	Planificar control y seguimiento de riesgos	■ DIR	2 h	53,34 €
1.8.1.	Establecer metodologías de desarrollo	■ DIR	1 h	26,67 €
1.8.2.	Registrar tecnologías	■ DIR	2 h	53,34 €
1.8.3.	Crear Plan de Gestión de la Configuración	■ DIR	4 h	106,68 €
2.1.1.	Realizar cursos introductorios Python	■ ID ■ CD ■ DB ■ DF ■ TES	5 h	411,15 €
2.1.2.	Leer artículo introducción GNN	■ ID ■ CD ■ DB ■ DF ■ TES	3 h	246,69 €

EDT	Nombre tarea	Roles implicados	Esfuerzo planificado	Coste planificado
2.1.3.	Realizar ejemplo práctico GNN	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	7 h	575,61 €
2.1.4.	Revisar tecnologías a usar	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	2 h	164,46 €
2.1.5.	Revisar datos a usar	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	3 h	246,69 €
2.2.1.	Instalar tecnologías	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	3 h	246,69 €
2.2.2.	Configurar entorno de desarrollo	▪ ID ▪ CD ▪ DB ▪ DF ▪ TES	3 h	246,69 €
2.2.3.	Preparar repositorios de GitLab y el Projects	▪ DIR	1 h	26,67 €
2.3.1.	Procesar datos	▪ ID ▪ CD	26 h	858,00 €
2.3.2.	Entrenar modelos	▪ ID ▪ CD	4 h	132,00 €
2.3.3.	Validar modelos	▪ ID ▪ CD ▪ TES	2 h	94,20 €
2.3.4.	Generar resultados	▪ CD ▪ DB	3 h	93,84 €
2.3.5.	Revisar resultados	▪ ID ▪ CD ▪ DB ▪ TES	2 h	128,56 €

EDT	Nombre tarea	Roles implicados	Esfuerzo planificado	Coste planificado
2.4.1.	Procesar datos	▪ ID ▪ CD	7 h	231,00 €
2.4.2.	Entrenar modelos	▪ ID ▪ CD	15 h	495,00 €
2.4.3.	Validar modelos	▪ ID ▪ CD ▪ TES	1 h	47,10 €
2.4.4.	Generar resultados	▪ CD ▪ DB	3 h	93,84 €
2.4.5.	Revisar resultados	▪ ID ▪ CD ▪ DB ▪ TES	3 h	192,84 €
2.5.1.	Registrar y almacenar usuarios	▪ DB ▪ TES	3 h	93,84 €
2.5.2.	Consultar precio de la vivienda	▪ DB ▪ TES	12 h	375,36 €
2.5.3.	Visualizar resultados	▪ DB ▪ TES	8 h	250,24 €
2.5.4.	Realizar interfaz	▪ DF ▪ TES	7 h	224,35 €
3.1.1.	Redactar capítulo Introducción	▪ DT	1 h	13,21 €
3.1.2.	Redactar capítulo Planificación	▪ DT	7 h	92,47 €
3.1.3.	Redactar capítulo Ejecución	▪ DT	24 h	317,04 €
3.1.4.	Redactar capítulo Cierre	▪ DT	11 h	145,31 €
3.1.5.	Redactar capítulo Anexos	▪ DT	1 h	13,21 €
3.1.6.	Revisión memoria	▪ EQ	15 h	188,10 €
3.2.1.	Redactar manuales	▪ DT	8 h	105,68 €
3.2.2.	Revisar manuales	▪ EQ	5 h	62,70 €
3.3.1.	Elaborar diapositivas	▪ JP	2 h	68,72 €
3.3.2.	Ensayar presentación	▪ JP	1 h	34,36 €
3.3.3.	Realizar presentación	▪ JP	1 h	34,36 €
TOTAL			276 h	8.409,47 €

Tabla 2.54: Desglose detallado de tareas del proyecto

Costes indirectos			
Nombre coste	Precio consumo	Cantidad consumida	Cst. P
Tarifa Internet	29,99 € / mes	7 meses	209,93 €
Consumo energético	0,34 € Kw/h	276 h.	93,84 €
TOTAL			303,77 €

Tabla 2.55: Desglose de costes indirectos reales del proyecto

Costes materiales			
Nombre coste	Precio	Cantidad usada	Cst. P
Licencia Overleaf Pro	8 € / mes	7 meses	56,00 €
Guía del PMBOK	221,82 €	-	221,82 €
Ordenador portátil	1.250,00 €	-	1.250,00 €
Página memoria	0,05 €	170	8,50 €
Encuadernado memoria	4,00 €	1	4,00 €
TOTAL			1.540,32 €

Tabla 2.56: Desglose costes materiales reales del proyecto

Tras calcular el valor real de los costes según el tipo se puede obtener el presupuesto real del proyecto, reflejado en el cuadro 2.57:

Presupuesto real	
Grupo de costes	Coste total
Costes directos	8.409,47 €
Costes indirectos	303,77 €
Costes materiales	1.540,32 €
TOTAL	10.253,56 €

Tabla 2.57: Presupuesto real del proyecto

Finalmente, el presupuesto real representa un ahorro del 23,82 % respecto al presupuesto inicialmente planificado. Esto se debe a que algunas tareas del proyecto que estaban estimadas con varias horas de dedicación se completaron más rápidamente, lo que indica que en las estimaciones de ciertas actividades no se calculó con precisión, aunque esta desviación fue favorable para reducir los costos del proyecto.

2.6. Gestión de la Calidad

Redactar un Plan de Gestión de la Calidad ayudará a garantizar que el proyecto cumpla con los estándares y procesos de calidad establecidos, consiguiendo de esta forma entregar en el cierre un producto software que no solo sea apto, sino que esté bien calificado.

Es por ello que esta sección está dedicada a definir las herramientas de trabajo que ayudarán a garantizar la calidad de los entregables y productos del proyecto, además de reflejar los procesos, técnicas y métricas que se usarán para hacer entrega de un Trabajo de Fin de Grado de calidad.

2.6.1. Herramientas

Tras investigar qué herramientas son más convenientes para el ámbito del proyecto, se han seleccionado las siguientes:

- **GitHub [2]:** plataforma destinada a gestionar el ciclo de vida de todo el desarrollo software, incluyendo herramientas para repositorios Git [8], integración continua y gestión de incidencias, entre otros. Será de gran utilidad como medio de comunicación con los representantes del cliente, ya que podrán ver el código alojado y cómo se completan las tareas a través del tablero de gestión.
- **Codacy [9]:** se trata de una herramienta de análisis de código que ayuda a mejorar la calidad del software programado mediante la detección automática de errores, vulnerabilidades, problemas de estilo o duplicación de código. Además, ofrece métricas y medidores que indican la calidad del software, así como porcentajes para la cobertura de pruebas del software, siendo compatible con GitHub [2].
- **Overleaf [10]:** aplicación web para la escritura y edición de documentos en formato LaTeX. Será de gran utilidad para la redacción de la memoria, ya que ofrece un estándar y formatos de calidad al nivel de exigencia del proyecto.

2.6.2. Procesos y técnicas de calidad

Se definen los siguientes procesos y técnicas para el cumplimiento de la calidad:

- **Revisión de documentos:** se harán continuas revisiones tanto de la memoria como cualquier otro tipo de documento ligado a algún entregable del proyecto. Esto se hará en colaboración con los representantes del cliente, quienes serán también encargados de revisar el documento para detectar incoherencias de estructura, que sean legibles y que refleje todo el contenido que se demanda, además de posibles faltas ortográficas.
- **Gestión de tareas e incidencias:** GitHub [2] ofrece una sección para proyectos, donde es posible llevar a cabo una gestión de las tareas relacionadas con la

etapa de ejecución del proyecto, es decir, toda tarea que tenga que ver con el software del producto. Es por ello que se definen todas estas tareas por medio de un tablero de gestión, además de los *milestones* del proyecto. Esto ayudará a controlar el avance del proyecto y a tener constancia de los cambios dentro del software, ya que todo movimiento quedará reflejado.

- **Control de versiones:** se establecerá una política de ramas y unos estándares a seguir, así como la protección de algunas ramas para garantizar que el software cumple con estándares de calidad para ser entregado.
- **Automatización Codacy [9]:** junto a GitHub [2], la herramienta puede ser configurada de tal forma que al implementar nuevo código al proyecto, se puedan identificar fallas en el software, con la intención de que sean corregidas para que el software sea de la máxima calidad posible.

2.6.3. Métricas de calidad

Ofrecer métricas de calidad a las herramientas y procesos aplicados durante el proyecto ayudará a conocer en qué grado estamos mejorando la calidad de los entregables y productos.

Para el software a desarrollar, se aplican las siguientes métricas:

- **Panel de Codacy:** la herramienta ofrece un panel de información sobre el estado del código elaborado. Para el contexto del problema, son importantes los siguientes elementos y estadísticas:
 - **Grado de calidad del repositorio:** Codacy se encarga de asignar una nota global a cada una de las ramas que tenga el repositorio que aloja el código, que representa una media ponderada del grado de cumplimiento de las métricas (problemas en el código, duplicados, cobertura de pruebas, complejidad, etc.). La calificación está entre A y F, como se puede observar en la figura 2.8, con A representando el nivel máximo de calidad del código y F el nivel más bajo.

Esta calificación se tendrá en cuenta para todas las ramas del repositorio, pero sobre todo para la rama principal del proyecto. Además, para que el código de la rama sea considerado de calidad, deberá obtener una calificación igual o superior a B. De lo contrario, no superará el umbral establecido y no podrá ser integrado en la rama principal.

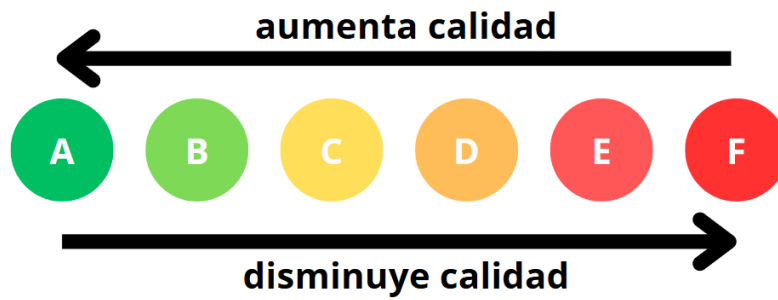


Figura 2.8: Rango de calificación de Codacy

- **Cobertura de pruebas:** el área de cobertura en el panel muestra en qué porcentaje están cubiertas las líneas del código con pruebas. Tener un alto porcentaje de cobertura será un gran indicativo de que estamos entregando un software de calidad, por lo que la rama principal deberá tener al finalizar la aplicación una cobertura igual o superior al 70 %.
- **Documentación:** junto a la aplicación software a desarrollar, también se redactarán documentos alusivos a la misma, como es la memoria o los posibles manuales de usuario y anexos. Es por ello que todos estos documentos deberán tener métricas de calidad para la entrega durante las revisiones periódicas en la etapa de cierre. Para que un documento sea considerado de calidad, deberá tener entonces el menor número de errores posibles, además de recibir el apto por parte de los representantes, lo cual indicará que la documentación entregada está al nivel de detalle requerido.

2.6.4. Control de calidad

Para asegurar que la calidad se mantenga a lo largo del proyecto, un especialista de la calidad tomará la responsabilidad de comprobar semanalmente que los productos y entregables alcanzan el grado de calidad establecido para cada una de las métricas. En caso necesario, se tomarán nuevas medidas y se replanificará el trabajo.

2.7. Gestión de los Riesgos

Planificar la Gestión de los Riesgos tiene como objetivo identificar los riesgos que puedan surgir durante la elaboración del proyecto y que afectan negativamente al éxito del mismo. Por ello, además de registrar los riesgos, se evaluará su impacto y se elaborarán planes de contingencia para mitigar en caso de que se activen.

Para crear un Plan de Gestión de los Riesgos, se usarán técnicas de estimación análoga, tomando como referencia otros Trabajos de Fin de Grado y proyectos realizados por el estudiante.

Antes de comenzar a registrar los riesgos y elaborar los consecuentes planes de contingencia, es importante definir una serie de conceptos previos:

- **Impacto:** es el grado de consecuencia que puede tener un riesgo sobre el proyecto si este se activa. Será evaluado para el alcance, los costes y el tiempo con una nota entre 1 y 5, siendo 1 un impacto bajo y 5 un alto impacto. El impacto final del riesgo se calculará como una media ponderada entre estos tres valores mencionados.
- **Probabilidad:** se trata de la posibilidad de que un riesgo del proyecto se active. La probabilidad será evaluada entre 1 y 5, siendo 1 poco probable y 5 muy probable de que ocurra.
- **Prioridad:** se trata del nivel de urgencia con el que debe ser tratado un riesgo activo en el proyecto. El grado de Prioridad del Riesgo NPR, se calcula por medio de la fórmula 2.1:

$$\text{NPR} = \text{Probabilidad} \times \text{Impacto} \quad (2.1)$$

En el contexto del problema encontramos tres grados distintos de prioridad:

- **Alto:** indica que el riesgo deberá ser tratado con total prioridad frente a otros porque afecta gravemente al éxito del proyecto. Un riesgo se considerará de prioridad alta si su NPR se encuentra comprendido entre 17 y 25, ambos incluidos.
- **Medio:** indica que el riesgo no requiere una atención inmediata, pero deberá ser tratado rápidamente. Un riesgo se considerará de prioridad media si su NPR se encuentra comprendido entre 10 y 16, ambos incluidos.
- **Bajo:** indica que el riesgo no se compromete demasiado con el éxito del proyecto, y por tanto no es urgente su mitigación. Un riesgo se considerará de prioridad baja si su NPR se encuentra comprendido entre 0 y 9, ambos incluidos.

2.7.1. Registro de Riesgos

Conocidos los atributos y pautas a seguir en el Plan de Gestión de los Riesgos, se puede dar paso a indagar en los detalles de los riesgos identificados para el proyecto en el cuadro 2.58:

Listado de riesgos							
ID	Riesgo	A	C	T	Prob.	NPR	Prio.
RSK-001	Alcance mal definido	5	2	2	2	6	Baja
RSK-002	Fallos en el software	3	3	4	3	10	Media
RSK-003	Fallos en el hardware	3	3	4	1	4	Baja
RSK-004	Falta de conocimientos por parte del equipo	1	4	5	5	17	Alta

ID	Riesgo	A	C	T	Prob.	NPR	Prio.
RSK-005	Cambios en las tecnologías usadas	1	2	3	1	2	Baja
RSK-006	Fallos en la configuración del entorno	1	2	4	3	7	Baja
RSK-007	El proyecto no puede finalizar en las fechas previstas	2	4	5	5	19	Alta
RSK-008	Cambios en los requisitos	5	3	3	3	11	Media
RSK-009	Problemas de comunicación con los representantes	2	2	5	1	3	Baja
RSK-010	Pérdida de documentos	4	3	3	1	4	Baja

Tabla 2.58: Listado de riesgos del proyecto

2.7.2. Planes de Contingencia

Las acciones diseñadas en control y respuesta a posibles activaciones de riesgos se detallan en el cuadro 2.59:

Planes de contingencia	
Plan	Riesgos ligados
Volver a realizar una revisión de los requisitos con los interesados del proyecto hasta tener más claro los requisitos	RSK-001, RSK-008
Realizar revisiones de código con frecuencia	RSK-002
Preparar todo el entorno de desarrollo en un nuevo equipo de trabajo	RSK-003
Buscar nuevos tutoriales o ejemplos alusivos al proyecto en el que se está trabajando, además de pedir ayuda a los representantes	RSK-002, RSK-004, RSK-005, RSK-006
Evaluar el impacto en el proyecto, y diseñar arquitecturas modulares para facilitar cambios en este aspecto	RSK-005, RSK-002
Redefinir hitos y cronograma, extendiendo el tiempo de duración del proyecto a la segunda convocatoria	RSK-007
Establecer reuniones más periódicas y bien planeadas para que sean más efectivas	RSK-009
Tener un historial de cambio para retornar a algún punto anterior del software en caso de ser necesario	RSK-002

Tabla 2.59: Listado de planes de contingencia para los riesgos

2.7.3. Monitoreo y control de riesgos

En cuanto al control y seguimiento de los riesgos, se realizará semanalmente, de la misma forma que en la gestión del cronograma y de los costes, evaluando qué riesgos tienen posibilidad de activarse para tratarlos con rapidez.

Este rol estará cubierto por el personal de dirección del proyecto, al ser quienes conocen en profundidad el manejo en estos casos.

2.8. Gestión de la Comunicación

Establecer un Plan para la Gestión de la Comunicación garantizará que la información relativa al proyecto fluya correctamente, de modo que éste pueda avanzar sin problemas.

Es por ello que en esta sección se comentan los procesos y herramientas a poner en práctica para una buena comunicación con el cliente.

2.8.1. Estrategia de comunicación

Las reuniones principales se harán al finalizar una etapa del proyecto, con el principal objetivo de que ambos representantes del cliente conozcan el estado del mismo, evaluando su viabilidad y sugiriendo cambios y sugerencias de mejora para la entrega final en el cierre. También se podrán celebrar reuniones para consulta de dudas o aportar soluciones al proyecto en el caso de que fuesen necesarias.

Las reuniones se realizarán de forma presencial o telemática; en este último caso, el canal de comunicación será Microsoft Teams [11]. Se acuerdan vía correo electrónico para establecer la fecha y la hora, así como para confirmar las asistencias.

Para tener constancia de todo lo hablado durante las reuniones, se levantará un Acta de Reunión, que incluirá datos como el motivo de la reunión, duración de la misma, asistentes, puntos tratados y acuerdos encontrados. Estas actas estarán recogidas por medios de cuadros en un anexo de la memoria del proyecto.

2.9. Metodología

La siguiente sección está dedicada a conocer las metodologías que se llevarán a cabo durante el desarrollo del Trabajo de Fin de Grado, así como estándares que se seguirán y las herramientas y tecnologías que se utilizarán.

2.9.1. Flujo de trabajo

El desarrollo del proyecto será llevado a cabo aplicando metodologías ágiles, centradas en las que dan un enfoque iterativo e incremental. Esto permite una mayor adaptabilidad a posibles cambios en los requisitos, además de la entrega continua de un valor incremental a los interesados.

En un principio, el proyecto contará con tres iteraciones para el desarrollo del software, de duración fija (dos semanas por iteración). En caso de que el proyecto no pudiese completarse en el plazo establecido para la primera convocatoria, se podrá extender el número de iteraciones, en función del tiempo y el trabajo que quede en el momento en el que se decide la replanificación.

2.9.2. Tecnologías

El entorno de desarrollo para la implementación del código del proyecto está compuesto por las siguientes tecnologías:

- **Visual Studio Code [12]:** es el entorno de desarrollo integrado del proyecto, elegido por su capacidad de soportar distintos lenguajes de programación, entre ellos, los que se usarán para la implementación. Además, posee un Marketplace con múltiples extensiones para ampliar su funcionalidad y es compatible con Git [8], lo que resulta de utilidad para el control de versiones con el repositorio en la nube.
- **Python [13]:** es el lenguaje de programación usado para el desarrollo del proyecto. Se trata de un lenguaje de alto nivel y multipropósito diseñado para ser fácil de leer, escribir y mantener. Además contiene librerías que serán de gran utilidad a la hora de trabajar con el tratamiento de los datos y el entrenamiento de los modelos. Algunas de estas bibliotecas son las siguientes:
 - **Paquetes de manipulación y análisis de datos:** se tratan de aquellas bibliotecas que ayudarán a manejar la carga de trabajo con grandes volúmenes de datos, dando carta libre a establecer distintos formatos y estructuras. Estos serán esenciales para leer y transformar los datos. Para la experimentación y el desarrollo del sistema, se usarán los siguientes:
 - **numpy [14]:** paquete fundamental para realizar cálculos numéricos. Proporciona arrays multidimensionales y funciones matemáticas de alto rendimiento que resultan eficientes para la investigación.
 - **pandas [15]:** facilita la manipulación de datos, ofreciendo un formato de tablas conocido como DataFrames. Resulta ideal para el análisis, limpieza y transformación de los datos.
 - **geopandas [16]:** se trata de una extensión de la biblioteca de pandas usado para el tratamiento de datos geoespaciales. Ofrece funciones útiles para manipular archivos shapefile o calcular distancias geográficas, entre otros.

- **shapely** [17]: relacionado con el paquete anterior, este se centra también en la manipulación y análisis de datos geométricos como puntos, líneas o polígonos.
- **matplotlib** [18]: permite crear representaciones gráficas, útil para crear gráficos, mapas o histogramas.
- **scikit-learn** [19]: ofrece funciones útiles para tareas de aprendizaje automático, además de prestar herramientas para la validación y el preprocesamiento de datos.
- **Redes neuronales y aprendizaje automático:** se tratan de bibliotecas especializadas en la construcción, entrenamiento y evaluación de modelos que aprenden a partir de un conjunto de datos de entrenamiento. Estos paquetes resultan esenciales para el desarrollo de la predicción. Entre las bibliotecas que se usarán, se encuentran las siguientes:
 - **torch** [20]: se trata de la biblioteca principal de *PyTorch*, una plataforma para la construcción y entrenamiento de modelos de redes neuronales.
 - **torchvision** [20]: proporciona utilidades para tareas de visión por computador.
 - **torch_geometric** [20]: esta extensión de *PyTorch* ofrece funciones y métodos para Redes Neuronales de Grafos (GNN), proporcionando arquitecturas, datasets o herramientas específicas de aprendizaje en grafos, entre otros.
 - **optuna** [21]: framework de optimización automática de hiperparámetros de los modelos de machine learning. Resulta útil para mejorar el rendimiento de los modelos mediante una búsqueda eficiente.
- **Análisis de grafos:** se tratan de aquellos paquetes que han sido diseñados para representar, manipular y analizar estructuras de datos basadas en grafos. Para la investigación y el desarrollo de la aplicación web, se usará **networkx** [22], biblioteca útil para la creación y representación de grafos.
- **Django** [23]: se trata de un framework web compatible con el lenguaje de programación del proyecto, diseñado para facilitar el desarrollo de aplicaciones web. Está basado en el patrón MTV (*Model-Template-View*) y será de gran utilidad para la creación de los modelos del proyecto, así como la interfaz web de proyecto.
- **SQLite** [24]: se trata de la base de datos relacional que usa el framework de Django [23] por defecto, el cuál no requiere un servidor para su funcionamiento.

2.9.3. Gestión de tareas

La gestión de las tareas se realizará por medio de la herramienta GitHub [2]. Es por ello que esta sección está dedicada a explicar cómo se crearán las Issues, qué atributos tienen y cómo se organizan estas en un tablero Kanban.

El tablero se compone de cinco columnas bien diferenciadas como se puede observar en la figura 2.9, que servirán para indicar el estado en el que se encuentra cada tarea del proyecto. Los estados son los siguientes:

- **Product Backlog:** que una tarea se encuentre en esta columna, indica que la tarea no está empezada ni inicializada y está a la espera de ser gestionada para ser desarrollada en alguna de las iteraciones del proyecto. Las tareas de esta columna, estarán ordenadas en función de la prioridad establecida por los requisitos y serán desarrolladas según como se indique en la planificación.
- **Tareas iteración:** estado que indica qué tareas se estarán desarrollando durante la iteración en la que se encuentre el proyecto. En este punto, las tareas están inicializadas y listas para que puedan empezar a ser desarrolladas. De la misma forma que para el Product Backlog, estarán ordenadas en función de la prioridad en la que se estén clasificadas.
- **En progreso:** las tareas que se encuentren en este estado, indican que están siendo desarrolladas en ese mismo momento. Para aplicar las buenas prácticas, la columna no podrá contener más de una tarea asignada a un mismo miembro del equipo, para que de esta forma el esfuerzo siempre esté centrado en una única tarea.
- **En revisión:** punto en el que las tareas han terminado de ser desarrolladas y están a la espera de ser revisadas por algún miembro del equipo para que sean integradas en la rama principal. Es en este momento donde se aplican los procesos de calidad y las técnicas de revisión para evitar en la medida de lo posible que la rama principal esté libre de código erróneo.
- **Completadas:** la columna indica que las tareas han sido terminadas y revisadas y, por tanto, integradas en la rama principal del proyecto. Las tareas de esta columna deberán estar cerradas y podrán volver a ser abiertas y trasladadas a estados anteriores en caso de que fuese necesario.

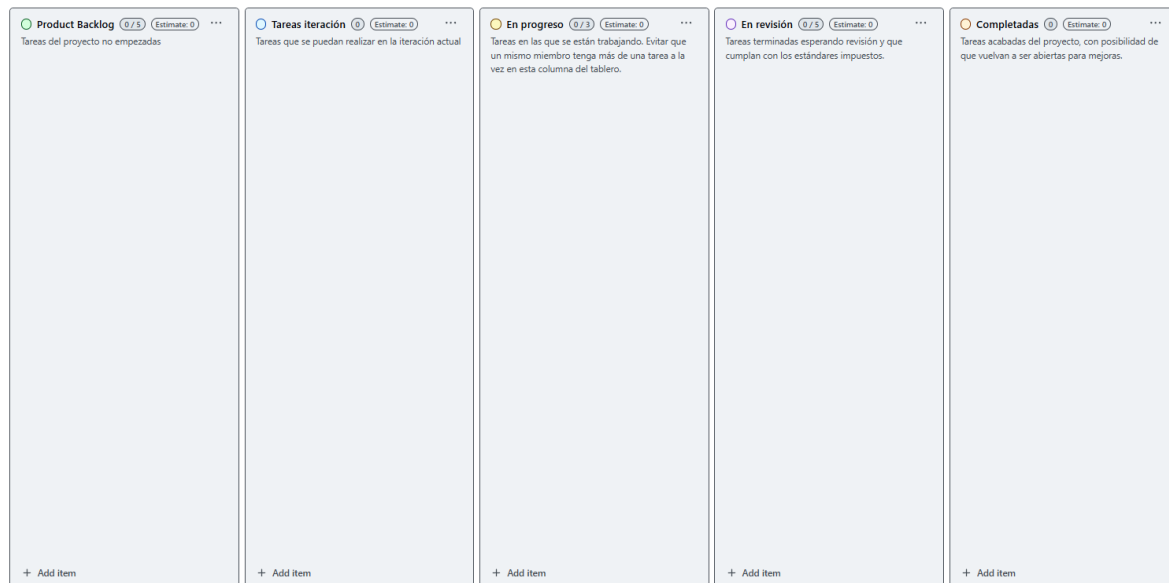


Figura 2.9: Estructura del tablero Kanban en GitHub

2.9.4. Patrón de Issues

Las Issues en GitHub [2], serán usadas para definir las tareas a implementar durante la etapa de ejecución del proyecto. Se trata de una unidad de trabajo que permite descomponer a los requisitos en tareas más manejables. Cada Issue está compuesta por una serie de atributos, que ayudan a detallar la tarea. Para el proyecto, se tendrán en cuenta los siguientes campos:

- **Título:** sirve para dar un nombre claro de la funcionalidad o tarea a realizar. Deberá estar escrito en imperativo y no superar los 50 caracteres.
- **Descripción:** atributo que dedica un espacio a explicar de forma más detallada que requiere la tarea, así como los objetivos que tiene y los requisitos a los que está ligada. Las descripciones serán redactadas usando estructuras Markdown [25] y darán una explicación más rigurosa de la tarea, además de un *To Do List* de subtareas a completar para poder cerrar la tarea.
- **Etiqueta:** atributo que colabora en la clasificación de las tareas del proyecto en función de que implementen en el código. Para el contexto de desarrollo del proyecto, se han creado las siguientes etiquetas:
 - **Etiqueta config:** indica que la tarea a desarrollar está ligada a cambios en la configuración del proyecto.
 - **Etiqueta doc:** hace alusión a tareas que incluyan la creación o edición de documentación para el proyecto.
 - **Etiqueta feat:** una tarea clasificada con esta etiqueta, indica que la tarea a desarrollar incluye nuevas funcionalidades en el código.
 - **Etiqueta refactor:** clasifica aquellas tareas que están dedicadas a revisiones de código para la mejora del mismo, como pueden ser

refactorizaciones, detección de duplicados, sugerencias de las métricas de Codacy [9], etc.

- **Etiqueta test:** indica que la tarea a desarrollar implica la creación de pruebas para el código implementado.
 - **Etiqueta style:** indica que la tarea a implementar traerá consigo la creación o edición de interfaz de la aplicación para mejorar la experiencia de usuario.
 - **Etiqueta bug:** indica que la tarea implica arreglos del código por fallas del mismo.
- **Miembro asignado:** atributo que indica quién es el responsable de desarrollar la tarea.
 - **Milestone:** indica en qué iteración del proyecto se desarrollará la tarea.
 - **Rama de trabajo:** indica en qué rama del proyecto se estará realizando la tarea.

2.9.5. Estrategia de ramificado

En cuanto a la jerarquía de ramas para el proyecto a desarrollar, se ha decidido poner en práctica Git Flow [26], que es una estrategia de ramificación que organiza el código en distintas ramas con el objetivo de facilitar el control del software que se va implementando. Es por ello que el proyecto contendrá las siguientes ramas:

- **Rama main:** es la rama principal del proyecto y será la que contenga la versión estable y lista para poner en producción.
- **Rama develop:** rama de desarrollo donde se irán integrando el código producido para la iteración en la que se encuentre el proyecto.
- **Rama feature:** son aquellas ramas temporales creadas con la intención de trabajar en ellas una tarea en concreto. Habrá una rama feature por tarea y su código se integra siempre en la rama develop. En cuanto a la forma de nombrarlas, se seguirá siempre el patrón de 'feat + / + nombre-alusivo-tarea'. Por ejemplo: 'feat/gestion-usuarios'.
- **Rama release:** son aquellas ramas destinadas a preparar nuevas versiones antes del lanzamiento a producción. Su forma de nombrarlas, será siguiendo el patrón 'release + / + v + version-codigo'. Por ejemplo: 'release/v1.0.1'.
- **Rama hotfix:** rama destinada a corregir errores urgentes en la rama principal tras la puesta en producción. Su forma de nombrarlas, será siguiendo el patrón 'hotfix/error-identificado'. Por ejemplo: 'hotfix/login'.

2.9.6. Política de commits

Los commits serán la base de un buen flujo de trabajo para el proyecto, por lo que escribirlos adecuadamente contribuye de forma positiva a mantener un historial

limpio y fácil de entender para el desarrollo.

Es por ello que esta sección está dedicada a establecer reglas y patrones para la creación de commits. Estos deberán seguir las siguientes reglas:

- Los commits deberán ser atómicos, es decir, deben contener solo un cambio lógico y completo, para que sean independientes y fáciles de entender, identificándolos rápidamente en el flujo de trabajo si estos requiriesen cambios.
- Los mensajes del commit deberán ser claros y descriptivos, explicando siempre qué añaden y por qué de forma breve.

Además de esto, los commits deberán seguir el patrón mostrado en la figura 2.10 a la hora de ser redactados, para que de un simple vistazo se pueda reconocer su naturaleza e intención rápidamente. Los atributos que sigue el patrón son los siguientes:

- **Atributo type:** sirve para identificar qué tipo de código está ligado al commit a realizar. El tipo corresponde a alguno de las etiquetas que tenga la tarea que se esté desarrollando.
- **Atributo description:** usado para indicar una pequeña descripción alusiva al commit a realizar. No deberá superar los 50 caracteres y será claro y conciso.
- **Atributo tag id:** indica el identificador de la Issue a la que está ligado el commit a realizar.

```
<type>: <description> #<tag_id>

<body>

<footer>
```

Figura 2.10: Formato del commit

2.9.7. Patrones y procedimientos de revisión

Con el fin de llevar un control del estado de las nuevas implementaciones que se incorporan en el sistema, se plantea la necesidad de elaborar un procedimiento de revisión del código.

Es por ello que la sección está dedicada a explicar los pasos que se deberán seguir para actuar de forma correcta ante la revisión del código:

1. Cuando una tarea esté considerada por el responsable como terminada, deberá moverla en el tablero Kanban al estado de revisión.
2. Se creará una Pull Request, que no es más que una solicitud para fusionar los cambios realizados en la rama de desarrollo (develop), indicando en esta la finalización de la tarea, qué incluye y a qué tareas afecta.

3. Tras su creación, otro miembro del equipo ajeno al responsable de la tarea, deberá comprobar que el código funciona correctamente y que por lo tanto puede ser integrado con la rama de desarrollo. En este momento, pueden presentarse dos escenarios distintos:

- **Código correcto:** tras comprobar el código y que este es correcto, se marca la Pull Request como aprobada con un comentario elogiando el trabajo realizado.
- **Código incorrecto:** si tras la revisión se detectan errores en la nueva implementación, el revisor podrá no aceptar la Pull Request, en cuyo caso indicará por qué motivo no puede ser integrado y comentando aquellas líneas de código donde se detectaron errores para que puedan ser corregidas. Tras realizar los cambios sugeridos, se podrá pedir de nuevo una revisión.

4. Cuando el código sea correcto, el revisor deberá cerrar la Pull Request y la Issue ligada.

Además, durante el proceso de revisión también entrarán las métricas de calidad de Codacy [9], evaluando cómo de bueno es el software a niveles de calidad. Esto se hará por un proceso automático tras la creación de las Pull Requests.

2.9.8. Documentación del sistema

Para conseguir un software que pueda ser mantenido por un buen periodo de tiempo, se documentará toda la funcionalidad del sistema, consiguiendo de esta forma que cualquier desarrollador que acceda al código pueda identificar rápidamente para qué sirve y qué aporta en el sistema.

Es por ello que esta sección se centra en definir cómo se documenta el software, que seguirá el patrón mostrado en las líneas de código 2.1:

```
1 def dividir(a, b):
2     """
3     Divide dos numeros.
4
5     Este metodo maneja la division de dos numeros, y si el divisor es 0, lanza
6     una excepcion ValueError.
7
8     Parameters:
9     a (int): El dividendo.
10    b (int): El divisor.
11
12    Returns:
13    float: El resultado de la division.
14
15    Raises:
16    ValueError: Si el divisor es 0.
```

```

16     """
17     if b == 0:
18         raise ValueError("El divisor no puede ser 0.")
19     return a / b

```

Extracto de código 2.1: Ejemplo de función documentada

2.10. Replanificación

Durante la propia etapa de planificación, se activó el riesgo RSK-007 ya que en un principio, el proyecto estaba pensado para que fuera entregado en la primera convocatoria del proyecto. Sin embargo, factores externos al proyecto han hecho que los tiempos de ejecución queden muy justos para la etapa del cierre, por lo que se ha decidido tomar acción cuanto antes y poner en marcha el plan de contingencia para el riesgo.

El nuevo plan propone extender el tiempo de entrega un mes más, por lo que el proyecto podrá ser entregado y expuesto en la segunda convocatoria. Con esto, se podría reajustar el cronograma, volviendo a tener un tiempo idóneo para completar las iteraciones de forma adecuada.

Es por ello que este apartado excepcional del documento está dedicado a presentar las modificaciones realizadas en la Gestión del Tiempo y la Gestión de los Costos, mostrando los nuevos hitos y el presupuesto final tras la replanificación.

2.10.1. Hitos

A continuación, se exponen los nuevos hitos del proyecto, así como algunos detalles relativos a los mismos en el cuadro [2.60](#):

ID	Nombre	Fecha inicio	Fecha finalización
HT-001	Planificación	03/03/2025	23/03/2025
HT-002	Estudio previo	24/03/2025	30/03/2025
HT-003	Configuración del entorno	24/03/2025	30/03/2025
HT-004	Iteración 1	31/03/2025	20/04/2025
HT-005	Iteración 2	21/04/2025	11/05/2025
HT-006	Iteración 3	12/05/2025	01/06/2025
HT-007	Memoria y manuales	02/06/2025	18/06/2025
HT-008	Presentación	19/06/2025	02/07/2025

Tabla 2.60: Lista de hitos replanificados del Trabajo de Fin de Grado

2.10.2. Presupuesto

En cuanto al presupuesto la variación es mínima, ya que para el riesgo activado, se tenía establecido un fondo de reservas para planes de contingencia y al ser el precio de plan menor al de la reserva, estos costes siguen intactos en la planificación. Aplicar el plan de contingencia, ha supuesto tres horas de trabajo para la replanificación, realizado por el personal de dirección, por lo que en total esta nueva tarea ha tomado 80,01 € del fondo.

Sin embargo, atrasar el proyecto durante un mes, supone un aumento de los costes indirectos y materiales, ya que las licencias y la tarifa de Internet se usarán durante un mes más.

Se muestran entonces el nuevo coste planificado para los costes indirectos y materiales, para finalmente mostrar en nuevo presupuesto replanificado:

Costes indirectos			
Nombre coste	Precio consumo	Cantidad consumida	Cst. P
Tarifa Internet	29,99 € / mes	7 meses	209,93 €
Consumo energético	0,34 € Kw/h	300 h.	102 €
TOTAL			311,93 €

Tabla 2.61: Desglose replanificación de costes indirectos del proyecto

Costes materiales			
Nombre coste	Precio	Cantidad usada	Cst. P
Licencia Overleaf Pro	8 € / mes	7 meses	56,00 €
Guía del PMBOK	221,82 €	-	221,82 €
Ordenador portátil	1.250,00 €	-	1.250,00 €
Página memoria	0,05 €	200	10,00 €
Encuadernado memoria	4,00 €	1	4,00 €
TOTAL			1.533,82 €

Tabla 2.62: Desglose replanificación de costes materiales del proyecto

Presupuesto replanificado	
Grupo de costes	Coste total
Costes directos	10.091,66 €
Costes indirectos	311,93 €
Costes materiales	1.541,82 €
Contingencia	2.000,00 €
TOTAL	13.945,41 €

Tabla 2.63: Presupuesto replanificado del proyecto

Finalmente, el nuevo presupuesto del proyecto será de 13.945,41€, aumentando 37,99 € más.

3. Marco teórico

3.1. Introducción

El proyecto encierra un marco teórico para ser comprendido, el cual constituye un elemento fundamental para comprender conceptos y definiciones que sustentan a la fase de ejecución del trabajo realizado.

Es por ello que se redacta este capítulo de la memoria, cuyo objetivo es comprender las ideas principales de los grafos y, en consecuencia, de su implementación en las redes neuronales.

3.2. Fundamentos de Grafos

Un grafo [27] [28] es un tipo de estructura de datos que contiene dos elementos principales:

- **Nodos:** se trata de la representación de una entidad individual dentro del grafo. En el contexto del problema que se presenta, los nodos serían las viviendas.
- **Aristas:** representan las relaciones o conexiones que existen entre dos nodos. Estas conexiones pueden ser:
 - **Según su dirección:** se identifican dos tipos de aristas:
 - **Dirigida:** se trata de una relación con dirección, donde existe un nodo raíz y un nodo destino (al que apunta la flecha).
 - **No dirigidas:** la relación de la arista no lleva una dirección asociada y por tanto se considera como una conexión bidireccional.
 - **Según su ponderación:** se identifican otros dos tipos de aristas:
 - **Ponderada:** indica que la arista que asocia a los dos nodos incluye un peso numérico que representa alguna característica de la relación. En el contexto del problema, podría ser la distancia entre dos viviendas o la similitud entre las mismas.
 - **No ponderada:** indica relaciones entre dos nodos, sin medir por medio de algún peso la importancia de esta.



Figura 3.1: Representación de aristas dirigidas y no dirigidas

La representación gráfica más sencilla del grafo viene dada por una serie de puntos, que serán los nodos y sus conexiones que serán las aristas, como se observa en la figura 3.2.

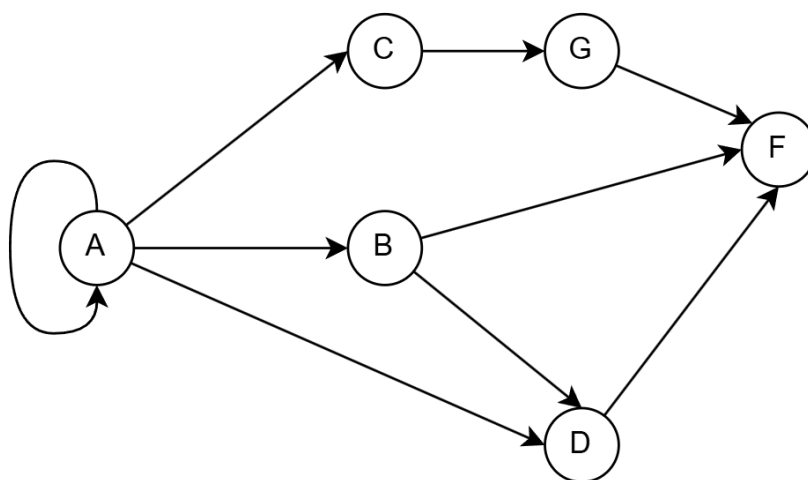


Figura 3.2: Representación gráfica de un grafo

Imagen creada por Saúl Zalimben, disponible en [Szalimben](#).

La definición formal de un grafo, se define como un par ordenado:

$$G = (V, E) \quad (3.1)$$

Donde:

- V es el conjunto de todos los nodos (o vértices).
- $E \subseteq V \times V$ es el conjunto de aristas.
 - Si se trata de un grafo no dirigido, entonces: $E \subseteq \{\{u, v\} | u, v \in V\}$
 - En el caso de ser un grafo dirigido, entonces: $E \subseteq \{(u, v) | u, v \in V\}$

En cuanto a las estructuras de datos para la representación de grafos, existen distintas formas de almacenar grafos por un ordenador. Entre las estructuras más sencillas y más usadas para su representación computacional, se encuentran las siguientes:

- **Lista de adyacencia:** se trata de una estructura donde cada nodo mantiene una lista de los nodos con los que mantiene una conexión (arista). Si el grafo

es ponderado, incluirá además el peso de cada conexión.

Su definición formal es una función que asocia a cada nodo $v \in V$ un subconjunto de nodos vecinos:

$$\begin{aligned} Adj : V &\rightarrow 2^V \\ Adj(v) &= \{u \in V \mid (v, u) \in E\} \end{aligned} \quad (3.2)$$

Si el grafo es ponderado:

$$Adj(v) = \{(u, w_{vu}) \mid (v, u) \in E, w_{vu} \in \mathbb{R}\} \quad (3.3)$$

- **Lista de incidencia:** se trata de una lista de listas, donde cada elemento representa una arista como un par (o tripleta, en caso de ser un grafo ponderado) de los dos nodos que forman la conexión.

Su definición formal, es una enumeración del conjunto de aristas:

$$E = \{e_1, e_2, \dots, e_m\} \quad (3.4)$$

Donde cada $e_i = (u, v)$ si se tratase de un grafo dirigido o $\{u, v\}$ si fuese no dirigido.

Si el grafo es ponderado, cada arista se define entonces como una tripleta, donde se incluye el peso:

$$e_i = (u, v, w_{uv}), w_{uv} \in \mathbb{R} \quad (3.5)$$

- **Matriz de adyacencia:** el grafo se construye a través de una matriz cuadrada, donde las filas y las columnas representan los nodos y cada celda indicará si hay conexión entre ellos.

Su definición formal es una matriz cuadrada $A \in \mathbb{R}^{n \times n}$, donde $n = |V|$, tal que:

$$A_{ij} = \begin{cases} 1 & \text{si } (v_i, v_j) \in E \\ 0 & \text{e.o.c.} \end{cases} \quad (3.6)$$

Si se trata de un grafo ponderado:

$$A_{ij} = w_{ij} \quad (3.7)$$

Si $(v_i, v_j) \in E$

- **Matriz de incidencia:** consiste en una matriz donde las filas representan los nodos y las columnas a las aristas del grafo.

Su definición formal es una matriz $M \in \mathbb{R}^{n \times m}$, donde:

- $n = |V|$ (nodos)
- $m = |E|$ (aristas)

Cada columna j representa una arista e_j y cada fila una relación con un nodo v_i :

$$M_{ij} = \begin{cases} 1 & \text{si } v_i \text{ pertenece a } e_j \\ 0 & \text{e.o.c.} \end{cases} \quad (3.8)$$

Si el grafo es dirigido:

$$M_{ij} = \begin{cases} -1 & \text{si } v_i \text{ es el nodo de salida de } e_j \\ 1 & \text{si } v_i \text{ es el nodo de llegada de } e_j \\ 0 & \text{e.o.c.} \end{cases} \quad (3.9)$$

3.3. Redes Neuronales de Grafos

Las redes neuronales de grafos [27] [28] se trata de una variante del algoritmo diseñada específicamente para tratar con datos que están estructurados en forma de grafos. Su estructura está muy influenciada por las redes neuronales de convolución (CNN) y la incrustación de grafos.

La búsqueda de implementar GNN comienza a principios de la década de los 2000, pero no es hasta el año 2009 cuando se formula una definición más clara y formal de este nuevo contexto, siendo un punto importante en un artículo seminal creado por *Michelle Scarselli*. El artículo surge por la motivación de encontrar soluciones a problemas que las redes neuronales convolucionales no podían abordar resultados óptimos, debido al tamaño arbitrario del grafo y a su estructura compleja.

Entre los tipos de redes neuronales de grafos más recurrentes, destacan las siguientes:

- **Redes convolucionales gráficas (GCN):** son similares a las CNN tradicionales y se comprometen a sustituirlas ofreciendo un aprendizaje de las características de los nodos por medio de inspecciones a sus relaciones vecinas, es decir, aquellas con las que mantiene una conexión por medio de una arista. Estas traen como novedad la agregación de vectores a los nodos, pasando el resultado a una capa densa y aplicando la no linealidad mediante funciones de activación. Entre las GCN existentes, conviene resaltar las siguientes:
 - **Espaciales:** definen la convolución dentro del espacio del grafo, aplicando funciones de agregación sobre los vecinos de cada nodo, lo que las hacen más intuitivas respecto al funcionamiento de una CNN tradicional. Son flexibles, escalables y adaptables a nuevos nodos, aunque menos interpretables desde una vista matemática profunda.

Entre sus aplicaciones, destaca su uso en grafos dinámicos o cambiantes, como los usados por los sistemas de recomendación.

- **Espectrales:** este enfoque se basa en el análisis espectral de grafos, haciendo uso de autovalores y autovectores del Laplaciano del grafo. Su funcionalidad principal, es aplicar un filtro que permite transformar las características de los nodos en función de la estructura global del grafo. Se usan en casos donde la estructura del grafo es fija, como podría ser una tarea de clasificación de nodos para predecir etiquetas.
- **Redes de autocodificación de grafos:** estas aprenden a representar grafos por medio de un codificador e intentan reconstruir los grafos de entrada con un decodificador. El codificador y los decodificadores se unifican por medio de una capa cuello de botella. Se trabajan con ellas en tareas de predicción de enlaces, ya que los *Auto-Encoders* que establecen resultan de gran utilidad a la hora de tratar equilibrios entre clases.
- **Redes neuronales de grafos recurrentes (RGNN):** el modelo aprende el mejor patrón de difusión y con esto consiguen manejar grafos heterogéneos, es decir, aquellos en los que un único nodo pueda tener múltiples aristas con distinto significado. Estas son usadas en la generación de textos o traducción automática, entre otros.
- **Redes neuronales de grafos cerrados (GGNN):** mejoran las tareas de dependencia a largo plazo de los RGNN añadiendo un nodo, una arista y unas compuertas temporales en las dependencias a largo plazo. Usadas para aplicaciones de recordar y olvidar información en diferentes estados.

En cuanto a las tareas más generales que abordan las GNNs, destacan las siguientes:

- **Clasificación de grafos:** tarea encargada de clasificar a los grafos en diversas categorías, por ejemplo, en redes sociales o clasificación de textos.
- **Clasificación de nodos:** tarea que aborda la función de trabajar con los atributos de los nodos vecinos para predecir las características de nuevos nodos que se integren en el grafo.
- **Predicción de enlaces:** esta tarea se encarga de predecir los enlaces entre un par de nodos vecinos de un grafo con una matriz de adyacencia que está incompleta. Se aplica en análisis de redes sociales, donde las interacciones (lo que se traduce en aristas para los grafos) son de importancia en el contexto.
- **Detección de comunidades:** se trata de una tarea que divide los nodos en diversos *clusters* basándose en la estructura de los bordes y aprendiendo así los pesos de las aristas, la distancia y los objetos del grafo.
- **Incrustación de grafos:** la tarea contempla el mapeo de los grafos en vectores, preservando la información relevante sobre los nodos, aristas y su estructura.
- **Generación de grafos:** se trata de una tarea que tiene como objetivo aprender de la distribución de grafos de muestra para generar una estructura de grafos nueva pero similar.

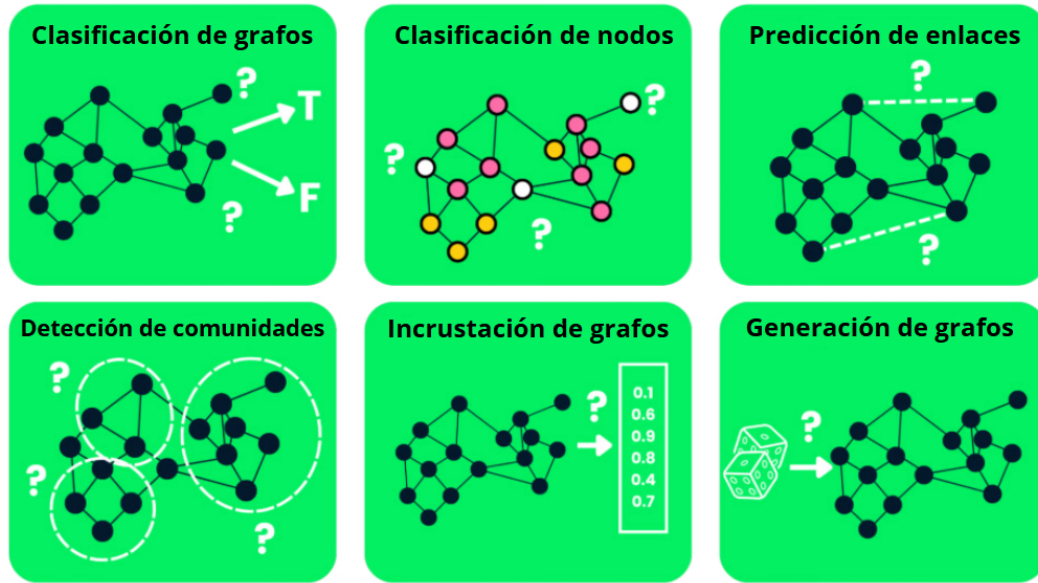


Figura 3.3: Tareas de GNN

Imagen creada por Abid Ali Awan, disponible en [Data Camp](#).

Para el contexto del problema, nos centraremos en la tarea de predicción de atributos de nodos por medio de GCN, que tiene como objetivo construir un modelo que prediga el atributo de un nodo, en este caso, el precio por metro cuadrado de una vivienda. El modelo junto al grafo, realizará la predicción usando:

- Sus propias características, esto es, los datos básicos de la vivienda, comodidades, orientaciones o distancias a puntos importantes de la ciudad de Valencia, entre otros.
- Las conexiones que tienen las viviendas con otros nodos del grafo, como pueden ser las aristas de distancia entre nodos vecinos o similitud de características.
- Los campos que tengan sus vecinos del grafo.

La tarea se completa siguiendo los siguientes pasos:

1. **Representación de los datos:** como paso inicial, se deberá definir el grafo que representará al conjunto de viviendas y sus distintas relaciones como aristas.
2. **Definición de la capa:** una capa de GCN, transforma las representaciones de los nodos según la siguiente fórmula:

$$H^{(l+1)} = \sigma(\tilde{A}H^{(l)}W^{(l)}) \text{ para } l = 0, 1, \dots, L - 1 \quad (3.10)$$

Donde:

- $H^{(l)}$: es la matriz de representación de los nodos para la capa l .
- $W^{(l)}$: define la matriz de pesos entrenables de la capa l , transformando las características a una nueva dimensión.

- $\tilde{A}$: es la matriz de adyacencia del grafo con el preprocesamiento y normalización aplicada, que mezcla la información del nodo con sus vecinos, asegurando que se promedien de forma estable.
- σ : se trata de la función de activación no lineal. Normalmente, para modelos de GCN, se usa la función ReLU dada por la siguiente expresión matemática:

$$ReLU(x) = \max(0, x) \quad (3.11)$$

Es decir:

- Si la entrada $x > 0$, entonces $ReLU(x) = x$
- Si la entrada $x \leq 0$, entonces $ReLU(x) = 0$
- L representa el número de capas.

Este paso, se puede aplicar varias veces para realizar la propagación de información desde los vecinos más lejanos.

3. **Arquitectura:** la más común en una tarea de predicción es la arquitectura GCN de dos capas:

- **Capa 1:**

$$H^1 = ReLU(\tilde{A}XW^{(0)}) \quad (3.12)$$

- **Capa 2 (última capa):**

$$Z = softmax(\tilde{A}H^{(1)}W^{(1)}) \quad (3.13)$$

La última capa será la encargada de producir una distribución de probabilidad para cada nodo.

4. **Función de pérdida:** se trata de la medida matemática que cuantifica cómo está funcionando el modelo de aprendizaje automático en la tarea de predicción. Durante el entrenamiento, el modelo ajustará sus parámetros (en este caso, los pesos) para minimizar la función de pérdida lo máximo posible. Cuanto menor sea la pérdida, mejor se ajustará el modelo al grafo de entrenamiento.

La función viene dada por la siguiente expresión matemática:

$$\mathcal{L} = - \sum_{i \in \mathcal{Y}_L} \sum_{c=1}^C y_{ic} \log Z_{ic} \quad (3.14)$$

Donde:

- $\mathcal{L}$ es la función de pérdida total que se quiere minimizar durante el entrenamiento.
- $\sum_{i \in \mathcal{Y}_L}$ es la suma sobre todos los nodos i que tienen etiqueta conocida (nodos de entrenamiento).

- $\sum_{c=1}^C$ es la suma sobre todas las clases posibles, siendo C el número total de clases.
- y_{ic} se trata de un valor binario que indica si el nodo i pertenece a la clase c .
- Z_{ic} es la probabilidad que el modelo le asigna al nodo i para pertenecer a la clase c .

5. **Optimización:** el entrenamiento, se va realizando ajustando los pesos $W^{(l)}$ para minimizar el impacto en la función de pérdida. Normalmente, se usa el algoritmo del gradiente descendiente o Adam:

$$W^{(l)} \leftarrow W^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (3.15)$$

Donde:

- η es la tasa de aprendizaje del modelo.
6. **Hiperparámetros:** se tratan de parámetros externos al modelo que se configuran antes de comenzar el proceso, es decir, son aquellos valores que controlar el comportamiento del modelo frente a la predicción, afectando al comportamiento del mismo. Colaboran en:
- Ajustar la capacidad y complejidad del modelo entrenado.
 - Controlan la velocidad y estabilidad del aprendizaje.
 - Previenen el sobreajuste.
 - Influyen en el tiempo de entrenamiento y en la calidad final de modelo.

Entre los hiperparámetros a definir para los modelos de entrenamiento, destacan:

- **Número de capas:** define cuántas capas tiene la red neuronal. A más capas, más probabilidad de captar relaciones complejas entre los nodos, aunque también aumenta el riesgo de sobreajustar el modelo.
- **Canales ocultos:** define el número de neuronas o dimensiones internas dentro de cada capa.
- **Épocas:** es el número de veces que el modelo pasa por todo el conjunto de datos del grafo para tomar el aprendizaje.
- **Tasa de aprendizaje:** indica cuánto se deben ajustar los pesos en cada paso de la optimización durante el entrenamiento.
- **Dropout:** se trata del porcentaje de neuronas que se apagan aleatoriamente durante el entrenamiento del modelo para evitar sobreajuste.

Se observa el flujo de trabajo de la convolución de grafos de forma más compacta por medio del algoritmo [18](#).

Algorithm 1 Convolución en redes neuronales de grafos

Entrada: Grafo $G = (V, E)$, características X , etiquetas Y

Salida: Predicciones de clase por nodo

- 1: **Paso 1: Representación de los datos**
 - 2: Definir el grafo G con nodos (viviendas) y aristas (relaciones)
 - 3: **Paso 2: Definición de la capa GCN**
 - 4: **para** $l = 0$ **a** $L - 1$ **hacer**
 - 5: $H^{(l+1)} \leftarrow \sigma(\tilde{A}H^{(l)}W^{(l)})$
 - 6: **end para**
 - 7: Donde $\tilde{A}$ es la matriz de adyacencia normalizada y σ es una función de activación (usualmente ReLU)
 - 8: **Paso 3: Arquitectura típica (2 capas)**
 - 9: $H^{(1)} \leftarrow \text{ReLU}(\tilde{A}XW^{(0)})$
 - 10: $Z \leftarrow \text{softmax}(\tilde{A}H^{(1)}W^{(1)})$
 - 11: **Paso 4: Cálculo de la función de pérdida**
 - 12: $\mathcal{L} \leftarrow -\sum_{i \in \mathcal{Y}_L} \sum_{c=1}^C y_{ic} \log Z_{ic}$
 - 13: **Paso 5: Optimización**
 - 14: **para todo** capas l **hacer**
 - 15: $W^{(l)} \leftarrow W^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W^{(l)}}$
 - 16: **end para**
 - 17: **Paso 6: Ajuste de hiperparámetros**
 - 18: Seleccionar: número de capas, canales ocultos, épocas, tasa de aprendizaje, dropout, etc.
-

Y una representación más visual de su funcionamiento en la figura 3.4

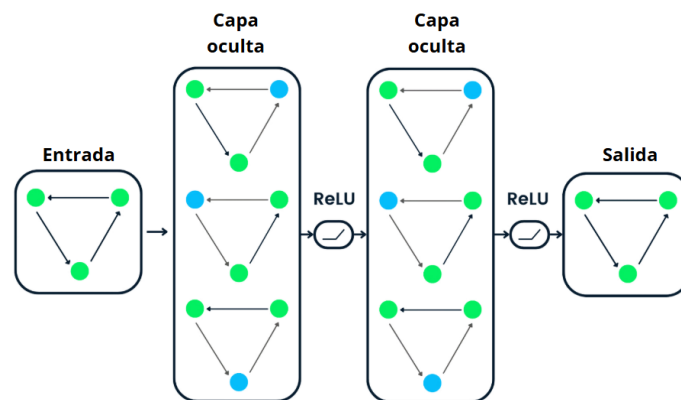


Figura 3.4: Esquema del entrenamiento

Imagen creada por Abid Ali Awan, disponible en [Data Camp](#).

3.4. Inteligencia Artificial en el Mercado Inmobiliario

Bien es sabido que la Inteligencia Artificial está transformando el mercado inmobiliario a pasos agigantados, optimizando procesos que antes requerían del tiempo y el esfuerzo humano.

Al orientar el Trabajo de Fin de Grado a este campo tan extenso, conviene conocer algunas de las principales aplicaciones que se encuentran dentro del sector inmobiliario y que están mejorando la eficiencia y la precisión en diversas áreas del mismo [29].

Entre las tareas que aborda la Inteligencia Artificial destacan las siguientes:

- **Optimización en la compraventa de viviendas:** gracias a la IA la compraventa de viviendas se realiza de forma más ágil y efectiva, ofreciendo algoritmos que analizan grandes volúmenes de datos para predecir las tendencias del mercado y sugerir oportunidades de inversión a compradores. Entre los beneficios más destacados de esta aplicación se encuentran:
 - **Búsquedas personalizadas:** existen algoritmos que ofrecen a los compradores sugerencias de propiedades según sus preferencias y comportamientos de búsqueda, como pueden ser sistemas de recomendación basados en contenido.
 - **Automatización de procesos:** los *chatbots* y asistentes virtuales están a la orden del día en este aspecto, respondiendo preguntas de los que agilizan la comunicación entre los compradores, vendedores y agentes inmobiliarios involucrados.
 - **Análisis de viabilidad:** los modelos evalúan la rentabilidad de inversión de las viviendas considerando factores como la ubicación, tendencias del mercado o la demanda, entre otros.
- **Experiencia de compradores:** la Inteligencia Artificial también está cambiando la forma en la que los clientes interactúan con los portales web inmobiliarios. Entre las innovaciones que trae destaca la realidad virtual, algoritmos que permiten realizar visitas virtuales a las propiedades, evitando así la necesidad de desplazarse hacia la misma.
- **Valoración de viviendas:** esto es aplicar modelos de *machine learning* que analizan factores de la vivienda para aplicar su tasación como pueden ser:
 - Características físicas de la vivienda.
 - Precios de inmuebles similares en la misma zona.
 - Condiciones económicas y tendencias de mercado.
 - Datos históricos de ventas.

Siguiendo con esto, existe en la actualidad una interfaz de predicción ofrecido por el portal web inmobiliario de Fotocasa [30], que utiliza modelos que están

en constante aprendizaje y trabaja con los siguientes datos de la vivienda con el objetivo de predecir sus precios:

- **Histórico de precios:** datos del comportamiento de las zonas en los últimos 20 años.
- **Tendencias de oferta y demanda:** con datos del número de propiedades disponibles vs. las búsquedas activas de los compradores.
- **Factores macroeconómicos:** como pueden ser la inflación, el PIB, las tasas de interés o el índice de paro nacional.
- **Indicadores sociales:** renta per cápita, edad media de los usuarios o movimientos migratorios de los mismos, entre otros.
- **Datos de comportamiento de los usuarios:** tiempos de permanencia en la vivienda, zonas consultadas o por inmuebles visualizados en el portal.

El trabajo a realizar se centrará en la tarea de predicción, elaborando un plan de investigación para el tratamiento de los datos con el fin de entrenar un algoritmo que permita realizar predicciones de la vivienda. Sabiendo todo esto se puede dar paso al capítulo de Ejecución, que se centra en detallar todo el camino seguido hasta la implementación.

4. Ejecución

4.1. Introducción

Este capítulo está dedicado a documentar toda la etapa de desarrollo del proyecto. Es por ello que se explican todos los procesos del *Knowledge Discovery in Databases* (KDD) [31], así como la experimentación seguida y los resultados obtenidos para cada una de las iteraciones dedicadas a la investigación.

Además, se explicará todo el proceso de desarrollo de la interfaz web a implementar para la predicción de las viviendas, conociendo en profundidad su estructura jerárquica, modelos, diagramas de clases y todas las funcionalidades que engloba.

4.2. Knowledge Discovery in Databases

Para la investigación y experimentación de los datos con los que se trabajará en el desarrollo, se aplicará *Knowledge Discovery in Databases* (KDD) [31], que consiste en un conjunto de procesos que tienen como objetivo descubrir conocimiento útil, válido y comprensible para el problema a resolver a partir de grandes volúmenes de datos.

En el contexto del problema, esta fase extracción del conocimiento será identificar, para las redes neuronales de grafos, qué algoritmos y mediante qué relaciones con aristas devuelven mejores resultados para realizar una predicción, siendo el modelo con mejores métricas el que será finalmente implementado en la aplicación web.

El proceso KDD se compone, a su vez, de las siguientes fases para llegar a la extracción del conocimiento:

- **Selección:** se trata de la primera fase del KDD y tiene como objetivo aplicar procesos para identificar y obtener los datos que serán importantes en la investigación. Además se analizan los atributos y se realiza la siguiente clasificación:
 - **Variables objetivo:** son aquellos atributos de los datos que se quieren predecir, calcular o inferir.
 - **Variables independientes:** es el conjunto de atributos de la selección de datos que serán usados para hacer las predicciones o los cálculos de la variable objetivo.

Tras finalizar la fase, se obtiene un conjunto de datos limpios para el KDD.

- **Preprocesamiento:** etapa encargada de realizar análisis sobre los datos con los que se trabajará para limpiar y corregir posibles valores erróneos detectados, garantizando así la calidad de los mismos. Se aplican procesos y técnicas para el tratamiento de valores faltantes, corrección de inconsistencias o eliminación de duplicados, entre otros. Esta etapa dará como resultado unos datos limpios y listos para ser transformados.
- **Transformación:** etapa del KDD encargada de adaptar los datos a un formato más adecuado para el contexto del problema, preparando los mismos para que puedan ser bien tratados por los modelos posteriores que trabajarán con ellos. El resultado de esta fase devuelve unos datos preparados para la minería.
- **Minería de datos:** momento en el que se seleccionan los algoritmos que trabajarán con los datos para descubrir patrones, modelos y relaciones ocultas.
- **Interpretación y evaluación de los modelos:** se evalúan si los patrones y modelos encontrados son válidos, útiles, comprensibles y generalizables. Esto se realiza aplicando métricas de evaluación para los algoritmos e interpretando los resultados devueltos.
- **Extracción del conocimiento:** etapa final del KDD que consiste en la traducción de la experimentación en conocimiento útil para la toma de decisiones finales en el sistema a implementar.

Se puede observar cual es el flujo del proceso KDD de forma gráfica por medio de la figura 4.1:

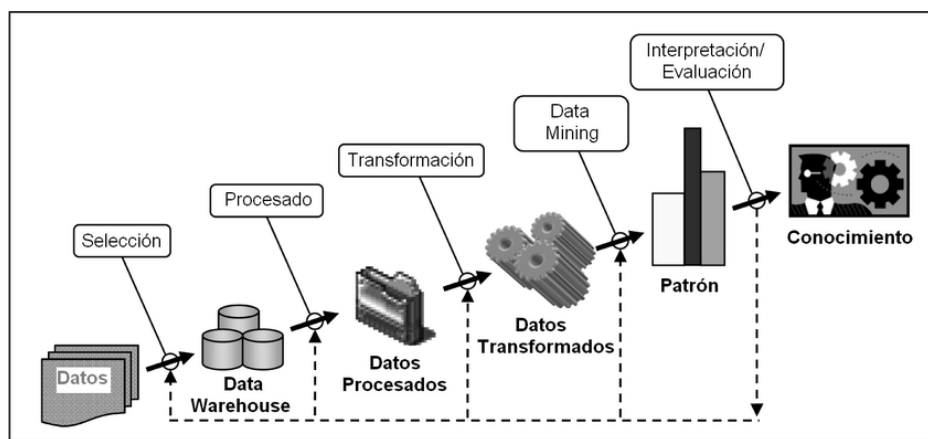


Figura 4.1: Fases del proceso KDD

Imagen subida por Juan A. Lara, disponible en [ResearchGate](#).

4.2.1. Selección de datos

Es durante esta primera fase del proceso KDD donde se lleva a cabo la recopilación y la selección de los datos que se han considerado útiles para el problema que se plantea.

Concretamente los datos seleccionados para la investigación han sido obtenidos de un artículo de Idealista [32], donde un grupo de científicos de datos pone a libre disposición más de 180.000 viviendas con una serie de características de las principales ciudades españolas: Madrid, Barcelona y Valencia.

Estos están disponibles como un paquete en R a través de un repositorio de GitHub [33], preparados principalmente para usuarios que deseen investigar con la vivienda, siendo potencialmente valioso en ámbitos de trabajo como:

- **Análisis del Mercado Inmobiliario:** permite estudios detallados sobre precios de viviendas y la segmentación del mercado.
- **Modelos de Valoración Automática:** ideales para probar y comparar distintas especificaciones de modelos hedónicos y de comparables.
- **Pruebas de Modelos de Aprendizaje Automático:** útil para el desarrollo y evaluación de modelos de *Machine Learning* aplicados al sector inmobiliario.
- **Análisis de Submercados:** facilita la identificación de submercados de la vivienda.

Para la investigación de este proyecto, se han seleccionado los datos de la vivienda relativos a la ciudad de Valencia, recogidos por medio del archivo 'Valencia_Sale.RData', el cual contiene más de 33.600 filas de datos y 43 columnas de atributos, donde cada fila representa una vivienda. A continuación, se definen los atributos para saber qué característica de la vivienda representa según su documentación [34], además de analizar el valor aportado al contexto del problema.

- **Identificación de la vivienda (ASSETID):** se trata de un identificador único para la vivienda. El formato para la identificación, consiste en una letra mayúscula seguida de una secuencia de dígitos. Aunque no se trate de un atributo independiente que aporte valor al contexto del problema, es bueno mantenerlo al actuar de identificador de la vivienda.
- **Fecha anuncio (PERIOD):** la característica hace referencia a la fecha en la que se publicó el anuncio, representado como una secuencia numérica del año de publicación seguido del mes (formato YYYYMM). Al hacer alusión a la fecha de publicación, no tiene ningún tipo de relación en la predicción de precios de vivienda, por lo que se descarta a no poder sacarle provecho como atributo independiente.
- **Precio (PRICE):** define el precio total de la vivienda en euros. Este será el atributo objetivo del problema, ya que será por medio de este por el que se realice la predicción del coste de otras viviendas.
- **Precio metro cuadrado (UNITPRICE):** indica el precio por metro cuadrado individual. De la misma forma que para el precio total, se considerará usarlo como atributo objetivo para entrenar los modelos, por lo que es indispensable mantenerlo.
- **Metros contruidos (CONSTRUCTEDAREA):** se refiere al total de metros cuadrados usados para construir la vivienda. Este atributo será indispensable

para la predicción, ya que el precio total, está directamente relacionado con los metros cuadrados que tenga la vivienda.

- **Número de habitaciones (ROOMNUMBER):** indica el número de habitaciones que tiene la vivienda. El atributo será mantenido, ya que el número de habitaciones del inmueble está altamente ligado con los metros cuadrados, porque por regla general, a más metros posea la vivienda, mayor es la probabilidad de que tenga más habitaciones.
- **Número de aseos (BATHNUMBER):** número de baños que tiene la vivienda. El atributo se mantendrá para el contexto del problema, ya que de la misma forma que el número de habitaciones, está altamente ligado a los metros cuadrados de la vivienda. Además, según artículos publicados por ING [35], tener más de un baño en la vivienda puede incrementar hasta en un 25 % el valor del inmueble.
- **Terraza (HASTERRACE):** indica si la vivienda tiene terraza (valor 1) o no (valor 0). El atributo se considera como útil en el contexto del problema, ya que está altamente relacionado con el número de planta en la que se encuentre la vivienda dentro del edificio, al encontrar terrazas por lo general en bajos o últimas plantas del edificio. Además, estudios de Idealista [36] afirman que tener un piso con terraza en Valencia puede aumentar el valor del inmueble hasta un 49,7 %, porcentaje que seguirá en crecimiento debido a la demanda de espacios en exterior dentro de las viviendas tras la pandemia.
- **Ascensor (HASLIFT):** indica si la vivienda posee ascensor dentro del edificio (valor 1) o no (valor 0). Este atributo también será importante en el contexto del problema, ya que está altamente relacionado con atributos como la planta en la que se encuentre la vivienda. Además, según estudios de Api [37], las viviendas cuyo edificio no poseen ascensor pueden valer hasta un 40 % menos que las que si poseen ascensor.
- **Aire acondicionado (HASAIRCONDITIONING):** indica si la vivienda posee como instalación aire acondicionado (valor 1) o no (valor 0). Al ser una comodidad muy buscada por los ciudadanos en Valencia debido al clima cálido de la ciudad [38], se considera que está altamente relacionado con el precio de la vivienda y por ello se mantendrá para la investigación.
- **Amenidades (AMENITYID):** se podría interpretar como un identificador de amenidades, pero no se ha encontrado información en los artículos o en el propio repositorio, por lo que se necesitaría más información para especificar qué representa realmente. Es este motivo el que lleva al descarte del atributo, al no tener ninguna noción de si aporta valor o no al contexto del problema.
- **Plaza de aparcamiento (HASPARKINGSPACE):** atributo de la vivienda que indica si posee la misma una plaza de garaje (valor 1) o no (valor 0). Se trata de un atributo interesante a mantener, ya que existen estudios de Fotocasa [39] que afirman que poseer una plaza de garaje en la vivienda aumenta el precio hasta un 7,5 % de media en España.
- **Precio aparcamiento incluido (ISPARKINGSPACEINCLUDEDINPRICE):**

indica si el precio de la plaza de garaje del inmueble está incluido en el precio total (valor 1) o no (valor 0). Este atributo será descartado al haber detectado inconsistencia con los datos de las viviendas.

- **Precio aparcamiento (PARKINGSPACEPRICE):** corresponde al precio de la plaza de aparcamiento si esta no se incluye en el precio final de la vivienda. De la misma forma que para el parámetro anterior, este será descartado, al estar ligado al problema descrito anteriormente.
- **Orientación norte (HASXORIENTATION):** se trata de cuatro atributos distintos, que indican si una vivienda tiene orientación hacia ese lado (valor 1) o no (valor 0). Existen los siguientes atributos:
 - Orientación norte (HASNORTHORIENTATION)
 - Orientación sur (HASSOUTHORIENTATION)
 - Orientación este (HASEASTORIENTATION)
 - Orientación oeste (HASWESTORIENTATION)

Las viviendas podrán tener una o varias orientaciones en el inmueble, siendo las que más influyen en el precio aquellas casas que estén orientadas hacia el norte y/o hacia el sur [40], ya que son las que más horas de luz natural aprovechan durante el día. Es por este motivo, que los atributos relativos a la orientación se mantendrán durante la investigación.

- **Trastero (HASBOOXROOM):** indica si la vivienda posee un trastero (valor 1) o no (valor 0). El atributo resulta interesante a mantener en el contexto del problema por el mismo motivo que las habitaciones y los baños, al ser una característica relacionada con el espacio y, por tanto, con los metros cuadrados que tenga el inmueble.
- **Armarios empotrados (HASWARDROBE):** indica si la vivienda tiene armarios empotrados (valor 1) o no (valor 0). Al ser otro atributo relacionado con el espacio, aunque sea en menor medida que otras características ya vistas, el precio de un inmueble se puede ver influenciado en el hecho de poseer armarios empotrados o no, por lo que se mantendrá en el contexto del problema.
- **Piscina (HASSWIMMINGPOOL):** atributo de la vivienda que indica si esta consta de piscina comunitaria en el edificio (valor 1) o no (valor 0). Se conservará el atributo para el contexto del problema, ya que existen artículos de Idealista [41] que dicen que comodidades dentro del edificio como puede ser una piscina comunitaria, aumenta el precio de la vivienda entre un 5 y un 15 %.
- **Portero (HASDOORMAN):** indica si la vivienda consta de portero en el edificio (valor 1) o no (valor 0). Al ser una comodidad del edificio que aporta seguridad en las viviendas y al estar además altamente relacionado con las comodidades de viviendas lujosas, se mantendrá por tener dependencia con el precio de la vivienda.

- **Jardín (HASGARDEN):** indica si la vivienda posee jardín comunitario (valor 1) o no (valor 0). Una zona comunitaria en el edificio puede influir en el precio de la misma, por lo que aunque sea menor medida respecto a otras comodidades, resulta interesante de mantener para la investigación.
- **Dúplex (ISDUPLEX):** indica si la vivienda se trata de un dúplex (valor 1) o no (valor 0). Aporta valor al contexto del problema, ya que tener una distribución a dos niveles, es algo buscado por los compradores, debido a la privacidad y la separación de espacios, por lo que se mantendrá para la predicción.
- **Estudio (ISSTUDIO):** el parámetro representa si la vivienda se trata de un estudio (valor 1) o no (valor 0). Este atributo está altamente relacionado con los metros cuadrados de la vivienda, ya que los inmuebles catalogados como estudios, tienen menos espacio por norma general, por lo que es interesante mantenerlo en la investigación por la extracción del conocimiento.
- **Planta alta (ISTOPFLOOR):** indica si la vivienda se encuentra en la última planta del edificio (valor 1) o no (valor 0). Este atributo se mantendrá para la predicción, ya que existen estudios que afirman que por cada piso de altura en la que se encuentre la vivienda, aumenta el precio en un 2 % [42] (siempre que el edificio posea ascensor), ya que los pisos más altos gozan también de más luz natural, mejores vistas y ventilación, entre otros.
- **Año de construcción (CONSTRUCTIONYEAR):** el atributo define el año completo en el que se construyó la vivienda. En el caso de que el valor de esta característica sea desconocido para la vivienda, será marcado con el valor NA. Pese a ser un buen atributo para el cálculo de precios del inmueble, este se descartará, ya que más de un 40 % de las viviendas registradas en el CSV, poseen valor desconocido para este campo. Además, los mismos científicos de datos han tenido esto en cuenta y por ello, han añadido un atributo extra con el año del catastro, que se explicará a continuación.
- **Planta vivienda (FLOORCLEAN):** indica la planta en la que se encuentra la vivienda dentro del edificio, siendo el valor 0 un bajo y el valor -1 un sótano o entresuelo. En caso de no conocerse el valor de la planta en la que se encuentra la vivienda, el atributo se marcará como NA para indicar que el valor es desconocido. El atributo se conservará para la predicción, ya que de la misma forma que si se encuentra en la última planta del edificio, ofrece en cierta medida una influencia en el precio del inmueble.
- **Localización vivienda (FLATLOCATIONID):** podría tratarse de un identificador de localización de en qué edificio o barriada se encuentra la vivienda, pero no existe documentación alusiva a este parámetro en los artículos o el repositorio, por lo que no se puede asumir su verdadera representación. Es por este motivo que el atributo se descartará en su totalidad de la investigación.
- **Año catastro (CADCONSTRUCTIONYEAR):** indica el año de construcción catastral de la vivienda más cercano siempre al valor que estuviera pendiente. El atributo se mantiene para el contexto del problema, ya que como se viene mencionando con anterioridad, el año que refleja el catastro para la vivienda,

indica el último registro que se realizó de la misma, pudiendo haber sufrido reformas considerables a pesar de ser un edificio de antigüedad.

- **Plantas catastro (CADMAXBUILDINGFLOOR):** indica el número de plantas totales que tiene el edificio según lo reflejado en el catastro. Relacionado con la planta en la que se encuentre la vivienda, este atributo es útil para el contexto del problema.
- **Viviendas catastro (CADDWELLINGCOUNT):** refleja el número de viviendas totales dentro del edificio donde se encuentra el inmueble a representar. Este atributo será descartado para el contexto del problema, al no encontrar datos de influencia en el precio por el número de viviendas dentro del mismo edificio que la registrada.
- **Calidad catastro (CADAstralQUALITYID):** indica el nivel de calidad de la vivienda que refleja el catastro de la misma, en un rango de valores que va de 0 (calidad máxima) a 10 (calidad más baja). Este resulta crucial, ya que una vivienda en buen estado aumentaría el precio considerablemente [35], de la misma forma que si no lo está descendería, por lo que se mantendrá para el contexto del problema.
- **Tipo de construcción (BUILTTYPEID_1, _2, _3):** se tratan de valores que indican el estado en el que se encuentra la vivienda. Encontramos los siguientes estados:
 - **TYPE 1:** este estado indica si la vivienda representada es de nueva construcción (valor 1) o no (valor 0).
 - **TYPE 2:** el estado refleja si la vivienda es de segunda mano y necesita ser reformada (valor 1) o no (valor 0).
 - **TYPE 3:** el estado indica si la vivienda es de segunda mano y está en buenas condiciones (valor 1) o no (valor 0).

Conocer el estado de la vivienda resulta un factor fundamental para fijar el valor del inmueble, pudiendo aumentar o decrementar hasta un 25 % en su precio [35], por lo que el atributo se mantendrá para la predicción.

- **Distancia al centro (DISTANCE_TO_CITY_CENTRE):** indica la distancia en kilómetros desde la localización de la vivienda hasta el centro de la ciudad. El atributo es acertado para el contexto del problema, ya que está relacionado con la posición de la vivienda, que a su vez tiene alta relación con el precio del inmueble. Es por este motivo que se mantiene.
- **Distancia al metro (DISTANCE_TO_METRO):** el atributo refleja la distancia en kilómetros entre la vivienda y la parada de metro más cercana a la misma. De la misma forma que para la distancia al centro, esta característica resulta interesante para relacionar grafos o hallar distancias, ya que existen estudios que dicen que las viviendas que se encuentren en un radio de 250 metros de paradas significativas de la ciudad, puede aumentar los precios entre un 12 % y un 18 % [43], por lo que se mantendrá para la predicción y cálculo de relaciones en la investigación.

- **Distancia a Avenida de Ibáñez Blasco (DISTANCE_TO_BLASCO):** refleja en kilómetros la distancia que hay entre la vivienda y la Avenida de Ibáñez Blasco, punto característico de la ciudad de Valencia. De la misma forma que para la distancia al centro, se trata de una avenida significativa para la ciudad, por lo que su cercanía a la misma contribuye de forma positiva al precio de la vivienda, siendo conveniente entonces mantener el atributo para la predicción.
- **Longitud (LONGITUDE):** representa la coordenada de longitud de la vivienda, que es la distancia de un punto respecto al meridiano de Greenwich.
- **Latitud (LATITUDE):** el atributo indica la coordenada de latitud de la vivienda, que es la distancia de un punto respecto al ecuador de la Tierra. Estos dos atributos resultan cruciales para la predicción, ya que la localización de la vivienda es un factor determinante en el precio y siendo uno de los campos más influyentes a la hora de establecer precios de la vivienda [35]. Además, da juego a crear nuevos campos que favorezcan al contexto del problema, por lo que sin lugar a dudas, se mantendrán para la investigación.
- **Geometría (geometry):** se trata de un atributo espacial que define las coordenadas de la vivienda como un objeto geométrico. Para este caso, está representando a la vivienda por un punto, mediante sus coordenadas de latitud y longitud. El atributo será descartado, ya que se tiene la latitud y la longitud y no necesariamente se requiere de que estos representen un punto para la predicción.

Para mejorar los datos de la vivienda, se integra junto al archivo principal otro CSV que hace alusión a los barrios de la ciudad de la Valencia, consiguiendo así enriquecer la investigación con información útil. Los datos se recogen por medio del archivo 'barris_barrios.csv', creado por el Portal de Datos Abiertos del Ayuntamiento de Valencia [44], el cual contiene 89 filas de datos, donde cada fila representa un barrio por medio de 6 parámetros. Los barrios del CSV vienen definidos por los siguientes campos:

- **Identificador distrito/barrio (coddistbar):** se trata de un atributo que representa un identificador único para el barrio dentro de su distrito, con un formato de 3 dígitos.
- **Nombre (nombre):** indica el nombre del barrio.
- **Código distrito (coddistrit):** se trata de un identificador único para indicar a qué distrito pertenece el barrio, por medio de una secuencia de dígitos.
- **Código barrio (codbarrio):** se trata de un identificador único para el barrio dentro del distrito al que pertenece, representado como una secuencia de dígitos.
- **Área del barrio (gis_gis_barrios_area):** el campo representa el número total de metros cuadrados que ocupa la geometría de la barriada.

- **Punto barrio (geo_point 2d):** se trata de un atributo que representa un objeto de tipo punto que indica el centro del barrio.
- **Geometría barrio (geo_shape):** el atributo representa un objeto de tipo geometría que define el área que ocupa la barriada en Valencia.

Resulta crucial localizar el barrio al que pertenece una vivienda, sobre todo por el auge de la gentrificación, que es el hecho que está llevando a que los barrios más populares o cercanos al centro, se vuelvan zonas más caras debido a la llegada de clases media-alta a las barriadas. Por ello, cuando se habla de ubicación, el valor influirá principalmente en la provincia, aumentando su valor si se habla a nivel de distrito y aún más cuando es para un barrio determinado.

Anonimato de la información

Cabe destacar que en los datos seleccionados, existen pequeñas transformaciones para ciertos parámetros que hacen que la información no termine de ser del todo precisa, debido a las normas de protección de datos que se rigen en España. Es por ello que los datos del precio de la vivienda y las coordenadas (latitud y longitud), llevan un proceso de enmascaramiento para anonimizar sus valores reales [34], explicados en los siguientes apartados:

- **Precio vivienda:** el proceso para añadir ruido al precio de la vivienda, viene determinado por la siguiente fórmula:

$$P = RP \cdot \varepsilon \quad (4.1)$$

Donde:

- **P:** resultado de enmascarar el precio de la vivienda.
- **RP:** es el precio real de la vivienda.
- ε : es una variable aleatoria creada por medio de la distribución uniforme con parámetros $a = 0,975$ y $b = 1,025$. La distribución uniforme de la variable viene dada por:

$$f(\varepsilon) = \begin{cases} \frac{1}{b-a} & \text{para } a \leq \varepsilon \leq b \\ 0 & \text{e.o.c.} \end{cases} \quad (4.2)$$

La expectativa de ε dados los parámetros seleccionados es la siguiente:

$$E[\varepsilon] = \frac{a+b}{2} = \frac{0,975 + 1,025}{2} = 1 \quad (4.3)$$

Y su varianza por tanto:

$$V[\varepsilon] = \frac{(b-a)^2}{12} = \frac{1}{4800} \quad (4.4)$$

Finalmente, la expectativa de los precios enmascarados es la siguiente:

$$E[P] = E[RP \cdot \varepsilon] = E[RP] \cdot E[\varepsilon] = E[RP] \quad (4.5)$$

En otras palabras, los precios que se enmascaran no son más que una versión no sesgada de los precios en bruto. Por lo que considerando que los atributos del precio real RP y la variable ε son independientes, la varianza de los precios enmascarados es la siguiente:

$$V[P] = V[RP \cdot \varepsilon] = V[RP] \cdot V[\varepsilon] + V[RP] \cdot (E[\varepsilon])^2 + V[\varepsilon] \cdot (E[RP])^2 \quad (4.6)$$

Y como $E[\varepsilon] = 1$, se tiene la siguiente forma:

$$\begin{aligned} V[P] &= V[RP] \cdot V[\varepsilon] + V[RP] + V[\varepsilon] \cdot (E[RP])^2 \\ &= V[RP] \cdot (1 + V[\varepsilon]) + V[\varepsilon] \cdot (E[RP])^2 \end{aligned} \quad (4.7)$$

Por último, resolviendo $V[RP]$ y reemplazando la expectativa de los precios reales por su versión enmascarada $E[P]$ se obtiene la siguiente varianza de los precios reales:

$$V[RP] = \frac{V[P] - \frac{1}{4800} \cdot (E[P])^2}{1 + \frac{1}{4800}} \quad (4.8)$$

- **Coordenadas:** respecto a las coordenadas, también se vieron enmascaradas por medio de un proceso especial que trata de conservar en la medida de lo posible las propiedades espaciales del conjunto de datos original, tratando de realizar un desplazamiento estocástico de los datos de localización. Esto se consigue aplicando el algoritmo 12:

Algorithm 2: Enmascarado de coordenadas por procesos de desplazamientos

- 1: **Datos:** viviendas de Idealista
- 2: **Resultados:** viviendas de Idealista con coordenadas desplazadas
- 3: Inicialización;
- 4: **para** cada vivienda L **hacer**
- 5: tomar la posición geográfica L como (X, Y)
- 6: **repetir**
- 7: coger un ángulo α que esté entre 0 y 360 grados
- 8: coger una distancia R como un valor aleatorio entre 30 y 60 metros
- 9: determinar una nueva coordenada (X', Y') calculada con R y el ángulo α
- 10: **hasta que** la condición termine
- 11: añadir (X', Y') como la nueva localización de la vivienda L
- 12: **end para**

En la figura 4.2, se muestra el histograma de la distancia en metros de todos los desplazamientos realizados en las viviendas de la ciudad de Valencia, siendo la distancia media entre las coordenadas originales y las ocultas unos 45 metros.

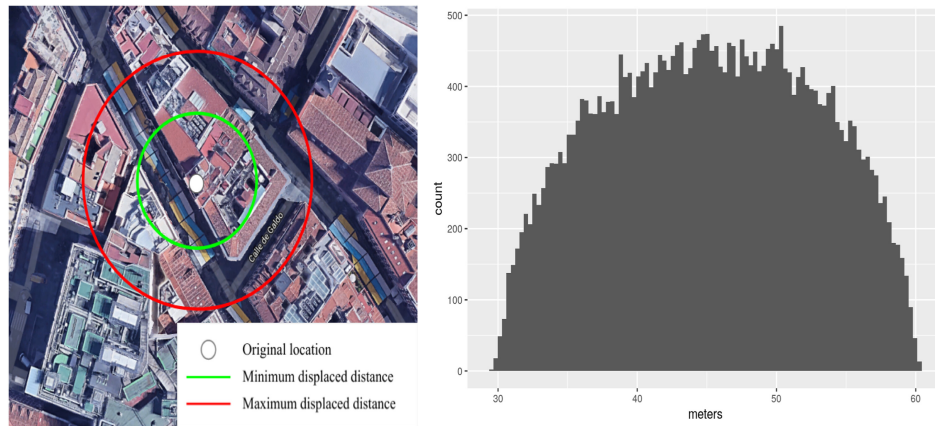


Figura 4.2: Enmascarado de coordenadas en Valencia
Imagen disponible en [Sage Journals](#).

4.2.2. Preprocesamiento de datos

La etapa de preprocesamiento de datos resulta crucial como preparativo a la minería de datos, ya que será esta quién defina la calidad de los datos de la vivienda al contexto del problema que se presenta, dándole la usabilidad y la interpretación necesarios para ser receptados por los modelos de GNN.

Es por ello que esta sección está dedicada en exponer todos los procesos y técnicas seguidos de cara a adaptar los datos de la mejor forma posible a la predicción, para posteriormente explicar paso a paso como se aplican a las viviendas de Valencia. A continuación se exponen los enfoques seguidos:

- **Reducción de datos:** se trata del proceso de reducir el volumen o la dimensión de los datos, tratando de mantener la mayor cantidad de información relevante posible. Esto es útil de cara a mejorar la eficiencia computacional de los modelos y funciones, además de facilitar la visualización e interpretación de los datos. Algunas de las técnicas a utilizar en el contexto del problema son las siguientes:
 - **Selección de atributos:** consiste en elegir un subconjunto de variables originales y relevantes para el problema. En el caso de la predicción, consistirá en descartar todas aquellas características que no aporten valor al atributo objetivo.
 - **Extracción de atributos:** se trata de definir atributos nuevos que resulten útiles para el contexto del problema, ya sean derivados o extraídos de otros ficheros que estén altamente relacionados con el que se trabaja.
 - **Discretización:** consiste en agrupar valores continuos en rangos o categorías.
- **Limpieza de datos:** es el proceso de identificar y corregir posibles errores que se encuentren entre los datos. Resulta útil de aplicar, ya que mejora la calidad de los datos, aumenta la precisión del modelo y evita generar resultados engañosos. Entre las técnicas a usar se encuentran:

- **Eliminación de duplicados:** se trata de localizar entre los datos duplicados entre ellos, para que sean eliminados.
- **Corrección de errores:** consiste en arreglar valores mal escritos o codificados en campos de los datos.
- **Estandarización de formatos:** tiene como objetivo asegurar que el formato de un mismo campo se cumpla por igual para todos y sea coherente.
- **Detección de inconsistencias lógicas:** se trata de detectar datos que no sean coherentes y carezcan de sentido en lo que representan. A modo de ejemplo para el contexto del problema que se presenta, esto podría ser una vivienda con escasos metros cuadrados y gran número de habitaciones y baños.
- **Manejo de valores faltantes:** el tratamiento consiste en identificar y gestionar los valores nulos en el dataset presentado, debido a que pueden ser errores de captura o registros incompletos que perjudiquen la adaptación de los datos. Como tratamiento, se aplicará la **eliminación de registros**, cuyo objetivo es descartar aquellas filas que poseen muchos valores nulos.

Una vez conocidas las técnicas y procesos que se seguirán en el preprocesamiento, se pasa a explicar paso por paso cómo se han ido tratando los datos hasta llegar al punto de tenerlos preparados para ser transformados.

En primer lugar, se realizan análisis de los archivos CSV para clasificar las características de la vivienda en variables relevantes o irrelevantes, indicando además el tratamiento que recibirán. Esta clasificación se expone en los cuadros 4.1 y 4.2, uno por cada archivo:

Archivo Valencia_Sale.csv				
Nombre	Formato	Tipo	Acción	Motivo
Identificador	str	ID	Mantener	No es un feat pero identifica de forma única a la vivienda
Fecha publicación	int	-	Eliminar	No aporta valor
Precio	int	objetivo	Eliminar	Se toma la decisión de predecir por el precio del metro cuadrado
Precio metro cuadrado	float	objetivo	Mantener	Será el atributo objetivo del contexto del problema
Metros construídos	int	feat	Mantener	Influye en la predicción
Número de habitaciones	int	feat	Mantener	Influye en la predicción

Nombre	Formato	Tipo	Acción	Motivo
Número de baños	int	feat	Mantener	Influye en la predicción
Terraza	bool	feat	Mantener	Influye en la predicción
Ascensor	bool	feat	Mantener	Influye en la predicción
Aire acondicionado	bool	feat	Mantener	Influye en la predicción
Amenidades	int	feat	Eliminar	No se tiene información de qué aporta, por lo que se elimina la columna en su totalidad
Plaza aparcamiento	bool	feat	Mantener	Influye en la predicción
Precio aparcamiento incluido	bool	feat	Eliminar	Se elimina la columna por incoherencias con los datos
Precio aparcamiento incluido	int	feat	Eliminar	Se elimina la columna por incoherencias con los datos
Orientación norte	bool	feat	Mantener	Influye en la predicción
Orientación sur	bool	feat	Mantener	Influye en la predicción
Orientación este	bool	feat	Mantener	Influye en la predicción
Orientación oeste	bool	feat	Mantener	Influye en la predicción
Trastero	bool	feat	Mantener	Influye en la predicción
Armarios empotrados	bool	feat	Mantener	Influye en la predicción
Piscina	bool	feat	Mantener	Influye en la predicción
Portero	bool	feat	Mantener	Influye en la predicción
Jardín	bool	feat	Mantener	Influye en la predicción
Dúplex	bool	feat	Mantener	Influye en la predicción
Estudio	bool	feat	Mantener	Influye en la predicción
Planta alta	int	feat	Mantener	Influye en la predicción
Año de construcción	int	feat	Eliminar	Se elimina la columna debido a la gran cantidad de datos con valor nulo en este parámetro (más del 40 %)
Planta vivienda	int	feat	Eliminar	Se eliminan las filas cuyo valor sea nulo

Nombre	Formato	Tipo	Acción	Motivo
Año catastro	int	feat	Mantener	Influye en la predicción
Plantas catastro	int	feat	Mantener	Influye en la predicción
Viviendas catastro	int	feat	Eliminar	No aporta valor al problema
Calidad catastro	int	feat	Modificar	Cambiar el orden, para que 9 signifique alta calidad y 0 baja
Tipo de construcción 1	bool	feat	Mantener	Aplica one hot encoding
Tipo de construcción 2	bool	feat	Modificar	
Tipo de construcción 3	bool	feat	Modificar	
Distancia al centro	float	feat	Mantener	Influye en la predicción
Distancia al metro	float	feat	Mantener	Influye en la predicción
Distancia a Blasco	float	feat	Mantener	Influye en la predicción
Latitud	float	feat	Mantener	Influye en la predicción
Longitud	float	feat	Mantener	Influye en la predicción
Geometría	object	feat	Eliminar	No se necesitan tipos objetos

Tabla 4.1: Clasificación de atributos de la vivienda

Archivo barris-barrios.csv				
Nombre	Formato	Tipo	Acción	Motivo
Identificador distrito/barrio	int	ID	Modificar	Se cambia la forma de enumeración y su representación de la cabecera
Nombre	str	-	Modificar	Se modifica la cabecera para que siga el formato del CSV principal, no los nombres de los barrios
Código distrito	int	-	Eliminar	No se conoce más información de los distritos salvo su identificador, por lo que se descarta

Nombre	Formato	Tipo	Acción	Motivo
Código barrio	int	-	Eliminar	Ya existe un identificador de barrios
Área del barrio	float	-	Eliminar	No aporta valor
Punto barrio	point	-	Eliminar	No aporta valor
Geometría barrio	geometry	-	Mantener	Ayuda a conocer en qué barrio está la vivienda

Tabla 4.2: Clasificación de atributos de los barrios

Tras la clasificación, se empiezan a aplicar los procesos y técnicas de preprocesamiento preparados para los datos.

En primer lugar, debido a problemas de interpretación con el paquete `numpy`, se descarta el atributo `geometry` del fichero CSV principal, para que posteriormente pueda ser bien manejado como un objeto `DataFrame`.

Para ello, se implementa una función de lectura del CSV, donde dado un fichero CSV de viviendas en crudo, detecta en este la columna `geometry` por la posición en la que se encuentra en la cabecera y la descarta del diccionario de viviendas que se usará para la posterior escritura. Se expone la función desarrollada en las líneas de código de la extracción [4.1](#):

```

1 def leer_csv(ruta_archivo):
2     datos = []
3     try:
4         with open(ruta_archivo, mode='r', encoding='utf-8') as archivo:
5             lector = csv.reader(archivo)
6             encabezados = next(lector) # La primera fila es la de
encabezados
7             encabezados = encabezados[:41] # Elimina la columna "geometry"
8             for fila in lector:
9                 vivienda = {encabezados[i]: fila[i] for i in range(len(
encabezados))}
10                datos.append(vivienda)
11     except FileNotFoundError:
12         print(f"El archivo {ruta_archivo} no fue encontrado.")
13     except Exception as e:
14         print(f"Ocurrió un error al leer el archivo: {e}")
15     return datos

```

Extracto de código 4.1: Función de lectura del CSV

Continuando con la función anterior, el siguiente paso será crear un nuevo fichero CSV con el cambio aplicado, siendo este el archivo de viviendas con el que se trabajará finalmente para aplicar el preprocesamiento.

Es por ello que se desarrolla una nueva función de escritura de archivos CSV, donde dado los datos y el encabezado modificados por la función de lectura, escribirá un nuevo archivo de extensión CSV. Es posible conocer su implementación por medio de la extracción de código 4.2:

```
1 def escribir_csv(datos, encabezados, csv_entrada):
2     try:
3         base_dir = Path(__file__).resolve().parent
4         directorio = base_dir / "processed_data"
5         directorio.mkdir(parents=True, exist_ok=True)
6         nombre_base = Path(csv_entrada).stem + "_processed.csv"
7         ruta_salida_final = directorio / nombre_base
8
9         with open(ruta_salida_final, mode='w', encoding='utf-8', newline='')
10        as archivo:
11            escritor = csv.DictWriter(archivo, fieldnames=encabezados)
12            escritor.writeheader() # Encabezados
13            escritor.writerows(datos) # Filas
14            print(f"Archivo CSV creado correctamente en {ruta_salida_final}")
15    except Exception as e:
16        print(f"Ocurrió un error al escribir el archivo: {e}")
```

Extracto de código 4.2: Función de escritura del CSV

Ambas funciones son internas de la función principal `procesar_csv_viviendas()`, que se encargará de ejecutar ambas en conjunto.

Una vez conocido cómo se realiza esta primera modificación en el CSV de viviendas, se pasa a trabajar el otro archivo de datos en crudo que se usará para la predicción, que hace alusión a los barrios de Valencia.

Como preparación previa a ser usados para potenciar el fichero de datos principal, se realiza un procesado previo de los barrios y sus características, comenzando a aplicar algunas de las técnicas y procesos explicados con anterioridad.

Es por este motivo que se desarrolla una función de procesado de datos de los barrios, cuyas líneas de código y aplicaciones realizadas se exponen en la extracción 4.3:

```
1 def procesar_csv_barrios(ruta_csv):
2     try:
3         base_dir = Path(__file__).resolve().parent
4         processed_dir = base_dir / "processed_data"
5         processed_dir.mkdir(parents=True, exist_ok=True)
6
7         nombre_base = Path(ruta_csv).stem + "_processed"
8         ruta_csv_salida = processed_dir / "barrios-barrios_data.csv"
9         ruta_geojson_salida = processed_dir / f"{nombre_base}.geojson"
10
```

```

11     df = pd.read_csv(ruta_csv, sep=';', dtype=str) # Barrios
12
13     # Eliminacion de columnas
14     columnas_a_eliminar = ['codbarrio', 'gis_gis_barrios_area', '
15     geo_point_2d', 'coddistrit']
16     df = df.drop(columns=columnas_a_eliminar, errors='ignore')
17
18     # Estandarizacion de cabeceras
19     df = df.rename(columns={
20         'coddistbar': 'NEIGHBOURID',
21         'nombre': 'NEIGHBOURNAME',
22         'geo_shape': 'geometry'
23     })
24
25     # Identificador unico
26     if not pd.api.types.is_numeric_dtype(df['NEIGHBOURID']):
27         df['NEIGHBOURID'] = range(1, len(df) + 1)
28
29     # CSV de datos para Django
30     df[['NEIGHBOURID', 'NEIGHBOURNAME']].to_csv(ruta_csv_salida, index=False
31     , encoding='utf-8')
32
33     # Convertir la columna geometry (GeoJSON string) a objetos shapely
34     def geojson_to_shape(geojson_str):
35         try:
36             geojson = json.loads(geojson_str)
37             return shapely.geometry.shape(geojson)
38         except Exception:
39             return None
40
41     df['geometry'] = df['geometry'].apply(geojson_to_shape)
42
43     # GeoJSON de barrios
44     gdf = gpd.GeoDataFrame(df, geometry='geometry', crs="EPSG:4326")
45     gdf.to_file(ruta_geojson_salida, driver='GeoJSON')
46     print(f"Archivos guardados en {ruta_csv_salida} y {ruta_geojson_salida}")
47 )

```

Extracto de código 4.3: Función de procesamiento de barrios

La función mostrada aplica el siguiente preprocesamiento:

- **Selección de datos:** realizado en la línea de código 14, donde se le indica al objeto DataFrame con el método `drop()`, el nombre de las columnas que deberá descartar del fichero CSV a escribir.
- **Corrección de errores:** aplicado por distintos motivos en los siguientes fragmentos de código.
 - **Renombrado de columnas:** aplicado en las líneas que van de la 18 a la 22, donde se le indica al objeto DataFrame por medio del método `rename()`

qué columnas deberán ser renombradas y cual será su nueva cabecera. Esto se realiza de cara a que las cabeceras que se integrarán más adelante con el CSV principal, cumplan el mismo estándar que este.

- **Modificación del identificador:** intervienen las líneas de código 24 y 25, al encontrar durante el análisis de los ficheros que los identificadores de los barrios no seguían un estándar claro. Es por ello que se modifican realizando un incremento por cada barrio existente en el DataFrame.

La ejecución del bloque de código da lugar a la creación de dos archivos con distintos formatos:

- **Formato CSV:** se trata de un archivo destinado a definir los barrios acorde con el modelo a crear en la futura interfaz web de Django [23], ya que los datos de los barrios deberán ser almacenados en la base de datos.
- **Formato GeoJSON:** fichero a usar durante la integración con el CSV de datos de vivienda principal. Se usa esta extensión debido a que trabaja mejor con objetos geométricos, como puede ser el polígono que define el área que ocupa el barrio, atributo que se usará para calcular atributos derivados junto con las viviendas.

Tras las preparaciones previas de los ficheros con los que se trabajarán en la predicción, es posible plantear un preprocesamiento en conjunto, cuyos procesos y técnicas se aplican por medio del bloque de código 4.4:

```
1 def preprocesamiento(ruta_viviendas, ruta_barrios):
2     columnas_a_eliminar = [
3         "PERIOD", "PRICE", "AMENITYID", "ISPARKINGSPACEINCLUDEDINPRICE",
4         "PARKINGSPACEPRICE", "CONSTRUCTIONYEAR", "FLATLOCATIONID",
5         "CADCONSTRUCTIONYEAR", "CADDWELLINGCOUNT", "geometry",
6     ]
7
8     columnas_a_convertir_entero = [
9         "PRICE", "ROOMNUMBER", "BATHNUMBER", "FLOORCLEAN",
10        "CADMAXBUILDINGFLOOR", "CADASTRALQUALITYID",
11    ]
12
13    atributos_a_normalizar = [
14        "CONSTRUCTEDAREA", "ROOMNUMBER", "BATHNUMBER", "FLOORCLEAN",
15        "CADMAXBUILDINGFLOOR", "CADASTRALQUALITYID", "ANTIQUITY",
16        "DISTANCE_TO_CITY_CENTER", "DISTANCE_TO_METRO", "DISTANCE_TO_BLASCO"
17    ]
18
19    try:
20        df = pd.read_csv(ruta_viviendas, delimiter=',', encoding='utf-8') #
21        Viviendas
22        gdf_barrios = gpd.read_file(ruta_barrios) # Barrios
23
24        # Elimina de valores faltantes de FLOORCLEAN
25        df = df.dropna(subset=["FLOORCLEAN"])
```

```

25
26     # Elimina duplicados en ASSETID
27     df = df.drop_duplicates(subset=["ASSETID"], keep="first") # Conserva el
primero
28
29     # Conversion columnas numericas a entero
30     for columna in columnas_a_convertir_entero:
31         if columna in df.columns:
32             df[columna] = df[columna].fillna(0).astype(int)
33
34     # Atributo ANTIQUITY = 2018 - CADCONSTRUCTIONYEAR
35     if "CADCONSTRUCTIONYEAR" in df.columns:
36         df["ANTIQUITY"] = 2018 - df["CADCONSTRUCTIONYEAR"]
37
38     # Elimina columnas innecesarias
39     df = df.drop(columns=columnas_a_eliminar, errors='ignore')
40
41     # Modifica CADASTRALQUALITYID:
42     if "CADASTRALQUALITYID" in df.columns:
43         df["CADASTRALQUALITYID"] = df["CADASTRALQUALITYID"].apply(lambda x:
9 - x if pd.notnull(x) else x)
44
45     def buscar_barrio(row):
46         try:
47             point = shapely.geometry.Point(float(row['LONGITUDE']), float(
row['LATITUDE']))
48             match = gdf_barrios[gdf_barrios.geometry.contains(point)]
49             if not match.empty:
50                 barrio = match.iloc[0]
51                 return pd.Series([barrio['NEIGHBOURID']])
52             else:
53                 return pd.Series(['NA'])
54         except Exception:
55             return pd.Series(['NA'])
56
57     df[['NEIGHBOURID']] = df.apply(buscar_barrio, axis=1)
58     df = df[(df['NEIGHBOURID'] != 'NA')].reset_index(drop=True)
59
60     # Directorio y archivos de salida
61     base_dir = Path(__file__).resolve().parent
62     processed_dir = base_dir / "processed_data"
63     processed_dir.mkdir(parents=True, exist_ok=True)
64     ruta_salida_viviendas = processed_dir / "Valencia_Sale_graph.csv"
65     ruta_salida_no_normalizado = processed_dir / "Valencia_Sale_data.csv"
66     ruta_salida_barrios = processed_dir / "barris-barrios_data.csv"
67
68     # CSV no normalizado
69     df.to_csv(ruta_salida_no_normalizado, index=False, encoding='utf-8')
70
71     # Normalizacion

```

```

72     scaler = MinMaxScaler()
73     for col in atributos_a_normalizar:
74         if col in df.columns:
75             df[[col]] = scaler.fit_transform(df[[col]])
76
77     # CSV normalizado
78     df.to_csv(ruta_salida_viviendas, index=False, encoding='utf-8')
79
80     # Elimina barrios sin viviendas asociadas
81     if ruta_salida_barrios.exists():
82         barrios_df = pd.read_csv(ruta_salida_barrios, encoding='utf-8')
83         barrios_con_viviendas = df['NEIGHBOURID'].unique()
84         barrios_df_filtrado = barrios_df[barrios_df['NEIGHBOURID'].astype(
85 str).isin(barrios_con_viviendas.astype(str))]
86         barrios_df_filtrado.to_csv(ruta_salida_barrios, index=False,
87 encoding='utf-8')
88         print(f"Barrios sin viviendas eliminados de {ruta_salida_barrios}")
89
90     # Elimina el archivo de entrada tras procesar
91     if os.path.exists(ruta_viviendas):
92         try:
93             os.remove(ruta_viviendas)
94             print(f"Archivo {ruta_viviendas} eliminado tras el procesamiento
95 .")
96         except Exception as e:
97             print(f"No se pudo eliminar el archivo {ruta_viviendas}: {e}")
98
99     print(f"El DataFrame tiene {len(df)} filas y {len(df.columns)} columnas
100 despues del preprocesamiento.")
101     return df
102 except FileNotFoundError:
103     print(f"El archivo {ruta_viviendas} no fue encontrado.")
104 except Exception as e:
105     print(f"Ocurrio un error al leer el archivo: {e}")
106     return None

```

Extracto de código 4.4: Función de preprocesamiento

El preprocesamiento en conjunto aplica las siguientes técnicas y procedimientos:

- **Selección de atributos:** aplicado en las líneas de código que van de la 2 a la 6 y en la línea 39, donde se seleccionan por medio de la lista `columnas_a_eliminar` las cabeceras de las columnas que serán descartadas en su totalidad, para posteriormente eliminarlas del objeto `DataFrame` por medio del método `drop()`.
- **Extracción de atributos:** se aplica para los siguientes casos:
 - **Antigüedad:** se trata de un atributo derivado por medio de dos características que ofrece la vivienda, donde se calculan los años de antigüedad del inmueble, resultando de restarle el año de registro del

catastro al año de publicación del inmueble, que para todas las viviendas, el año es 2018. En el código, queda reflejado en el condicional de las líneas 35 y 36.

- **Localización barrio:** se trata de un atributo extraído por medio del fichero de barrios, donde se le asigna a la vivienda el barrio al que pertenece en función de si sus coordenadas se encuentran dentro del polígono definido para el barrio. Se aplica por medio de la función interna `buscar.barrio()`, que determina el barrio al que pertenece la vivienda y, si no encuentra ninguno, marcará el inmueble con valor nulo `NA` para que la fila sea eliminada posteriormente.
- **Discretización:** se aplica para los atributos numéricos de las viviendas, en los bloques que van de la línea 13 a la 17 y de la 71 a la 74. Se realiza por medio de un escalador ofrecido por el paquete `sklearn`, que transforma los datos, reescalándolos entre 0 y 1 por medio de `MinMaxScaler()`, que trabaja con la siguiente fórmula:

$$X' = \frac{X - X_{\min}}{X_{\max} - X_{\min}} \quad (4.9)$$

Donde:

- X corresponde al valor original del atributo a normalizar.
 - $X_{\min}$ es el valor mínimo de la variable X encontrado en el conjunto de datos.
 - $X_{\max}$ representa el valor máximo de la variable X encontrado en el conjunto de datos.
 - X' es el valor reescalado resultante de la normalización.
- **Eliminación de duplicados:** reflejado en la línea de código 27, aplicando al `DataFrame` el método `drop_duplicates()`, que sirve para indicar la columna en la que se han encontrado duplicados y cómo se quieren tratar, que para este caso, se ha decidido mantener el primero que aparezca dentro del objeto (argumento `keep="first"` del método) y borrar el resto.
- **Estandarización de formatos:** se aplica en diferentes partes del preprocesamiento:
 - **Tipos numéricos:** aplicado en las líneas de código que van de la 8 a la 11 y en el bucle implementado en las líneas de la 30 a la 32, para asegurar el formato como tipo entero.
 - **Nivel de calidad catastro:** durante el análisis, se contempló el tratamiento que recibía el atributo ligado al nivel de calidad indicado por el catastro, que en un rango comprendido entre 0 y 9 indicaba si la calidad era máxima (0) o mínima (9). Esto resulta poco práctico para el contexto del problema, ya que los algoritmos a usar trabajan mejor cuando se indica que la característica es mejor en función de si atributo numérico tiene un valor mayor. Por ello, en el condicional definido para

las líneas de código 42 y 43, se modifica el valor de la calidad, cambiándolo por el complementario comprendido en el rango.

- **Detección de inconsistencias lógicas:** se aplica en los atributo relativos a si la vivienda posee plaza de garaje y el precio de la misma. Esto es porque durante el análisis del fichero de viviendas, se detectó que aún indicando que si se incluyese el precio de la plaza de garaje a precio final de la vivienda, finalmente no se agregaba. Es por este factor que en el borrado se incluye la eliminación de las columnas `ISPARKINGSPACEINCLUDEDINPRICE` y `PARKINGSPACEPRICE`.
- **Eliminación de registros:** se aplica en los siguientes atributos:
 - **Planta de la vivienda:** en el análisis del fichero de inmuebles se detectó que existían viviendas de las que no se tenía ninguna información sobre en qué planta se encontraban. Por ello, en la línea de código 24, se aplica al DataFrame el método `dropna()`, el cual elimina del objeto todas aquellas viviendas que tengan un valor *NA* como planta del edificio.
 - **Localización de barrio:** de igual forma, se aplica esto para las viviendas que tras hacer una búsqueda del barrio al que pueden pertenecer, no se consiguen asociar a ninguno. Es por este motivo que en la línea 58, se eliminan aquellos registros de vivienda que posean un valor *NA* en el campo de identificador del barrio.
 - **Barrios:** se tratan de aquellos que tras tratar de localizar a las viviendas no poseen registros de inmuebles, y por ello son descartados del CSV al no poseer suficientes datos para poder realizar un entrenamiento que prediga en condiciones para la barriada.

El procesamiento genera los siguientes ficheros como resultado:

- **Normalizado:** se trata de un archivo CSV con todos los atributos trabajados y normalizados, generado para su posterior uso durante la transformación de los datos en grafos.
- **No normalizado:** el fichero recoge todos los registros de vivienda que quedan después del preprocesamiento, pero con la diferencia de que los atributos numéricos no aparecerán normalizados. Esto es debido a que este fichero será usado para cargar la base de datos de la aplicación y no deberá estar preparado de cara a ser usado por algoritmos.

Finalmente, la etapa de procesamiento genera un archivo CSV de viviendas que está preparado para la consecuente etapa de transformación de datos. El fichero, contiene 25998 filas de datos, que comparándolo con el fichero inicial supone que se han mantenido un 77 % de los datos. Además, cada vivienda poseerá 35 atributos, de los cuales 33 son considerados como independientes y uno de predicción.

En cuanto al fichero de barrios de Valencia, se reduce el número un 17 % aproximadamente, pasando de tener datos de 88 barrios a 73. Con ello, se reduce también su número de atributos, manteniendo tres para el archivo GeoJSON y dos para el fichero CSV de carga de datos.

4.2.3. Transformación de datos

En el proceso de extraer el conocimiento de los datos de la vivienda, la fase de transformación juega un papel importante para el modelado de la información en un formato correcto que enriquezca a los modelos de entrenamiento.

Es por ello que en esta sección se describirán todos los procesos seguidos para la transformación de los datos en una representación como grafo, así como el código implementado para conseguir las representaciones de cara a la siguiente fase del proceso.

Relaciones establecidas

Para transformar el dataset de viviendas en grafos, es fundamental definir qué atributos representarán a los nodos y qué tipo de conexiones se establecerán entre estos.

Es por ello que en esta sección, se exponen cómo se crean los nodos y los vínculos que se crean para trabajar con las viviendas del fichero de datos preprocesado. A continuación, se exponen los pasos seguidos durante esta etapa.

En primer lugar, se trabaja en los nodos, seleccionando las características necesarias para generar el grafo y usarlo en la predicción. Esto se trabaja por medio de la función definida en el bloque de código 4.5:

```
1 def exportar_nodos(csv_entrada):
2     node_cols = [
3         "UNITPRICE", "CONSTRUCTEDAREA", "ROOMNUMBER", "BATHNUMBER",
4         "HASTERRACE", "HASLIFT", "HASAIRCONDITIONING", "HASPARKINGSPACE",
5         "HASNORTHORIENTATION", "HASSOUTHORIENTATION", "HASEASTORIENTATION", "
6         HASWESTORIENTATION",
7         "HASBOXROOM", "HASWARDROBE", "HASSWIMMINGPOOL", "HASDOORMAN", "HAGARDEN
8         ",
9         "ISDUPLEX", "ISSTUDIO", "ISINTOPFLOOR", "FLOORCLEAN", "
10        CADMAXBUILDINGFLOOR",
11        "CADASTRALQUALITYID", "BUILTTYPEID_1", "BUILTTYPEID_2", "BUILTTYPEID_3",
12        "DISTANCE_TO_CITY_CENTER", "DISTANCE_TO_METRO", "DISTANCE_TO_BLASCO",
13        "LONGITUDE", "LATITUDE", "ANTIQUITY", "NEIGHBOURID"
14    ]
15
16    df = pd.read_csv(csv_entrada) # Viviendas
17    barrios = df["NEIGHBOURID"].unique()
18
19    # Directorio de salida
20    BASE_DIR = Path(__file__).resolve().parent
21    nodes_dir = BASE_DIR / "graphs" / "nodes"
22    nodes_dir.mkdir(parents=True, exist_ok=True)
23
24    # Archivo CSV por barrio
25    for barrio in barrios:
```

```

23 df_barrio = df[df["NEIGHBOURID"] == barrio][node_cols].copy()
24 df_barrio.insert(0, "NODEID", df_barrio.index)
25 nodes_out = nodes_dir / f"Valencia_nodes_{barrio}.csv"
26 df_barrio.to_csv(nodes_out, index=False, encoding='utf-8')
27 print(f"Nodos del barrio {barrio} exportados a {nodes_out}")

```

Extracto de código 4.5: Función de creación de los nodos

La función genera un fichero CSV por cada barrio, con la intención de separar las viviendas que pertenecen a una misma barriada en distintos datasets.

Esto se realiza de cara a hacer entrenamientos individuales de cada barrio y no como el conjunto de viviendas de toda Valencia, ya que quedaría un grafo de grandes dimensiones que sería difícil de ejecutar de forma rápida.

Una vez se tienen los nodos de las viviendas por barrio, llega el momento de generar relaciones entre las viviendas que resulten interesantes para el contexto del problema. Para la investigación, se proponen los siguientes tratamientos en las conexiones entre los nodos:

- **Aristas por vecindad:** el objetivo es conseguir conexiones con peso entre las viviendas en función de la distancia que haya entre estas. Esto se consigue por medio de la función exportada en el bloque de código 4.6:

```

1 def exportar_aristas_vecindad(nodos, radio_km):
2     # Lectura del CSV
3     df = pd.read_csv(nodos)
4
5     # Coordenadas
6     coords = df[["LATITUDE", "LONGITUDE"]].values
7     coords_rad = np.radians(coords)
8     tree = BallTree(coords_rad, metric='haversine')
9     radio_rad = radio_km / 6371.0
10
11     # Búsqueda de vecinos
12     ind_array, dist_array = tree.query_radius(coords_rad, r=radio_rad,
13     return_distance=True)
14     edge_rows = set()
15     n = len(coords)
16     for i, (neighbors, distances) in enumerate(zip(ind_array, dist_array)):
17         for j, dist in zip(neighbors, distances):
18             if i < j:
19                 edge_rows.add((i, j, dist * 6371.0))
20
21     # Detección de nodos aislados
22     vecinos = {i for edge in edge_rows for i in edge[:2]}
23     aislados = set(range(n)) - vecinos
24
25     # Búsqueda del vecino mas cercano en nodos aislados
26     if aislados:
27         dists, inds = tree.query(coords_rad, k=n)

```

```

27         for i in aislados:
28             for j, dist in zip(inds[i], dists[i]):
29                 if i != j:
30                     edge_rows.add((min(i, j), max(i, j), dist * 6371.0))
31                     break
32
33     df_edges = pd.DataFrame(list(edge_rows), columns=["source", "target", "
DISTANCE"])
34
35     # Formateo del radio para nombres de archivo
36     radio_str = f"{radio_km:.3f}".replace('.', '').zfill(4)
37
38     # Directorio de salida
39     BASE_DIR = Path(__file__).resolve().parent
40     edges_dir = BASE_DIR / "graphs" / "edges" / "kdd" / f"{radio_str}_km"
41     edges_dir.mkdir(parents=True, exist_ok=True)
42
43     # Ficheros de salida por barrio
44     barrios = df["NEIGHBOURID"].unique()
45     if len(barrios) == 1:
46         edges_out = edges_dir / f"Valencia_kdd_edges_{radio_str}_{barrios
[0]}.csv"
47         df_edges.to_csv(edges_out, index=False, encoding='utf-8')
48         print(f"Aristas por vecindad exportadas a {edges_out}")
49     else:
50         for barrio in barrios:
51             idxs = df[df["NEIGHBOURID"] == barrio].index.values
52             mask = df_edges["source"].isin(idxs) & df_edges["target"].isin(
idxs)
53             df_edges_barrio = df_edges[mask]
54             if not df_edges_barrio.empty:
55                 edges_out = edges_dir / f"Valencia_kdd_edges_{radio_str}_{
barrio}.csv"
56                 df_edges_barrio.to_csv(edges_out, index=False, encoding='
utf-8')
57                 print(f"Aristas por vecindad exportadas a {edges_out}")

```

Extracto de código 4.6: Función de creación de aristas por vecindad

La función lleva el siguiente procedimiento para generar las aristas del fichero de nodos:

1. Lee el fichero de nodos de un archivo CSV que contiene las viviendas de un barrio concreto.
2. Selecciona la columna de coordenadas (latitud y longitud) y los transforma a radianes de cara a calcular la distancia Harvesine, que requiere que los datos estén en ese formato. Seguido de esto se crea un BallTree por medio de la librería scikit-learn, para que con la métrica de Harvesine, se indexen las posiciones geográficas de las viviendas.

3. Acto seguido, se empiezan a generar las aristas, donde se tienen dos posibles casos para su generación:

- Si la vivienda tiene vecinos dentro del radio impuesto como parámetro, se establecerá una arista ponderada por cada vivienda dentro de su radio de vecindad, siendo el peso la distancia entre las dos. Se muestra el funcionamiento de una forma más visual a través de la figura 4.3:

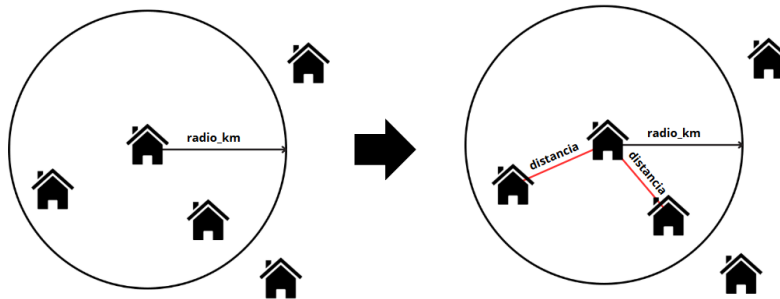


Figura 4.3: Generación de aristas por vecindad primer caso

- En el caso de que la vivienda no tenga ninguna vivienda dentro de su radio de vecindad, para evitar tener nodos aislados en el grafo se generará de igual forma una arista con peso a la vivienda más cercana, aunque no haya cumplido el radio impuesto. Se muestra la idea del funcionamiento de una forma más visual a través de la figura 4.4:

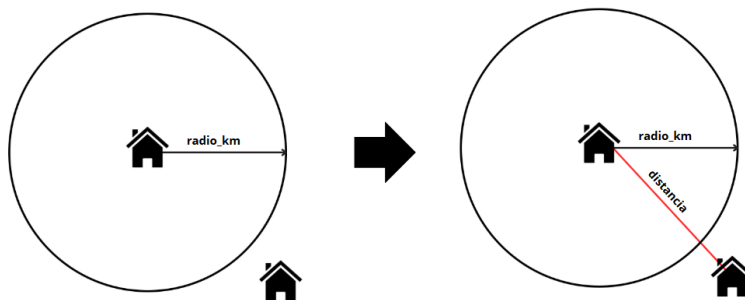


Figura 4.4: Generación de aristas por vecindad segundo caso

4. Finalmente, se genera un archivo CSV para representar las aristas entre las viviendas con su peso asociado.

Para conseguir diversas variaciones del grafo, se generan distintos ficheros de aristas modificando el radio de vecindad, probando hasta con seis radios.

- **Aristas por similitud:** se trata de conseguir conexiones entre las viviendas en función de un umbral de similitud de características, calculado por medio de

atributos de la vivienda como pueden ser las comodidades o sus datos básicos. Para la creación de estas aristas, se implementa la función expuesta en el bloque de código 4.7:

```

1 def exportar_aristas_similitud_caracteristicas(nodos, threshold):
2     df = pd.read_csv(nodos)
3
4     # Atributos para calcular la similitud
5     atributos = [
6         "CONSTRUCTEDAREA", "ROOMNUMBER", "BATHNUMBER", "HASTERRACE", "
7         HASLIFT",
8         "HASAIRCONDITIONING", "HASPARKINGSPACE", "HASNORTHORIENTATION", "
9         HASSOUTHORIENTATION",
10        "HASEASTORIENTATION", "HASWESTORIENTATION", "HASBOXROOM", "
11        HASWARDROBE",
12        "HASSWIMMINGPOOL", "HASDOORMAN", "HASGARDEN", "ISDUPLEX", "ISSTUDIO
13        ",
14        "ISINTOPFLOOR", "FLOORCLEAN"
15    ]
16    X = df[atributos].values
17    n_features = len(atributos)
18    dist_threshold = 1 - threshold
19    dist_threshold = dist_threshold * np.sqrt(n_features)
20
21    # Similitud euclidea
22    nn = NearestNeighbors(radius=dist_threshold, metric='euclidean', n_jobs
23    =-1)
24    nn.fit(X)
25    ind_array = nn.radius_neighbors(X, return_distance=False)
26
27    # Creacion aristas en base a la similitud
28    edge_rows = set()
29    n = len(X)
30    for i, neighbors in enumerate(ind_array):
31        for j in neighbors:
32            if i < j:
33                dist = np.linalg.norm(X[i] - X[j]) / np.sqrt(n_features)
34                similarity = 1 - dist
35                edge_rows.add((i, j, similarity))
36
37    # Detectar nodos aislados (sin vecinos)
38    vecinos = {i for edge in edge_rows for i in edge[:2]}
39    aislados = set(range(n)) - vecinos
40
41    # Buscar el nodo mas similar para nodos aislados
42    if aislados:
43        dists = np.linalg.norm(X[:, None, :] - X[None, :, :], axis=2) / np.
44        sqrt(n_features)
45        for i in aislados:
46            dists_i = dists[i].copy()

```

```

41         dists_i[i] = np.inf
42         j = np.argmin(dists_i)
43         similarity = 1 - dists_i[j]
44         edge_rows.add((min(i, j), max(i, j), similarity))
45
46     df_edges = pd.DataFrame(list(edge_rows), columns=["source", "target", "
SIMILARITY"])
47
48     # Formateo del threshold para nombres de archivo
49     threshold_str = f"{threshold:.3f}".replace('.', '').zfill(4)
50
51     # Salida de archivos
52     BASE_DIR = Path(__file__).resolve().parent
53     sim_dir = BASE_DIR / "graphs" / "edges" / "similarity" / f"{
threshold_str}_sim"
54     sim_dir.mkdir(parents=True, exist_ok=True)
55
56     # Ficheros de salida por barrio
57     barrios = df["NEIGHBOURID"].unique()
58     if len(barrios) == 1:
59         edges_out = sim_dir / f"Valencia_similarity_edges_{threshold_str}_{
barrios[0]}.csv"
60         df_edges.to_csv(edges_out, index=False, encoding='utf-8')
61         print(f"Aristas por similitud exportadas a {edges_out}")
62     else:
63         for barrio in barrios:
64             idxs = df[df["NEIGHBOURID"] == barrio].index.values
65             mask = df_edges["source"].isin(idxs) & df_edges["target"].isin(
idxs)
66             df_edges_barrio = df_edges[mask]
67             if not df_edges_barrio.empty:
68                 edges_out = sim_dir / f"Valencia_similarity_edges_{
threshold_str}_{barrio}.csv"
69                 df_edges_barrio.to_csv(edges_out, index=False, encoding='
utf-8')
70                 print(f"Aristas por similitud exportadas a {edges_out}")

```

Extracto de código 4.7: Función de creación de aristas por similitud

La función lleva el siguiente flujo de ejecución para generar las aristas del fichero de nodos:

1. Lee el fichero de nodos de un archivo CSV que contiene las viviendas de un barrio concreto.
2. Selecciona las características relevantes de la vivienda, formando un vector de características de cada inmueble.
3. Para que todas las características tengan el mismo peso y no domine uno sobre otra, se normalizan los valores de cada columna.
4. Se calcula la matriz de distancias, refiriéndose a distancia como la

similitud entre las viviendas. Esto se consigue por medio de la similitud euclídea.

5. Seguido de esto, se comienzan a generar las aristas, donde se tienen dos posibles casos para su creación:

- Para cada par de viviendas si la distancia es menor que el umbral impuesto, entonces significa que ha superado el umbral de similitud y por tanto se forma una arista ponderada entre las dos viviendas, donde el peso viene determinado por la similitud entre los domicilios.
- En caso de que alguna vivienda no supere el umbral impuesto, para evitar nodos aislados, se conectará igualmente a la vivienda con la que sea más similar.

6. Finalmente se guardan las aristas generadas en un fichero CSV, que indica el ID de los nodos que se relacionan y el peso entre los mismos.

Para tener diferentes variaciones del grafo, se generan distintos ficheros de aristas probando con varios umbrales, llegando a obtener hasta cinco umbrales distintos.

- **Aristas por barrio:** con esto, se pretende conseguir conexiones entre las viviendas en función de que pertenezcan al mismo barrio, es decir, los inmuebles que se encuentren en la misma barriada establecerán una arista no ponderada por cada domicilio del vecindario. Esto se obtiene por medio del bloque de código 4.8:

```
1 def exportar_aristas_barrio(nodos):
2     # Lectura del CSV
3     df = pd.read_csv(nodos)
4     edge_rows = []
5     barrios = df["NEIGHBOURID"].unique()
6
7     # Agrupar por barrio y exportar solo por barrio
8     for barrio in barrios:
9         group = df[df["NEIGHBOURID"] == barrio]
10        idx = group.index.to_numpy()
11        if len(idx) > 1:
12            comb = np.array(list(itertools.combinations(idx, 2)))
13            edge_rows = [{"source": i, "target": j} for i, j in comb]
14            df_edges = pd.DataFrame(edge_rows)
15
16            # Directorio de salida
17            BASE_DIR = Path(__file__).resolve().parent
18            edges_dir = BASE_DIR / "graphs" / "edges" / "neighbour"
19            edges_dir.mkdir(parents=True, exist_ok=True)
20
21            # Ficheros de salida por barrio
22            edges_out = edges_dir / f"Valencia_neighbour_edges_{barrio}.csv"
```

```

23         df_edges.to_csv(edges_out, index=False, encoding='utf-8')
24         print(f"Aristas exportadas a {edges_out}")
25     else:
26         print(f"Barrio {barrio} no tiene suficientes nodos para crear
aristas.")

```

Extracto de código 4.8: Función de creación de aristas por barrio

La función sigue los siguientes pasos hasta llegar a generar las aristas para el fichero de nodos:

1. Realiza una lectura del fichero de nodos pasado como argumento, a través del paquete pandas.
2. A continuación, agrupa a las viviendas por el campo de barrio y obtiene la lista de índices de los inmuebles que pertenecen a esa barriada.
3. Tras esto, se generan todas las posibles combinaciones de pares de viviendas dentro del barrio.
4. Finalmente guarda las aristas en un fichero CSV, donde cada fila representa al par de índices (viviendas) que forman una conexión.

Como resultado, se obtienen 12 archivos CSV de aristas formadas de distintas formas para los barrios, lo cual da lugar a un número considerable de grafos para trabajar en la siguiente fase para los modelos de *machine learning*.

Generación y representación

Una vez obtenidos los pares de nodos-aristas, es posible plantear la generación y representación de un grafo.

En el bloque de código 4.9, se expone la función desarrollada para la creación de grafos, usando el paquete de pytorch geometric:

```

1 def cargar_grafo(nodos, aristas, aristas_extra=None):
2     # Lectura de los CSV
3     nodes_df = pd.read_csv(nodos)
4     edges_df = pd.read_csv(aristas)
5
6     # Indicar nodos y características
7     node_features = nodes_df.drop(columns=["NODEID"]).values
8     x = torch.tensor(node_features, dtype=torch.float)
9
10    # Primer tipo de aristas
11    edge_index1 = torch.tensor(edges_df[["source", "target"]].values.T, dtype=
torch.long)
12    if edges_df.shape[1] > 2:
13        weight_col1 = edges_df.columns[2]
14        edge_attr1 = torch.tensor(edges_df[weight_col1].values, dtype=torch.
float).unsqueeze(1)

```

```

15     nombre_arista1 = f"{weight_col1.lower()}"
16 else:
17     edge_attr1 = None
18     nombre_arista1 = "vivienda_simple"
19
20 if aristas_extra is None:
21     if edge_attr1 is not None:
22         data = Data(x=x, edge_index=edge_index1, edge_attr=edge_attr1)
23     else:
24         data = Data(x=x, edge_index=edge_index1)
25     return data
26
27 # Segundo tipo de aristas
28 edges_df2 = pd.read_csv(aristas_extra)
29 edge_index2 = torch.tensor(edges_df2[["source", "target"]].values.T, dtype=
torch.long)
30 if edges_df2.shape[1] > 2:
31     weight_col2 = edges_df2.columns[2]
32     edge_attr2 = torch.tensor(edges_df2[weight_col2].values, dtype=torch.
float).unsqueeze(1)
33     nombre_arista2 = f"{weight_col2.lower()}"
34 else:
35     edge_attr2 = None
36     nombre_arista2 = "vivienda_simple"
37
38 data = HeteroData()
39 data['house'].x = x
40 data['house', nombre_arista1, 'house'].edge_index = edge_index1
41 if edge_attr1 is not None:
42     data['house', nombre_arista1, 'house'].edge_attr = edge_attr1
43 data['house', nombre_arista2, 'house'].edge_index = edge_index2
44 if edge_attr2 is not None:
45     data['house', nombre_arista2, 'house'].edge_attr = edge_attr2
46
47 return data

```

Extracto de código 4.9: Función para la creación de un grafo

A continuación, se muestran los pasos que se van siguiendo durante la ejecución de la función:

1. En primer lugar hace la lectura de un fichero de nodos, cargándolo por medio del paquete pandas como un objeto DataFrame. Cada nodo representa una vivienda y sus características.
2. Tras esto se realiza la lectura de las aristas, teniendo dos posibles casos llegados a este punto:
 - Si se proporciona únicamente un archivo de aristas, se generará un grafo solo para el par de nodos-aristas.

- En el caso de recibir un archivo de aristas extras por medio del segundo parámetro, se generará un grafo heterogéneo combinando los distintos tipos de aristas y pesos (en el caso de que fuesen ponderados).
3. Una vez conocidos los datos a transformar en un grafo se preparan los tensores, convirtiendo los atributos numéricos de los nodos a un tensor de *PyTorch* por medio del método `torch.tensor()`. De la misma forma, prepara las aristas.
 4. Tras estos preparativos se genera el objeto grafo, pudiendo darse dos casos distintos:
 - Si solo existe un tipo de aristas se crea un objeto de tipo `Data`, que representa a un grafo homogéneo (solo contempla un tipo de nodo y de arista).
 - Si existe diferentes tipos de aristas se creará entonces un objeto de tipo `HeteroData`, que permite tener distintos tipos de relaciones en un mismo grafo.
 5. Finalmente, se devuelve el grafo para que pueda ser usado directamente en los modelos de aprendizaje de la siguiente etapa del KDD o bien para representarlos gráficamente.

Junto a la función de creación de grafos se ha implementado además otra que permite visualizar los grafos generados. Esto se consigue gracias a los paquetes de `matplotlib` y `networkx`. A continuación, se exponen las líneas de código que permiten la representación en la fase de experimentación por medio del bloque de código 4.10:

```

1 def mostrar_grafo(data):
2     if isinstance(data, Data):
3         # Grafo homogéneo
4         edge_index = data.edge_index.cpu().numpy()
5         G = nx.Graph()
6         G.add_nodes_from(range(data.x.shape[0]))
7         edges = list(zip(edge_index[0], edge_index[1]))
8         if hasattr(data, 'edge_attr') and data.edge_attr is not None:
9             weights = data.edge_attr.cpu().numpy().flatten()
10            for (u, v), w in zip(edges, weights):
11                G.add_edge(u, v, weight=w)
12        else:
13            G.add_edges_from(edges)
14            plt.figure(figsize=(8, 8))
15            pos = nx.spring_layout(G, seed=42)
16            nx.draw(G, pos, with_labels=True, node_color='skyblue', edge_color='gray',
17                    node_size=300, font_size=8)
18            if nx.get_edge_attributes(G, 'weight'):
19                edge_labels = nx.get_edge_attributes(G, 'weight')
20                edge_labels = {k: f"{v:.2f}" for k, v in edge_labels.items()}
21                nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels,
22                                            font_color='red', font_size=8)

```

```

21     plt.show()
22 elif isinstance(data, HeteroData):
23     # Grafo heterogeneo
24     G = nx.Graph()
25     num_nodes = data['house'].x.shape[0]
26     G.add_nodes_from(range(num_nodes))
27     colors = ['gray', 'orange', 'green', 'blue', 'red', 'purple']
28     edge_types = list(data.edge_types)
29     for idx, edge_type in enumerate(edge_types):
30         edge_index = data[edge_type].edge_index.cpu().numpy()
31         edges = list(zip(edge_index[0], edge_index[1]))
32         color = colors[idx % len(colors)]
33         if hasattr(data[edge_type], 'edge_attr') and data[edge_type].
edge_attr is not None:
34             weights = data[edge_type].edge_attr.cpu().numpy().flatten()
35             for (u, v), w in zip(edges, weights):
36                 G.add_edge(u, v, weight=w, color=color)
37         else:
38             for (u, v) in edges:
39                 G.add_edge(u, v, color=color)
40     plt.figure(figsize=(8, 8))
41     pos = nx.spring_layout(G, seed=42)
42
43     # Diferencia los tipos de aristas por color
44     for idx, edge_type in enumerate(edge_types):
45         color = colors[idx % len(colors)]
46         edges = [(u, v) for (u, v, d) in G.edges(data=True) if d.get('color'
) == color]
47         nx.draw_networkx_edges(G, pos, edgelist=edges, edge_color=color,
label=str(edge_type))
48         nx.draw_networkx_nodes(G, pos, node_color='skyblue', node_size=300)
49         nx.draw_networkx_labels(G, pos, font_size=8)
50
51     # Dibuja pesos si existen
52     edge_labels = {(u, v): f"{d['weight']:.2f}" for u, v, d in G.edges(data=
True) if 'weight' in d}
53     if edge_labels:
54         nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels,
font_color='red', font_size=8)
55     plt.legend()
56     plt.show()
57 else:
58     raise ValueError("El objeto data debe ser de tipo Data o HeteroData.")

```

Extracto de código 4.10: Función para la representación de un grafo

La ejecución de la función mostrada, lleva a cabo los siguientes pasos:

1. En primer lugar diferencia el tipo de grafo recibido, comprobando antes de comenzar a representarlo si se trata de un grafo de tipo Data o HeteroData.

2. Seguido de esto se comienza a construir la representación gráfica haciendo uso de la librería `networkx`, teniendo los siguientes casos:
 - Si se trata de un grafo homogéneo entonces crea un grafo simple, añadiendo el par de nodos-aristas.
 - En el caso de un grafo heterogéneo deberá crear el grafo a través de la tripleta de nodos y las aristas. Este grafo tendrá un tratamiento de representación distinto, debido a que tiene diferentes aristas en su composición como grafo. Se actuará diferenciando las aristas por el tipo de relación y un color.
3. Tras identificar el grafo y buscar su representación se usa un algoritmo de disposición dado por el método `sprint_layout()`, que hace que en el gráfico a generar se coloquen los nodos según su mejor disposición, evitando así solapamientos entre viviendas y que sea visualmente claro.
4. Finalmente, se representa al grafo y se muestra por medio de `pyplot`.

La etapa de transformación de datos, finaliza entonces con tres formas distintas de representar las conexiones entre las viviendas.

Con las variaciones de los parámetros para las aristas de vecindad, similitud, barrios y la posibilidad de combinación entre estas, la fase cierra con 42 grafos distintos a usar en los modelos de *machine learning*.

4.2.4. Minería de datos

Tras la transformación de los datos tenemos una estructura bien preparada para trabajar en la siguiente etapa del proceso KDD, la minería de datos.

La etapa de minería de datos constituye el núcleo del proceso de descubrimiento del conocimiento, ya que es en este momento donde se aplican técnicas analíticas y algoritmos para la extracción de resultados y conocimiento para la posterior toma de decisiones sobre qué par de grafo-modelo será el más óptimo a implementar en el sistema de predicción de viviendas.

Modelos

Antes de comenzar a generar los resultados que se obtienen es fundamental describir los modelos empleados durante el proceso de entrenamiento.

En particular se han usado variantes de GCN ajustándolas para las dos variantes de grafos presentes en el contexto del problema. los grafos homogéneos y los heterogéneos. A continuación, se detallan cada uno de los modelos:

- **Convolución para grafos homogéneos:** se define la clase GCN para el correcto tratamiento de los grafos homogéneos. Esto viene dado por la clase GCN de *PyTorch Geometric*, que colabora en la implementación del bloque de código [4.11](#):

```

1 class GCN(torch.nn.Module):
2     def __init__(self, in_channels, hidden_channels, num_layers, dropout):
3         super(GCN, self).__init__()
4         self.dropout = dropout
5         self.num_layers = num_layers
6
7         self.convs = torch.nn.ModuleList()
8
9         # Primera capa
10        self.convs.append(GCNConv(in_channels, hidden_channels))
11
12        # Capas ocultas intermedias (si hubieran)
13        for _ in range(num_layers - 2):
14            self.convs.append(GCNConv(hidden_channels, hidden_channels))
15
16        # Ultima capa
17        self.convs.append(GCNConv(hidden_channels, 1))
18
19    def forward(self, data):
20        x, edge_index = data.x, data.edge_index
21        edge_weight = data.edge_attr.squeeze() if data.edge_attr is not
22        None else None
23        for i, conv in enumerate(self.convs):
24            x = conv(x, edge_index, edge_weight=edge_weight)
25            if i < len(self.convs) - 1:
26                x = F.relu(x)
27                x = F.dropout(x, p=self.dropout, training=self.training)
28        return x.squeeze()

```

Extracto de código 4.11: Clase del modelo GCN homogéneo

La clase representa cómo se inicializa y se procesan los datos a través del modelo.

El entrenamiento del modelo GCN homogéneo se realiza por medio de las líneas de código mostradas en el bloque 4.12:

```

1 def train_node_regression(data, price_index, epochs, lr, hidden_channels,
2   num_layers, dropout):
3     # No entrena si hay menos de 10 nodos
4     if data.num_nodes < 10:
5         return float('nan'), float('nan'), float('nan')
6
7     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
8     model = GCN(data.num_features, hidden_channels, num_layers, dropout).to(
9       device)
10    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
11    data = data.to(device)
12
13    num_nodes = data.num_nodes

```

```

12     idx = torch.randperm(num_nodes)
13     split = int(0.8 * num_nodes)
14     train_idx = idx[:split]
15     test_idx = idx[split:]
16
17     y = data.x[:, price_index]
18
19     for epoch in range(epochs):
20         model.train()
21         optimizer.zero_grad()
22         out = model(data)
23         loss = F.mse_loss(out[train_idx], y[train_idx])
24         loss.backward()
25         optimizer.step()
26
27     # Evaluacion
28     model.eval()
29     with torch.no_grad():
30         out = model(data)
31         y_true = y[test_idx].cpu().numpy()
32         y_pred = out[test_idx].cpu().numpy()
33         mse = mean_squared_error(y_true, y_pred)
34         mae = mean_absolute_error(y_true, y_pred)
35         r2 = r2_score(y_true, y_pred)
36
37     return mse, mae, r2

```

Extracto de código 4.12: Función de entrenamiento para GCN homogéneo

El flujo que sigue la función de entrenamiento implementada es el siguiente:

1. En primer lugar recibe los argumentos principales para el entrenamiento, que se describen como:
 - **model:** se trata del modelo GCN a entrenar.
 - **data:** se trata del objeto de *PyTorch Geometric* que contiene los datos del grafo (nodos, aristas y los atributos independientes y objetivo de la predicción).
 - **Optimizador:** el optimizador de *PyTorch* a usar durante el entrenamiento.
 - **Función de pérdida:** ofrecida por el paquete funcional de *PyTorch NN*.
 - **Máscara:** máscara booleana que sirve para indicar qué nodos usar durante el entrenamiento.
 - **Hiperparámetros:** los pasados por parámetro para el entrenamiento.
2. Se indica que el modelo se pone en modo entrenamiento por medio del método `train()` y se restablecen los gradientes con `.zero_grad()`.

3. Realiza la propagación hacia delante (*foward*) calculando las predicciones del modelo para todos los nodos.
4. Calcula la pérdida de los nodos seleccionados por la máscara booleana.
5. Propaga el error hacia atrás (*backward*) por medio del método *backward()*.
6. Actualiza los pesos de modelo por medio de la línea de código `optimizer.step()`.
7. Finalmente devuelve las métricas de evaluación, cuyos detalles se conocerán en la siguiente sección del capítulo.

- **Convolución para grafos heterogéneos:** se implementa una clase que define el modelo de red neuronal que trabaja con grafos heterogéneos usando *PyTorch Geometric*. Esto es para aquellos grafos que poseen distintas aristas dentro del grafo y en el contexto del problema que se plantea, serán las distintas combinaciones de aristas de similitud y vecindad que se pueden complementar para la creación del grafo. A continuación, se expone el bloque de código 4.13 relativo al modelo implementado:

```

1 class HeteroGNN(torch.nn.Module):
2     def __init__(self, metadata, hidden_channels, num_layers, dropout):
3         super().__init__()
4         self.dropout = dropout
5         self.num_layers = num_layers
6
7         # Construye una lista de HeteroConv para multiples capas
8         self.convs = torch.nn.ModuleList()
9
10        # Primera capa
11        self.convs.append(HeteroConv({
12            edge_type: GCNConv(-1, hidden_channels)
13            for edge_type in metadata[1]
14        }, aggr='sum'))
15
16        # Capas ocultas intermedias (si hubieran)
17        for _ in range(num_layers - 2):
18            self.convs.append(HeteroConv({
19                edge_type: GCNConv(hidden_channels, hidden_channels)
20                for edge_type in metadata[1]
21            }, aggr='sum'))
22
23        # Ultima capa
24        self.convs.append(HeteroConv({
25            edge_type: GCNConv(hidden_channels, hidden_channels)
26            for edge_type in metadata[1]
27        }, aggr='sum'))
28        self.lin = torch.nn.Linear(hidden_channels, 1)
29
30    def forward(self, data):
31        x_dict = data.x_dict

```

```

32     edge_index_dict = data.edge_index_dict
33     for i, conv in enumerate(self.convs):
34         x_dict = conv(x_dict, edge_index_dict)
35         if i < len(self.convs) - 1: # Aplica activacion y dropout en
las capas intermedias
36             for node_type in x_dict:
37                 x_dict[node_type] = F.relu(x_dict[node_type])
38                 x_dict[node_type] = F.dropout(x_dict[node_type], p=self
.dropout, training=self.training)
39         x = x_dict['house']
40         x = self.lin(x)
41         return x.squeeze()

```

Extracto de código 4.13: Clase del modelo GCN heterogéneo

La clase representa cómo se deben inicializar y procesar los datos por medio del modelo.

El entrenamiento del modelo GCN heterogéneo se realiza por medio de la función 4.14:

```

1 def train_node_regression_hetero(data, price_index, epochs, lr,
hidden_channels, num_layers, dropout):
2     # No entrena si hay menos de 10 nodos
3     if data['house'].num_nodes < 10:
4         print(f"Saltando grafo con solo {data['house'].num_nodes} nodos.")
5         return None
6
7     device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
8     model = HeteroGNN(data.metadata(), hidden_channels, num_layers, dropout
).to(device)
9     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
10    data = data.to(device)
11
12    num_nodes = data['house'].num_nodes
13    idx = torch.randperm(num_nodes)
14    split = int(0.8 * num_nodes)
15    train_idx = idx[:split]
16    test_idx = idx[split:]
17
18    y = data['house'].x[:, price_index]
19
20    for epoch in range(epochs):
21        model.train()
22        optimizer.zero_grad()
23        out = model(data)
24        loss = F.mse_loss(out[train_idx], y[train_idx])
25        loss.backward()
26        optimizer.step()
27
28    # Evaluacion del modelo

```

```

29     model.eval()
30     with torch.no_grad():
31         out = model(data)
32         y_true = y[test_idx].cpu().numpy()
33         y_pred = out[test_idx].cpu().numpy()
34         mse = mean_squared_error(y_true, y_pred)
35         mae = mean_absolute_error(y_true, y_pred)
36         r2 = r2_score(y_true, y_pred)
37
38     return mse, mae, r2

```

Extracto de código 4.14: Función de entrenamiento para GCN heterogéneo

El flujo de trabajo que lleva la función tras su ejecución es el siguiente:

1. En primer lugar recibe los argumentos principales para la definición del entrenamiento, que se describen como:
 - **model:** se trata del modelo GCN adaptado a grafos heterogéneos que se va a entrenar.
 - **data:** para este caso el objeto que recibe la función será un objeto *HeteroData* de *PyTorch Geometric*, que representa al grafo y todas las características de los nodos y aristas.
 - **Optimizador:** se trata del optimizador que se usará durante el entrenamiento de predicción. En este caso se usa *Adam*, ofrecido por el paquete de *PyTorch*.
 - **Máscara:** se trata de una máscara booleana que sirve para indicar qué nodos se usan en el entrenamiento.
 - **Hiperparámetros:** son los configurados y pasados por parámetro en la función para el entrenamiento.
2. Se le indica al modelo que deberá activar su función de entrenamiento por medio del método `train()` y se restablecen los gradientes con `zero_grad()`.
3. Realiza la propagación hacia delante (*forward*) calculando las predicciones del modelo para todos los nodos.
4. Calcula la pérdida de los nodos seleccionados por la máscara booleana del modelo.
5. Propaga el error hacia detrás (*backward*).
6. Actualiza los pesos del modelo por medio de la línea de código `optimizer.step()`.
7. Finalmente, devuelve las métricas de evaluación que evalúan el trabajo del modelo durante el entrenamiento.

Las funciones de entrenamiento además tienen en cuenta la regla de negocio RN-021, renunciando a entrenar aquellos grafos que no posean al menos 10 viviendas. Esto ayuda a evitar recibir valores *NaN* en la evaluación de los modelos al poseer poca información como para realizar una predicción.

Finalmente se preparan dos convoluciones de grafos, detalladas en distintas clases para que trabajen de forma independiente en función del grafo que se reciba.

4.2.5. Evaluación e interpretación

Tras finalizar la etapa de implementación de los modelos, es posible pasar a la penúltima etapa para la extracción del conocimiento.

Con los modelos bien definidos en la etapa de minería de datos es necesario evaluar su calidad y comprender qué significan los resultados que devuelven.

De esto se encarga la etapa que cierra el proceso KDD, que resulta esencial para:

- Confirmar que los resultados devueltos por los modelos tienen sentido desde el punto de vista del dominio, esto es, que devuelven resultados lógicos dentro del contexto del problema que se presenta.
- Medir el rendimiento del modelo a través de métricas de evaluación.
- Traducir los resultados de estas evaluaciones en conocimiento útil para la toma de decisiones final de la aplicación de predicción a desarrollar.

Al tratarse de modelos de aprendizaje automático que realizan una tarea de predicción, se utilizan una serie de métricas específicas que cuantifican la discrepancia entre las predicciones realizadas por GNN y los valores reales.

Las métricas que evaluarán a los modelos del problema que se plantea son las siguientes:

- **Error Cuadrático Medio (MSE):** la métrica mide el promedio de los errores al cuadrado entre las predicciones del modelo y los valores reales, penalizando de forma más fuerte a aquellos errores grandes, potenciado por el cuadrado.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (4.10)$$

Donde:

- n es el número de ejemplos de prueba.
- y_i es el valor real del ejemplo i .
- $\hat{y}_i$ es el valor predicho por el modelo.

Un valor bajo del MSE indica que el rendimiento del modelo es bueno para esos datos en la predicción.

- **Error Absoluto Medio (MAE):** mide la media de las diferencias absolutas entre los valores reales y los predichos. A diferencia de MSE, esta métrica no penaliza de forma más fuerte a los errores grandes.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.11)$$

- **Coefficiente de determinación (R^2):** indica en qué proporción la varianza de la variable dependiente es explicada por el modelo. Su valor se comprende entre $-\infty$ y 1, siendo 1 el valor ideal de la métrica.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (4.12)$$

Donde:

- $\bar{y}$ es la media de los valores reales.

Un R^2 cercano a 1 es un buen indicativo de que el modelo explica bien la variabilidad del dato.

Grafos homogéneos

En total, para el modelo de grafos homogéneos definido, se ha entrenado 12 veces usando los siguientes grafos y parámetros:

- **Vecindad:** grafos definidos por un radio como argumento, que relaciona viviendas según se encuentren bajo ese rango o no, con posibilidad de establecer una arista al grafo más cercano aún sin superar el umbral para evitar nodos aislados. Se han generado grafos usando los siguientes radios:
 - Radio de 50 metros.
 - Radio de 100 metros.
 - Radio de 150 metros.
 - Radio de 200 metros.
 - Radio de 250 metros.
 - Radio de 300 metros.
- **Similitud:** grafos definidos por un umbral como argumento, que relaciona viviendas en función de si su similitud de características es alto, con posibilidad de establecer una conexión al más similar si se trata de nodos que se fuesen a quedar aislados. Los grafos generados han tomado los siguientes umbrales:
 - Umbral de 0.8.
 - Umbral de 0.85.

- Umbral de 0.9.
 - Umbral de 0.95.
 - Umbral de 0.99.
- **Barriada:** grafo que genera aristas en función de si dos viviendas se encuentran en el mismo barrio, sin pesos añadidos.

Estos grafos han sido procesados y entrenados por medio de la siguiente función:

```

1 def evaluar_gcn_por_pares(nodos_dir, aristas_dir, price_index=0):
2     nodos_dir = Path(nodos_dir)
3     aristas_dir = Path(aristas_dir)
4     resultados = []
5
6     # Directorio y fichero de salida
7     edges_name = aristas_dir.name
8     eval_dir = Path(__file__).resolve().parent / "evaluation" / "gcn"
9     eval_dir.mkdir(parents=True, exist_ok=True)
10    output_file = eval_dir / f"model_gcn_evaluation_{edges_name}.txt"
11    pesos_dir = eval_dir / "pesos"
12    pesos_dir.mkdir(parents=True, exist_ok=True)
13
14    id_pattern = re.compile(r"_(\d+)\.csv$")
15
16    # Obtencion de pares de nodos y aristas
17    def extraer_id(nombre):
18        m = id_pattern.search(nombre)
19        return m.group(1) if m else None
20
21    nodos_files = {}
22    for f in nodos_dir.glob("*.csv"):
23        id_ = extraer_id(f.name)
24        if id_ is not None:
25            nodos_files[id_] = f
26
27    aristas_files = {}
28    for f in aristas_dir.glob("*.csv"):
29        id_ = extraer_id(f.name)
30        if id_ is not None:
31            aristas_files[id_] = f
32
33    ids_comunes = sorted(set(nodos_files.keys()) & set(aristas_files.keys()),
34                          key=lambda x: int(x))
35    print(f"IDs comunes encontrados: {len(ids_comunes)}")
36    with open(output_file, "w", encoding="utf-8") as f:
37        for id_ in ids_comunes:
38            nodo_path = nodos_files[id_]
39            arista_path = aristas_files[id_]
40            try:
41                data = cargar_grafo(str(nodo_path), str(arista_path))

```

```

41
42 # Búsqueda de hiperparametros optimos con Optuna
43 def objective(trial):
44     hidden_channels = trial.suggest_int("hidden_channels", 8, 12
8)
45     num_layers = trial.suggest_int("num_layers", 2, 5)
46     dropout = trial.suggest_float("dropout", 0.0, 0.7)
47     lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
48     epochs = trial.suggest_int("epochs", 50, 200)
49     mse, _, _ = train_node_regression(
50         data,
51         price_index=price_index,
52         epochs=epochs,
53         lr=lr,
54         hidden_channels=hidden_channels,
55         num_layers=num_layers,
56         dropout=dropout
57     )
58     return mse if not math.isnan(mse) else float('inf')
59
60 study = optuna.create_study(direction="minimize")
61 study.optimize(objective, n_trials=10, show_progress_bar=False)
62 best_params = study.best_params
63
64 # Entrenamiento por medio de los mejores hiperparametros
65 result = train_node_regression(
66     data,
67     price_index=price_index,
68     epochs=best_params["epochs"],
69     lr=best_params["lr"],
70     hidden_channels=best_params["hidden_channels"],
71     num_layers=best_params["num_layers"],
72     dropout=best_params["dropout"]
73 )
74 if result is not None and not any(math.isnan(x) for x in result)
:
75     mse, mae, r2 = result
76     resultados.append((float(mse), float(mae), float(r2)))
77     # Entrena el modelo final para guardar los pesos
78     model = None
79     try:
80         device = torch.device('cuda' if torch.cuda.is_available
() else 'cpu')
81         model = GCN(
82             data.num_features,
83             best_params["hidden_channels"],
84             best_params["num_layers"],
85             best_params["dropout"]
86         ).to(device)
87         optimizer = torch.optim.Adam(model.parameters(), lr=

```

```

88     best_params["lr"])
89     data_device = data.to(device)
90     y = data_device.x[:, price_index]
91     idx = torch.randperm(data_device.num_nodes)
92     split = int(0.8 * data_device.num_nodes)
93     train_idx = idx[:split]
94     for epoch in range(best_params["epochs"]):
95         model.train()
96         optimizer.zero_grad()
97         out = model(data_device)
98         loss = torch.nn.functional.mse_loss(out[train_idx],
99         y[train_idx])
100         loss.backward()
101         optimizer.step()
102         # Pesos del entrenamiento
103         pesos_path = pesos_dir / f"pesos_gcn_{id_}.pt"
104         torch.save(model.state_dict(), pesos_path)
105     except Exception as e:
106         print(f"Error guardando pesos para ID {id_}: {e}")
107         f.write(f"Grafo {id_}:\n")
108         f.write(f"Parametros usados: {best_params}\n")
109         f.write(f"Metricas: MSE={mse:.4f}, MAE={mae:.4f}, R2={r2:.4f}
110         }\n\n")
111         print(f"Evaluado ID {id_}: MSE={mse:.4f}, MAE={mae:.4f}, R2
112         ={r2:.4f} | Hiperparametros: {best_params}")
113     else:
114         print(f"Saltando ID {id_} por metricas NaN o grafo pequeno.")
115     except Exception as e:
116         print(f"Error con ID {id_}: {e}")
117
118     # Calculo de media de metricas de todos los pares validos
119     if resultados:
120         mean_mse = sum(r[0] for r in resultados) / len(resultados)
121         mean_mae = sum(r[1] for r in resultados) / len(resultados)
122         mean_r2 = sum(r[2] for r in resultados) / len(resultados)
123         f.write(f"MEDIA DE METRICAS:\n")
124         f.write(f"MSE={mean_mse:.4f}, MAE={mean_mae:.4f}, R2={mean_r2:.4f}\n
125         ")
126         print(f"Media - MSE: {mean_mse:.4f}, MAE: {mean_mae:.4f}, R2: {
127         mean_r2:.4f}")
128     else:
129         print("No se encontraron pares validos de nodos y aristas.")

```

Extracto de código 4.15: Evaluación por pares en GCN homogéneo

Esta función será la encargada de evaluar el rendimiento del modelo GCN sobre diferentes pares de archivos de nodos y aristas. Su flujo de ejecución sigue los siguientes pasos:

1. En primer lugar busca el par de nodos y aristas correspondiente, a través del

identificador del barrio.

2. Seguido de esto, construye el grafo que se usará para el entrenamiento.
3. Carga e inicializa el modelo buscando los hiperparámetros más adecuados por medio de optuna, librería que se encarga de realizar varias iteraciones para encontrar los hiperparámetros que mejor se ajusten al modelo y al problema que se presenta.
4. Tras el entrenamiento se evalúa el modelo y se calculan las métricas de evaluación (MSE, MAE y R^2).
5. Finalmente se guardan los resultados por cada par de nodos aristas de los barrios, generando lo siguiente:
 - **Fichero TXT:** se trata de un fichero de texto plano que contiene los hiperparámetros usados para cada par, además de las métricas de evaluación que ha devuelto su entrenamiento, visualizando de una forma más clara los resultados devueltos. Al final del fichero se devuelve una media de las métricas, que indican como funciona de forma general para los distintos grafos de barrios el entrenamiento realizado.
 - **Ficheros PT:** se tratan de los ficheros que contienen los pesos de cada par de grafo entrenado. Esto se realiza ya que en caso de el modelo sea seleccionado para ser implementado en la aplicación web, no se deberá volver a entrenar el algoritmo, sino que directamente se aplicarán los pesos usados durante su entrenamiento, para de esta forma no volver a tener que pasar por el mismo proceso.

Grafos heterogéneos

En relación a los grafos heterogéneos que se pueden entrenar, se cuenta con hasta 30 variantes distintas, resultantes de las distintas combinaciones de tipo de aristas y parámetros que definen a las mismas. Estos grafos son trabajados y evaluados por medio del siguiente bloque de código:

```
1 def evaluar_hetero_por_pares(nodos_dir, aristas_dir1, aristas_dir2, price_index=  
2     0):  
3     nodos_dir = Path(nodos_dir)  
4     aristas_dir1 = Path(aristas_dir1)  
5     aristas_dir2 = Path(aristas_dir2)  
6     resultados = []  
7  
8     # Ruta y nombre del fichero de salida  
9     edges_name1 = aristas_dir1.name  
10    edges_name2 = aristas_dir2.name  
11    eval_dir = Path(__file__).resolve().parent / "evaluation" / "heterognn"  
12    eval_dir.mkdir(parents=True, exist_ok=True)  
13    output_file = eval_dir / f"model_hetero_evaluation_{edges_name1}_{  
    edges_name2}.txt"  
    pesos_dir = eval_dir / "pesos"
```

```

14 pesos_dir.mkdir(parents=True, exist_ok=True)
15
16 id_pattern = re.compile(r"_(\d+)\.csv$")
17
18 # Obtencion de pares de nodos y aristas
19 def extraer_id(nombre):
20     m = id_pattern.search(nombre)
21     return m.group(1) if m else None
22
23 nodos_files = {}
24 for f in nodos_dir.glob("*.csv"):
25     id_ = extraer_id(f.name)
26     if id_ is not None:
27         nodos_files[id_] = f
28
29 aristas_files1 = {}
30 for f in aristas_dir1.glob("*.csv"):
31     id_ = extraer_id(f.name)
32     if id_ is not None:
33         aristas_files1[id_] = f
34
35 aristas_files2 = {}
36 for f in aristas_dir2.glob("*.csv"):
37     id_ = extraer_id(f.name)
38     if id_ is not None:
39         aristas_files2[id_] = f
40
41 # Emparejamiento por pares de nodos y aristas
42 ids_comunes = sorted(set(nodos_files.keys()) & set(aristas_files1.keys()) &
43 set(aristas_files2.keys()), key=lambda x: int(x))
44 print(f"IDs comunes encontrados: {len(ids_comunes)}")
45 with open(output_file, "w", encoding="utf-8") as f:
46     for id_ in ids_comunes:
47         nodo_path = nodos_files[id_]
48         arista_path1 = aristas_files1[id_]
49         arista_path2 = aristas_files2[id_]
50         try:
51             data = cargar_grafo(str(nodo_path), str(arista_path1), str(
52 arista_path2))
53
54         # Busqueda de hiperparametros optimos con Optuna
55         def objective(trial):
56             hidden_channels = trial.suggest_int("hidden_channels", 8, 12
57 8)
58             num_layers = trial.suggest_int("num_layers", 2, 5)
59             dropout = trial.suggest_float("dropout", 0.0, 0.7)
60             lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
61             epochs = trial.suggest_int("epochs", 50, 200)
62             result = train_node_regression_hetero(
63                 data,

```

```

61         price_index=price_index,
62         epochs=epochs,
63         lr=lr,
64         hidden_channels=hidden_channels,
65         num_layers=num_layers,
66         dropout=dropout
67     )
68     mse = result[0] if result is not None else float('inf')
69     return mse if not math.isnan(mse) else float('inf')
70
71     study = optuna.create_study(direction="minimize")
72     study.optimize(objective, n_trials=10, show_progress_bar=False)
73     best_params = study.best_params
74
75     # Entrenamiento por medio de los mejores hiperparametros
76     result = train_node_regression_hetero(
77         data,
78         price_index=price_index,
79         epochs=best_params["epochs"],
80         lr=best_params["lr"],
81         hidden_channels=best_params["hidden_channels"],
82         num_layers=best_params["num_layers"],
83         dropout=best_params["dropout"]
84     )
85     if (
86         result is not None and
87         not any(math.isnan(x) for x in result)
88     ):
89         mse, mae, r2 = result
90         resultados.append((float(mse), float(mae), float(r2)))
91         # Entrena el modelo final para guardar los pesos
92         model = None
93         try:
94             device = torch.device('cuda' if torch.cuda.is_available
95         () else 'cpu')
96
97         model = HeteroGNN(
98             data.metadata(),
99             best_params["hidden_channels"],
100             best_params["num_layers"],
101             best_params["dropout"]
102         ).to(device)
103         optimizer = torch.optim.Adam(model.parameters(), lr=
104         best_params["lr"])
105
106         data_device = data.to(device)
107         y = data_device['house'].x[:, price_index]
108         idx = torch.randperm(data_device['house'].num_nodes)
109         split = int(0.8 * data_device['house'].num_nodes)
110         train_idx = idx[:split]
111         for epoch in range(best_params["epochs"]):
112             model.train()

```

```

109         optimizer.zero_grad()
110         out = model(data_device)
111         loss = torch.nn.functional.mse_loss(out[train_idx],
y[train_idx])
112         loss.backward()
113         optimizer.step()
114         # Pesos del entrenamiento
115         pesos_path = pesos_dir / f"pesos_hetero_{id_}.pt"
116         torch.save(model.state_dict(), pesos_path)
117     except Exception as e:
118         print(f"Error guardando pesos para ID {id_}: {e}")
119         f.write(f"Grafo {id_}:\n")
120         f.write(f"Parametros usados: {best_params}\n")
121         f.write(f"Metricas: MSE={mse:.4f}, MAE={mae:.4f}, R2={r2:.4f}
\n\n")
122         print(f"Evaluado ID {id_}: MSE={mse:.4f}, MAE={mae:.4f}, R2
={r2:.4f} | Hiperparametros: {best_params}")
123     else:
124         print(f"Saltando ID {id_} por metricas NaN o grafo pequeno.")
125     except Exception as e:
126         print(f"Error con ID {id_}: {e}")
127
128     # Calculo de media de metricas de todos los pares validos
129     if resultados:
130         mean_mse = sum(r[0] for r in resultados) / len(resultados)
131         mean_mae = sum(r[1] for r in resultados) / len(resultados)
132         mean_r2 = sum(r[2] for r in resultados) / len(resultados)
133         f.write(f"MEDIA DE METRICAS:\n")
134         f.write(f"MSE={mean_mse:.4f}, MAE={mean_mae:.4f}, R2={mean_r2:.4f}\n
")
135         print(f"Media - MSE: {mean_mse:.4f}, MAE: {mean_mae:.4f}, R2: {
mean_r2:.4f}")
136     else:
137         print("No se encontraron pares validos de nodos y aristas.")

```

Extracto de código 4.16: Evaluación por pares en GCN heterogéneo

Esta función será la encargada de evaluar el rendimiento que ofrece un modelo de red neuronal con grafos heterogéneos sobre los distintos pares de nodos y aristas que recibe. Su ejecución lleva el siguiente flujo:

1. En primer lugar buscar la triplete de nodos y aristas correspondiente, a través del identificador del barrio.
2. Seguido de esto, construye el grafo heterogéneo a través de los nodos y los dos pares de arista recibidos por parámetro.
3. Carga e inicializa el modelo, realizando previamente una búsqueda de los hiperparámetros que resultarán más adecuados para el grafo y el modelo. Estos se obtienen por medio del paquete optuna, que se encarga de realizar varias iteraciones con la intención de ajustar los hiperparámetros lo máximo

posible para sacarle partido a sus valores durante el entrenamiento.

4. Tras realizar el entrenamiento se evalúa el modelo y se calculan sus consecuentes métricas de evaluación (MSE, MAE y R^2).
5. Finalmente se guardan los resultados por cada tripleta de nodos y aristas correspondientes, generando los siguientes archivos:
 - **Fichero TXT:** se trata de un fichero de texto plano que muestra información del entrenamiento, como pueden ser los hiperparámetros seleccionados o las métricas devueltas tras la evaluación. Habrá un bloque de información por cada grafo de barrio. Al final del fichero se incluye además una media de las métricas entre todos los barrios participantes.
 - **Fichero PT:** se tratan de los ficheros que contienen los pesos de cada grafo entrenado.

4.2.6. Resultados

Tras evaluar a todos los pares de modelo-grafo es posible mostrar las métricas de evaluación resultantes. Para ello, se exponen una serie de tablas las cuales contienen la media de las métricas del conjunto de barrios, lo que da una idea conjunta de cómo funciona la predicción.

Es por ello que esta sección está dedicada a exponer los resultados de los pares entrenados, comparando por tipos y los posibles parámetros establecidos a los grafos, para finalmente exponer el mejor de cada modelo entrenado.

Resultados modelo homogéneo

El modelo homogéneo ha sido entrenado un total de 12 veces, donde se han usado:

- **Vecindad:** hasta 6 grafos, con distintos radios establecidos como parámetro.
- **Similitud:** hasta 5 grafos, probando con distintos umbrales del parámetro.
- **Barriada:** únicamente un grafo.

Es posible conocer la media de métricas devueltas por el conjunto de barrios entrenados para cada grafo a través de la tabla [4.3](#), la cuál resalta además el grafo que ha conseguido devolver mejores resultados:

Grafo	Parámetro	MSE	MAE	R ²
Vecindad	0.05 km	43055.61	129.59	0.85
Vecindad	0.1 km	203179.59	314.70	0.33
Vecindad	0.15 km	378049.63	439.04	-0.17
Vecindad	0.2 km	582765.86	539.20	-0.92
Vecindad	0.25 km	450936.35	499.31	-0.73
Vecindad	0.3 km	490464.34	502.80	-0.66
Similitud	0.8 umbral	581137.55	583.94	-1.29
Similitud	0.85 umbral	546060.29	551.31	-0.90
Similitud	0.9 umbral	532005.49	550.16	-1.23
Similitud	0.95 umbral	489189.79	533.20	-1.27
Similitud	0.99 umbral	295542.08	407.63	-0.05
Barriada	No procede	479559.51	477.81	-0.59

Tabla 4.3: Media de las métricas del modelo de convolución homogéneo

La tabla 4.3 proporciona información relevante para la resolución del problema de predicción, a partir de la cual pueden extraerse las siguientes observaciones:

- **Grafos por vecindad:** los mejores resultados los devuelve el grafo por vecindad con un radio de 0,50m por parámetro, al devolver muy bajo error de *MSE* y *MAE* y alto *R²*. Además a medida que aumenta el radio de vecindad, se observa que el rendimiento del modelo empeora considerablemente dejando caer valores de *R²* incluso a valores negativos, lo que indica que el modelo es peor que una media constante. Esto lleva a sugerir radios de vecindad pequeños para captar relaciones locales útiles, puesto que radios mayores introducen ruido a la predicción.
- **Grafos por similitud:** de forma general los resultados son bastantes malos, con valores *MSE* y *MAE* altos y *R²* negativos. De igual forma se destaca el grafo con umbral de similitud a 0,99, al ser el parámetro que más reduce el error generado para las relaciones de similitud. Comparando los resultados devueltos por la similitud euclídea, podemos concluir con el hecho de que umbrales bajos generar conexiones poco útiles en el contexto del problema, y solo cuando se exige una similitud muy alta mejora la calidad del entrenamiento
- **Grafo por barriada:** el grafo de estas características realiza un desempeño intermedio durante el entrenamiento, devolviendo métricas desfavorables, pero ligeramente mejores que la mayoría de grafos de similitud. Esto lleva a pensar que probablemente el modelo capte relaciones este grafo de forma más estructurada, aunque menos específica que la vecindad física.

Interpretados los resultados medios del conjunto de barrios en cada par modelo-grafo, resulta importante mostrar y comentar por medio del cuadro 4.4 los

resultados por barrio devueltos por el mejor grafo homogéneo entrenado, donde se resalta aquel barrio que devuelve la mejor evaluación:

Barrio	MSE	MAE	R ²
Torrefiel	26233.24	115.36	0.88
Morvedre	119835.07	246.52	0.50
Trinitat	13129.41	81.36	0.93
Campanar	16110.91	104.78	0.94
El Carme	123514.22	254.93	0.75
La Vega Baixa	5111.83	53.84	0.96
La Xerea	115009.79	255.70	0.86
L'Illa Perduda	23342.95	112.76	0.88
Tres Forques	105826.17	93.82	0.56
Patraix	51674.06	168.56	0.70
Russafa	137277.82	265.41	0.70
La Creu del Grau	39621.45	141.44	0.86
Sant Pau	21703.20	107.07	0.95
La Seu	68147.42	191.64	0.92
Aiora	26954.08	131.00	0.81
Natzaret	7708.93	63.27	0.92
La Creu Coberta	15714.60	85.93	0.88
L'Amistat	9604.47	77.65	0.97
El Grau	103824.17	275.46	0.68
La Carrasca	16497.63	58.37	0.94
Beniferri	2260.08	32.56	0.99
L'Hort de Senabre	17843.54	94.68	0.90
Ciutat Jordi	2312.02	40.62	0.98
Cami de Vera	9864.62	75.61	0.96
Vara de Quart	3477.37	40.99	0.96
Soternes	6971.62	59.52	0.95
La Font Santa	4175.28	51.92	0.96
El Botanic	167465.67	265.38	0.73
Favara	30819.16	120.26	0.85
Cami Real	7664.21	67.57	0.91
La Fonteta S.Lluis	3203.45	33.78	0.96
La Punta	50455.60	114.66	0.88
Sant Llorens	7943.10	53.44	0.98
Ciutat Fallera	2291.43	31.19	0.94
Nou Moles	21281.09	105.47	0.86

Barrio	MSE	MAE	R²
La Llum	6266.07	45.94	0.95
La Malva-Rosa	23295.25	108.86	0.88
El Calvari	12601.45	91.36	0.89
Les Tendetes	35544.55	169.44	0.64
Ciutat Universitaria	54129.91	207.67	0.73
El Mercat	170052.15	341.56	0.73
La Roqueta	74442.85	188.32	0.87
La Gran Via	65100.03	187.86	0.89
Cami Fondo	19665.42	104.66	0.87
La Raiosa	14639.55	87.92	0.87
Safranar	3864.04	38.14	0.94
Sant Isidre	7006.91	64.05	0.90
Malilla	19393.61	107.90	0.90
Benicalap	14921.79	80.77	0.96
Jaume Roig	65443.27	221.62	0.78
La Petxina	54110.14	150.75	0.78
Sant Marcel·li	11241.55	87.27	0.93
Montolivet	20303.08	103.39	0.91
Ciutat de les Arts i de les Ciències	20620.34	104.09	0.92
Na Rovella	29173.29	129.16	0.88
Tormos	27006.50	102.01	0.91
Sant Antoni	23156.06	109.14	0.83
Benimaclet	18468.36	108.07	0.94
Marxalenes	9774.83	64.25	0.92
Exposicio	52463.11	189.48	0.95
Betero	4947.64	54.23	0.96
Cabanyal-Canyamelar	27818.28	121.17	0.86
Mestalla	21267.83	97.90	0.98
El Pilar	61260.97	162.27	0.84
El Pla del Remei	161846.70	226.40	0.91
Albors	13551.75	75.44	0.95
Arrancapins	23196.99	107.06	0.90
En Corts	306992.93	490.56	-0.48
Poble Nou	60268.75	200.83	0.78
Sant Francesc	90375.22	225.51	0.90
Penya-Roja	50514.23	150.76	0.93

Tabla 4.4: Evaluación del mejor grafo homogéneo

En la tabla 4.4 se puede observar que a excepción de las métricas de evaluación devueltas por el barrio En Corts cuyo R^2 es negativo, el resto de barrios devuelven buenos resultados. Destaca la barriada de **Beniferri**, que con viviendas que rondan los 3800 - 593 euros el precio del metro cuadrado, devuelve un error de 32,56€, lo que supone un porcentaje de error mínimo del 1,48 %.

Resultados modelo heterogéneo

El modelo heterogéneo ha sido entrenado un total de 30 veces, a través de las distintas combinaciones de grafos entre las aristas de vecindad y similitud dados diversos argumentos para estos.

Es posible conocer la media de las métricas devueltas por el conjunto de barrios entrenados para cada grafo a través de la tabla 4.5, la cuál resalta además aquel grafo que ha conseguido devolver mejores métricas de evaluación:

Vecindad	Similitud	MSE	MAE	R^2
0.05 km	0.8 sim	370769.95	442.91	-0.17
0.05 km	0.85 sim	335930.61	429.70	-0.26
0.05 km	0.9 sim	366819.01	439.61	-0.40
0.05 km	0.95 sim	341527.09	423.93	-0.66
0.05 km	0.99 sim	231965.52	361.84	-0.36
0.1 km	0.8 sim	356719.74	436.66	-0.32
0.1 km	0.85 sim	342198.82	427.25	-0.44
0.1 km	0.9 sim	449388.25	467.47	-0.52
0.1 km	0.95 sim	335915.69	415.11	-0.17
0.1 km	0.99 sim	260697.30	369.62	0.03
0.15 km	0.8 sim	363657.36	424.65	-0.41
0.15 km	0.85 sim	367586.24	426.03	-0.34
0.15 km	0.9 sim	363219.49	436.52	-0.25
0.15 km	0.95 sim	353023.64	419.70	-0.58
0.15 km	0.99 sim	216082.91	333.89	0.05
0.2 km	0.8 sim	446983.43	464.19	-0.49
0.2 km	0.85 sim	360487.80	428.96	-0.15
0.2 km	0.9 sim	343809.25	419.48	-0.29
0.2 km	0.95 sim	435826.39	448.70	-0.55
0.2 km	0.99 sim	262935.99	357.34	-0.29
0.25 km	0.8 sim	346218.02	424.50	-0.41

Vecindad	Similitud	MSE	MAE	R ²
0.25 km	0.85 sim	367008.87	432.05	-0.28
0.25 km	0.9 sim	449858.38	453.22	-0.38
0.25 km	0.95 sim	337891.20	405.79	-0.12
0.25 km	0.99 sim	380154.81	391.20	-0.11
0.3 km	0.8 sim	329645.67	406.91	-0.13
0.3 km	0.85 sim	393392.82	428.57	-0.32
0.3 km	0.9 sim	374706.21	425.72	-0.20
0.3 km	0.95 sim	367163.63	420.84	-0.24
0.3 km	0.99 sim	233659.47	335.51	0.15

Tabla 4.5: Media de las métricas del modelo de convolución heterogéneo

La tabla 4.5 proporciona información relevante para la investigación y resolución del problema de predicción, a partir del cual pueden extraerse las siguientes observaciones:

- **Impacto vecindad:** los valores no muestran tendencias lineales que puedan indicar exactamente si influye el aumento o decremento del radio. Además las métricas devueltas dan indicios de que no es una buena forma de entrenar el algoritmo, al devolver valores R^2 negativos en prácticamente todos los casos. Sin embargo, destaca el par dado por el radio de 300m y el umbral de similitud alto, al ser la pareja de aristas que devuelven mejores resultados tras su evaluación. Quizás esto indique que con vecindades amplias y alta similitud, se puede ir obteniendo un mejor ajuste del modelo.
- **Impacto similitud:** de igual forma a como se observó durante el análisis de las métricas homogéneas, los umbrales altos de similitud parecen asociarse a valores de R^2 con tendencia menos negativa. Ligado con la vecindad, podría estar también relacionado el aumento de la similitud con la mejora del rendimiento.

De forma general la evaluación de estos modelos no ha sido la mejor, puesto que los errores devueltos por MSE y MAE son grandes en el contexto del problema, además de la cantidad de métricas R^2 que han resultado ser negativos, indicativo de que el modelo no está explicando bien la variabilidad de los datos en la mayoría de los grafos ofrecidos.

Interpretados los resultados medios del conjunto de barrios en cada par de modelo-grafo, resulta importante observar y comentar por medio del cuadro 4.6 los resultados por barrio devueltos por el mejor grafo heterogéneo entrenado, donde se resalta aquel barrio que devuelve la mejor evaluación:

Barrio	MSE	MAE	R ²
Torrefiel	79144.71	211.60	0.43

Barrio	MSE	MAE	R ²
Morvedre	150699.67	281.60	0.43
Trinitat	231221.31	366.58	0.14
Campanar	215683.60	387.56	0.18
El Carme	622417.43	636.74	-0.07
La Vega Baixa	108231.39	262.52	0.09
La Xerea	749468.06	567.57	0.24
L'Illa Perduda	164860.53	358.97	-0.71
Tres Forques	179816.09	256.22	0.09
Patraix	376403.25	306.07	0.11
Russafa	359221.56	445.17	0.15
La Creu del Grau	215385.54	334.13	0.02
Sant Pau	279962.28	423.39	0.24
La Seu	192258.73	353.44	0.63
Aiora	132371.25	292.34	0.12
Natzaret	93747.21	217.12	-0.32
La Creu Coberta	52888.08	176.69	0.12
L'Amistat	203503.32	321.60	0.36
El Grau	193212.87	328.35	0.21
La Carrasca	141678.75	319.28	0.25
Beniferri	67685.40	191.21	0.94
L'Hort de Senabre	128538.46	259.27	0.45
Ciutat Jardí	150176.14	314.66	0.07
Cami de Vera	173790.73	310.82	-0.21
Vara de Quart	63565.97	203.95	0.23
Soternes	58972.31	189.48	0.20
La Font Santa	71584.51	225.80	0.20
El Botanic	465995.59	535.81	-0.14
Favara	153670.10	312.33	-0.08
Cami Real	157857.28	313.06	-0.96
La Fonteta S.Lluís	36232.71	155.07	-0.56
La Punta	308319.00	369.98	0.33
Sant Llorens	274512.31	384.18	0.43
Ciutat Fallera	52517.67	183.67	-0.12
Nou Moles	102472.57	241.16	0.36
La Llum	82360.89	222.58	0.37
La Malva-Rosa	121007.78	280.17	0.26
El Calvari	88706.61	227.64	0.20
Les Tendetes	83318.98	219.55	0.01

Barrio	MSE	MAE	R ²
Ciutat Universitaria	273981.75	420.54	-0.50
El Mercat	199398.53	340.51	0.59
La Roqueta	683012.62	535.54	0.31
La Gran Via	291167.68	412.73	0.36
Cami Fondo	91273.05	241.67	-0.16
La Raiosa	117650.41	273.22	-0.01
Safranar	185095.43	291.80	-0.28
Sant Isidre	49557.59	175.70	0.30
Malilla	172315.93	322.08	0.35
Benicalap	151876.29	291.83	0.57
Jaume Roig	290247.15	426.40	0.07
La Petxina	260137.60	406.55	-0.23
Sant Marcel·li	72833.61	213.14	0.44
Montolivet	185413.78	332.45	-0.05
Ciutat de les Arts i de les Ciències	164416.01	295.69	0.60
Na Rovella	232678.04	339.66	0.29
Tormos	196728.85	256.02	0.33
Sant Antoni	153959.51	326.70	0.03
Benimaclet	196050.60	346.63	0.36
Marxalenes	96322.10	40.63	0.38
Exposicio	435438.46	511.41	0.52
Betero	119573.64	279.74	0.38
Cabanyal-Canyamelar	201485.10	341.70	0.17
Mestalla	473030.68	529.72	0.32
El Pilar	268310.28	394.11	0.16
El Pla del Remei	1984330.87	976.84	-0.24
Albors	166033.15	287.58	-0.17
Arrancapins	284118.56	430.75	-0.17
En Corts	199763.85	358.36	0.01
Poble Nou	85017.63	199.26	0.66
Sant Francesc	639744.25	588.10	0.33
Penya-Roja	425591.50	540.80	0.28
Els Orriols	93466.96	241.35	0.10

Tabla 4.6: Evaluación del mejor grafo heterogéneo

En la tabla 4.6 se puede observar que las métricas, que estas devuelven resultados menos decentes comparándolo con el mejor grafo homogéneo entrenado. Destaca de nuevo el barrio de **Beniferri**, puesto que ha sido la barriada

que mejores resultados devuelve, lo que quiere decir que el entrenamiento resulta apto para este barrio. Sin embargo, se trata de una excepción, ya que el resto de vecindarios de Valencia no cubrieron un buen desempeño durante el entrenamiento.

4.2.7. Uso del conocimiento

Tras conocer los resultados que ofrecen los modelos para cada uno de los grafos se concluye con que la característica de vecindad para radios pequeños, funciona bien para el problema de la predicción de precios de los domicilios.

Llegar a esta conclusión ha resultado fácil, puesto que se puede observar que la media de métricas es buena en todos los aspectos, destacando el Error Absoluto Medio, que indica un error del 3,16 % del rango total del precio, un buen indicio para el problema de predicción planteado.

Es por ello que finalmente se comenzará a desarrollar la aplicación web de predicción de precios con un modelo GCN para grafos homogéneos y un grafo de vecindad que relaciona viviendas en función de un radio de 50 metros, al no considerarse necesario una nueva iteración tras encontrar un par que devuelve métricas que indican gran calidad en el modelo.

Con esto concluye el proceso de descubrimiento del conocimiento y, en consecuencia, la etapa de investigación del proyecto que da paso a la implementación del software.

4.3. Aplicación

Esta última sección del capítulo de Ejecución presenta la aplicación finalmente desarrollada para la predicción de precios de la vivienda, respondiendo a la necesidad de los interesados de obtener una interfaz interactiva y eficiente para ello.

4.3.1. Introducción

Tras finalizar el proceso KDD con resultados útiles se puede dar paso a la fase de desarrollo de la aplicación, implementando una interfaz interactiva que permita a los usuarios interesados poder realizar predicciones con el modelo seleccionado tras la investigación.

Es por ello que la sección está destinada a comentar la arquitectura de la aplicación, así como las funciones principales que realiza para poder hacer una predicción con éxito.

Además de esto, se ofrecerá una guía de instalación local y las pruebas realizadas en el sistema para su mantenimiento.

4.3.2. Arquitectura

El proyecto que está desarrollado con el framework de Django [23] que sigue una arquitectura basada en el patrón Modelo-Vista-Template (MTV) de la figura 4.5, como una adaptación de la estructura Modelo-Vista-Controlador (MVC). Se compone de tres partes bien diferenciadas:

- **Modelo (Model):** representa la estructura de datos y la lógica de negocio de la aplicación. Su función principal es definir las tablas de la base de datos y las relaciones establecidas entre estas para su acceso y manipulación.
- **Vista (View):** se trata de la lógica que procesa solicitudes y respuestas. Su objetivo es interactuar con los modelos definidos en el sistema para obtener datos y pasarlos a las plantillas para su presentación por interfaz.
- **Plantilla (Template):** es la capa de presentación donde se define cómo se muestra la información a los usuarios que hacen uso de la aplicación. Está compuesto de archivos HTML con un lenguaje de plantillas que permiten incluir variables y lógica simple para mostrar el contenido.

Adicionalmente la arquitectura de un proyecto Django incluye:

- **Enrutamiento:** sistema encargado de mapear las URLs a las vistas correspondientes del sistema.
- **Middleware:** capas intermedias que procesan las solicitudes y las respuestas globalmente antes o después de las vistas.
- **Configuración:** archivo de configuración general del proyecto.

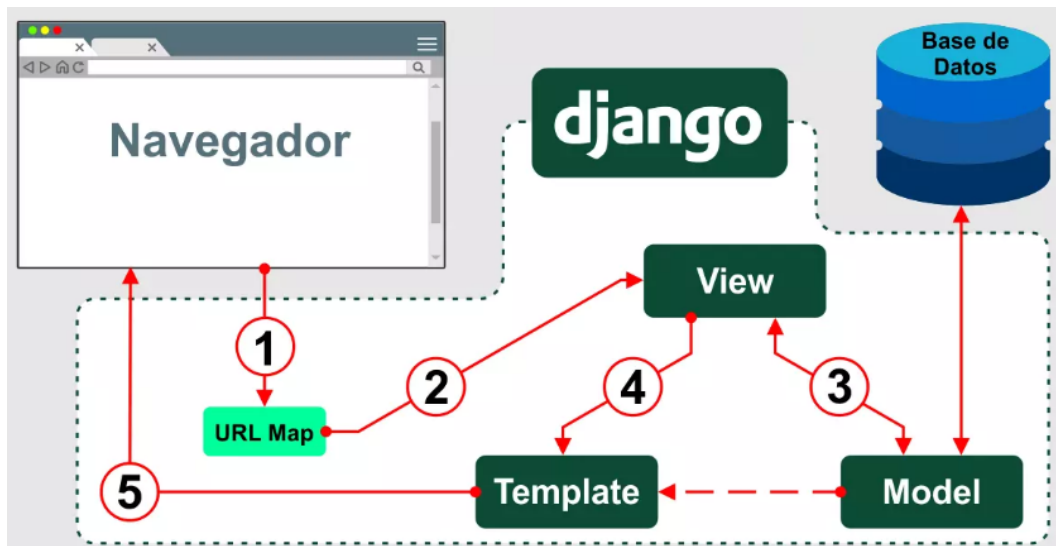


Figura 4.5: Esquema del modelo MTV para Django

Imagen disponible en [Espifreelancer](#).

Jerarquía

La estructura organizada de carpetas y archivos del proyecto es fundamental para la calidad del software, ya que ayuda al correcto desarrollo y futuro mantenimiento de la aplicación.

La jerarquía propuesta permite separar de forma clara y lógica los diferentes componentes y funcionalidades del proyecto, dando a cada carpeta y archivo un propósito específico para el proyecto.

Es por ello que se detalla la estructura seguida durante la implementación del proyecto, exponiendo las carpetas y su contenido principal:

- **Carpeta data:** engloba toda la lógica para la carga de datos dentro de la aplicación. Se compone de los siguientes elementos:
 - **Carpeta db:** contiene los ficheros CSV con los datos que se almacenarán en la base de datos como son las viviendas trabajadas fruto de la investigación, los barrios, los puntos geográficos de dónde se encuentran las bocas de metro en Valencia o los hiperparámetros del modelo usado.
 - **Fichero de carga de datos:** se trata de un archivo de extensión .py que transforma las líneas de los CSV expuestos anteriormente en sus modelos correspondientes para la base de datos.
- **Carpeta idealista:** se trata del módulo raíz del proyecto donde se agrupan los elementos esenciales que configuran y coordinan el funcionamiento general de la aplicación. Dentro de este módulo destacan los siguientes archivos:
 - **Archivo init:** no es más que un archivo que indica en lenguaje Python [13] que la carpeta se puede tratar como un paquete. Esto es para que se reconozca el módulo por el resto de carpetas y se puedan incorporar sus componentes desde otras partes del proyecto.
 - **Archivo de configuración (settings):** contiene toda la configuración principal del proyecto Django. Se definen aspectos como la conexión a la base de datos, aplicaciones instaladas o configuración de middleware, entre otros.
 - **Archivo de rutas (urls):** define el sistema de enrutamiento global del proyecto, estableciendo las rutas que mapean a las vistas correspondientes.
 - **Archivo de servidores web (wsgi):** se trata del punto de entrada para que Django se pueda desplegar en diversos servidores web y se atiendan a las solicitudes HTTP.
- **Carpeta aplicación (webapp):** se trata del módulo de la aplicación que contiene la lógica principal de la misma. Dentro de esta carpeta se encuentran los siguientes elementos:
 - **Carpeta de datos (data):** se trata del conjunto de datos que trabaja con el backend del proyecto en la funcionalidad. Contiene archivos como la

geometría de los barrios o los pesos de los entrenamientos generados durante la investigación para el modelo seleccionado.

- **Carpeta de plantillas (templates):** contiene los archivos HTML creados para generar la interfaz interactiva de los usuarios con la aplicación.
- **Archivo init:** de igual forma que para el módulo principal, es usado para reconocer la carpeta webapp como un módulo por Python.
- **Archivo de admin (admin):** archivo que define la vista de administrador en Django.
- **Archivo de aplicación (app):** contiene la configuración específica de la aplicación que define la clase que representa la aplicación y que se registra en el archivo de configuración para ser reconocida por Django.
- **Archivo de formulario (forms):** se utiliza para definir formularios en la aplicación. Estos formularios facilitan la validación y procesamiento de datos que los usuarios envían a la aplicación web, en este caso, las viviendas que desean predecir.
- **Archivo de modelos (models):** el fichero define las clases que representan las tablas en la base de datos y su lógica asociada. Cada modelo se representa como una entidad de datos y maneja las operaciones relacionadas con la misma.
- **Archivo de pruebas (tests):** se trata de un archivo que contiene casos de prueba usados para verificar que las distintas partes del módulo funcionan correctamente. Contiene pruebas para las vistas y formularios de la aplicación.
- **Archivo de herramientas (utils):** contiene las funciones que ayudan a realizar la lógica del backend de la aplicación, es decir, toda la funcionalidad para la predicción.
- **Archivo de vistas (views):** contiene toda la lógica que responde a las solicitudes del usuario, procesando sus peticiones e interactuando con los modelos para devolver respuestas.

Junto a esta estructura cabe destacar también dos ficheros importantes localizados en la carpeta raíz del proyecto, que son los siguientes:

- **Archivo manage.py:** se trata de una herramienta de línea de comandos para ejecutar comandos en el proyecto. Entre estos destacan los siguientes:
 - **runserver:** inicia el servidor de desarrollo en local.
 - **migrate:** aplica migraciones para crear o modificar la estructura de la base de datos según los modelos definidos en la aplicación.
 - **createsuperuser:** crea un usuario administrador para acceder al panel del mismo.
 - **shell:** su ejecución abre una consola interactiva para trabajar con el proyecto.

- **test:** ejecuta los casos de prueba definidos en la aplicación.
- **Archivo requirements.txt:** el archivo contiene la lista de dependencias y librerías externas requeridas en el proyecto para que funcione correctamente. Tiene como objetivo facilitar una instalación rápida de la misma.

Se puede observar con más detalle la jerarquía del proyecto a través de la figura 4.6:

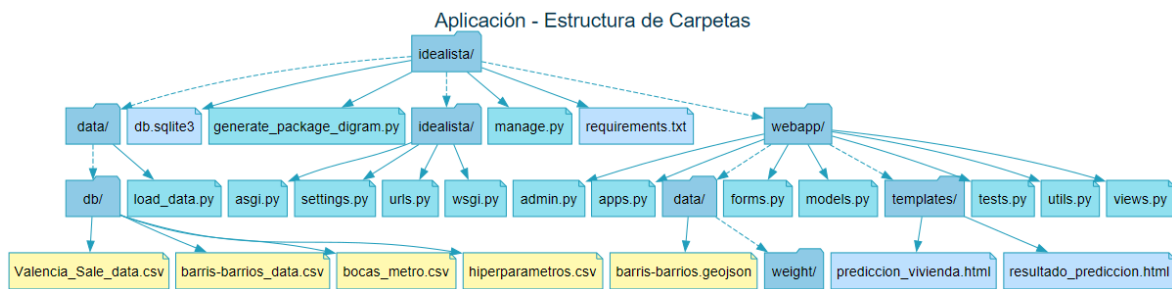


Figura 4.6: Diagrama UML de carpetas de la aplicación

Modelo relacional

Para comprender la estructura interna de los modelos que manejan la aplicación es conveniente analizar su modelo relacional.

El diagrama relacional permitirá visualizar de qué se componen los modelos creados y cómo interactúan entre ellos.

Para la aplicación se consta de cuatro entidades, cuyos atributos y relaciones se pueden conocer por medio del diagrama representado en la figura 4.7:

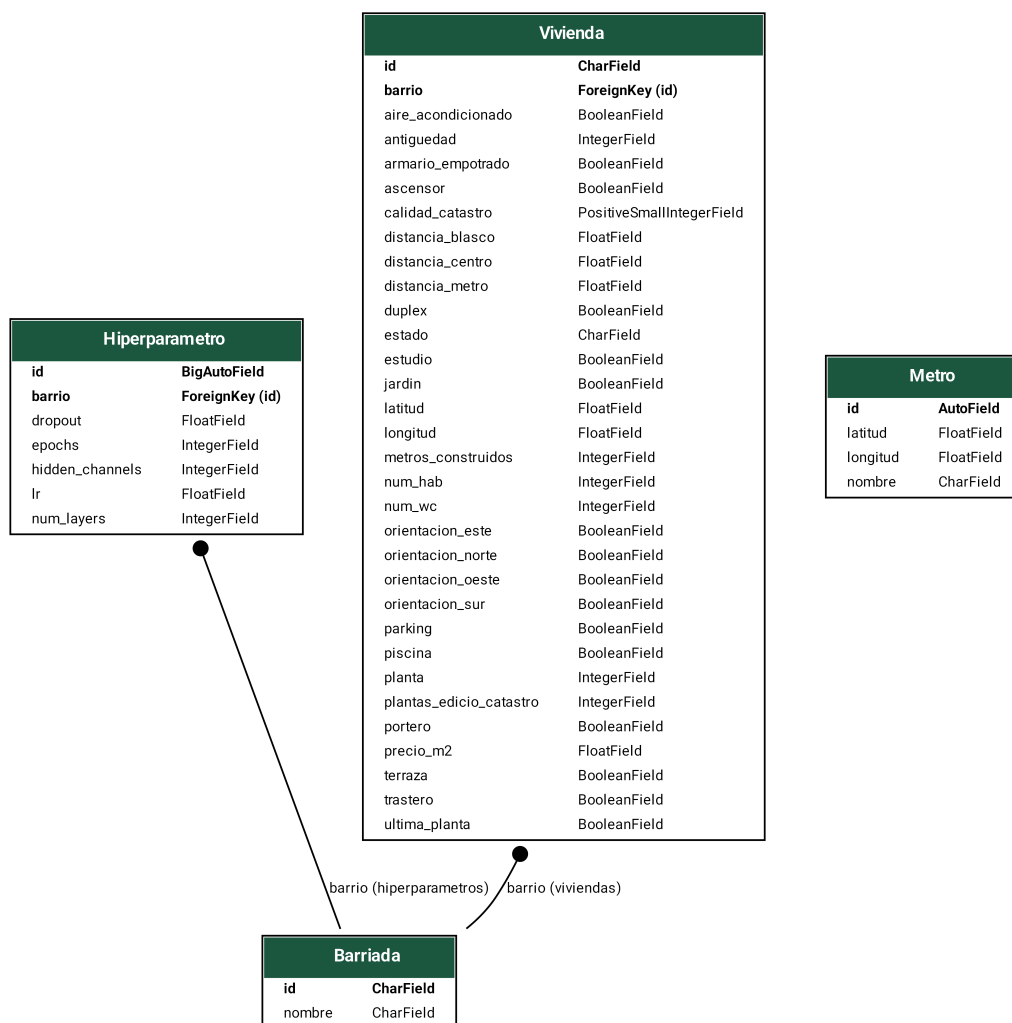


Figura 4.7: Modelo relacional de la aplicación

4.3.3. Manual de instalación

La sección está dedicada a detallar todo el proceso paso a paso para instalar la aplicación de predicción de forma local.

A partir de este punto se recomienda encarecidamente prestar atención a los elementos descritos, para que se pueda finalizar con una instalación de forma exitosa.

Antes de comenzar a guiar la instalación se presentan las tecnologías y herramientas requeridas para la misma, recogidas en la siguiente tabla:

Nombre	Descripción	Versión
Python [13]	Lenguaje de programación utilizado como base del proyecto. Es necesario para ejecutar Django y todas sus dependencias.	3.11+
Git [8]	Sistema de control de versiones utilizado para clonar y gestionar el repositorio del proyecto.	Última estable
Visual Studio Code [12]	Editor de código recomendado para trabajar con el proyecto. Se sugiere instalar extensiones para Python y Django.	Última estable
Virtualenv	Herramienta para crear entornos virtuales en Python, permitiendo aislar las dependencias del proyecto.	Última estable
Django [23]	Framework web en el que está construido el proyecto. Se instala automáticamente desde el archivo <code>requirements.txt</code> .	Última estable

Tabla 4.7: Tecnologías y herramientas requeridas

Una vez instaladas correctamente todas las tecnologías y herramientas necesarias, se procede a detallar paso a paso el proceso de instalación, configuración y arranque del proyecto. Es importante seguir las instrucciones al pie de la letra para evitar errores comunes y asegurar que el entorno quede preparado de forma adecuada.

Los pasos a seguir son los siguientes:

1. Como primer paso se accede a GitHub [2], donde está alojado el código por medio de un repositorio [45] en la nube y se realiza lo siguiente:
 - a) Presionando el botón verde de *Code* se despliega un menú.
 - b) En este mismo desplegable aparece la URL al repositorio, el cual se deberá copiar al portapapeles.
 - c) Acto seguido se debe abrir una nueva ventana de Visual Studio Code. Se inicia con una pestaña de *Welcome* que posee varias opciones. Seleccionando *Clone Git Repository* y pegando la URL, clonaremos el repositorio en remoto en el ordenador.
 - d) Tras esto se abrirá una pestaña de explorador de archivos, para indicar donde queremos alojar el código del repositorio.

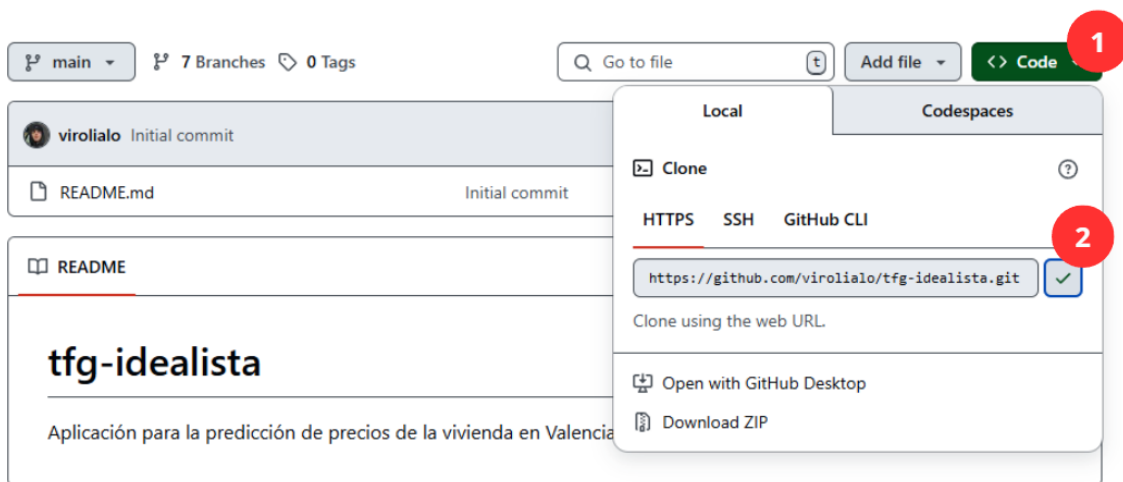


Figura 4.8: Pasos en GitHub para la instalación

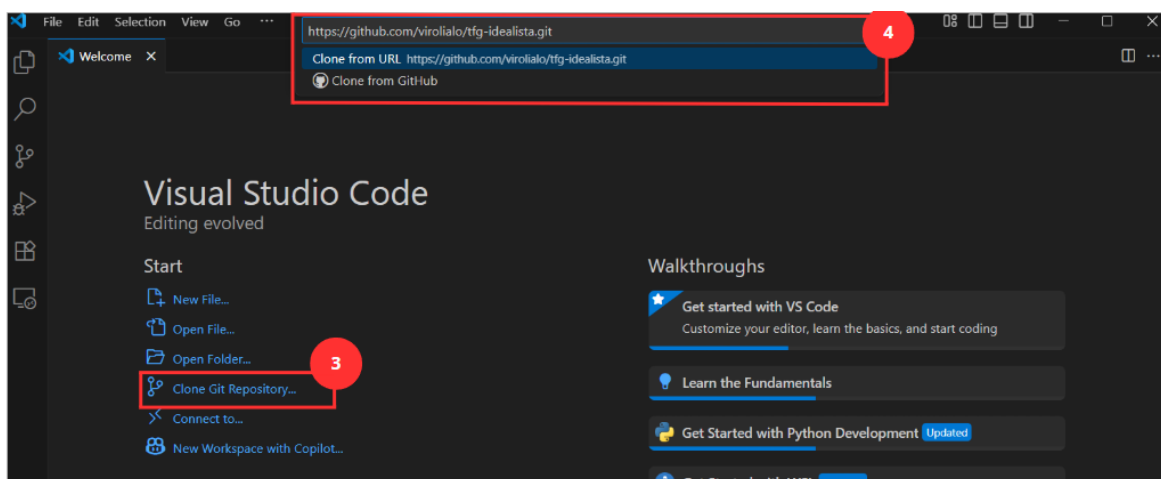


Figura 4.9: Pasos en Visual Studio Code para la instalación

Al finalizar este primer paso se deberá tener como resultado una pestaña de Visual Studio Code con el proyecto abierto.

2. El siguiente paso será crear un entorno virtual donde se alojará la posterior instalación de los paquetes y librerías necesarias para el proyecto. Para conseguir esto, aplicamos lo siguiente:

- a) Ejecutando el atajo de teclas CTRL + SHIFT + P, aparecerá en la barra de búsqueda superior del entorno de trabajo un desplegable.
- b) Con el desplegable abierto seguimos el siguiente curso:
 - Seleccionamos Python: Create Enviroment
 - Seleccionamos Venv

Con esto se creará y se estará directamente dentro del entorno virtual de Python.

3. Tras esto llega el momento de instalar las dependencias de librerías y paquetes necesarios para el proyecto. Estas dependencias están alojadas dentro del archivo `requirements.txt`, el cual se ejecutará siguiendo esta secuencia:

- a) Accedemos a la carpeta `idealista` por medio de un terminal de `cmd`. Para ello, ejecutamos desde la carpeta raíz:

```
1 cd idealista
2
```

- b) Una vez aquí nos encontramos en la carpeta que contiene las dependencias, el cual se instala por medio del comando `pip` de la siguiente forma:

```
1 pip install -r requirements.txt
2
```

Si todo es correcto deberá ver como se van instalando las dependencias por medio de la consola, hasta finalizar el proceso.

4. El siguiente paso será establecer la configuración inicial del proyecto. Para esto, se ejecutan los siguientes comandos desde la carpeta `idealista` que está en la misma altura de `manage.py`:

- a) Migración de los modelos:

```
1 python manage.py makemigrations
2 python manage.py migrate
3
```

- b) Creación de un usuario administrador:

```
1 python manage.py createsuperuser
2 > "Username (leave blank to use 'usuario'):"
3 admin
4 > "Email address:"
5 admin@admin.com
6 > "Password:"
7 admin
8 > "Password (again):"
9 admin
10 > "Bypass password validation and create user anyway? [y/N]: "
11 y
12 > "Superuser created successfully."
13
```

- c) Agregar los datos a la base de datos de Django (partiendo desde la carpeta idealista:

```
1      cd data
2      python load_data.py
3
```

Si todo ha ido bien la consola devuelve lo siguiente tras la ejecución del archivo:

```
1      > "Se han creado 72 barrios nuevos."
2      > "Se han cargado 72 hiperparametros en la base de datos."
3      > "Se han cargado 25995 viviendas en la base de datos."
4      > "Se han cargado 187 bocas de metro en la base de datos."
5
```

5. Tras esto ya están los modelos creados e integrados en la aplicación, por lo que es posible arrancar el sistema en local y probar que todo funciona. Para ello se ejecuta desde la carpeta idealista el siguiente comando, que deberá devolver un mensaje por consola como el mostrado:

```
1      python manage.py runserver
2      > "System check identified no issues (0 silenced)."
3      > "June 15, 2025 - 22:52:05"
4      > "Django version 5.2.3, using settings 'idealista.settings'"
5      > "Starting development server at http://127.0.0.1:8000/"
6      > "Quit the server with CTRL-BREAK."
7
```

Siguiendo estos pasos obtenemos como resultado una aplicación configurada y en ejecución.

4.3.4. Funciones principales

La aplicación desarrollada cuenta con dos funciones principales para resolver las necesidades pedidas por los interesados del proyecto. Estas representan el núcleo del sistema, por lo que a continuación se describirán brevemente dichas funciones, detallando sus especificaciones y propósitos.

Formulario de predicción

En primer lugar se ha creado la lógica para desarrollar una interfaz que permita a un usuario predecir el precio de una vivienda, dado unos datos de entrada que deberá rellenar a través de un formulario. Entre las características más importantes que ofrece el formulario:

- Ofrece restricciones en valores numéricos y tipos obligatorios, con el objetivo de impedir que se intente predecir viviendas con datos erróneos o disparados.
- La interfaz posee un mapa interactivo de tal forma que el usuario pueda seleccionar la ubicación de la vivienda de una manera cómoda, sin necesidad de tener que introducir las coordenadas a mano. De la misma forma al seleccionar la ubicación de la vivienda, la interfaz mostrará las coordenadas exactas seleccionadas, además de marcar geográficamente en el mapa la ubicación.
- Relacionado con lo anterior, el formulario calculará de forma automática las distancias al metro, al centro de la ciudad y a la Avenida de Blasco, tras presionar el botón de predicción y validar los datos del formulario. Esto ayuda a agilizar el registro de viviendas en el formulario y a ahorrar tiempo al usuario interesado.

La funcionalidad es accesible a través de la URL </prediccion/>.

Visualización de resultados

Junto al formulario se ha creado una vista adicional para mostrar el resultado de la predicción, además de las relaciones que presenta la vivienda dentro del barrio en el que se integra. Entre las características más importantes que ofrece la vista:

- Se muestra el precio predicho para la vivienda dadas las características del formulario del inmueble, junto con las viviendas que se ha relacionado en la predicción, es decir, con las que ha formado las aristas de vecindad a través de un mapa fijo que las ubica.
- Relacionado con lo anterior, es posible conocer el precio del metro cuadrado predicho haciendo click en el icono de la ubicación de la vivienda principal (coloreada de azul).
- De la misma forma, se pueden conocer las ubicaciones de las viviendas con las que se relaciona haciendo click en los distintos inmuebles expuestos en el mapa.
- Vuelven a aparecer los datos usados en el formulario para que se pueda comprobar que características se han usado durante la predicción, junto a un botón para volver a realizar una nueva predicción en caso de haber introducido datos equivocados.



Figura 4.10: Interfaz de los resultados - Mapa

La funcionalidad es accesible tras rellenar un formulario de vivienda con campos válidos a través de la URL /resultados/.

4.3.5. Pruebas de funcionalidad

En esta sección se presentan las distintas pruebas de funcionalidad implementadas en la aplicación. El objetivo principal de las pruebas es asegurar la calidad y mantenibilidad del código, verificando el correcto funcionamiento de las funcionalidades desarrolladas.

Estas pruebas se encuentran dentro del archivo `tests.py` del módulo `webapp`, que implementa dos tipos distintos de pruebas:

- **Pruebas unitarias:** estas verifican el comportamiento y la validación del formulario de `ViviendaPrediccionForm`, comprobando que el formulario sea válido o inválido según los datos proporcionados y que los mensajes de error se generen correctamente en los campos esperados. Algunas de las pruebas unitarias creadas para la aplicación son las mostradas en el bloque de código 4.17:

```

1 class ViviendaPrediccionFormTests(TestCase):
2     def setUp(self):
3         self.valid_data = {
4             'metros_construidos': 80,
5             'num_hab': 3,
6             'num_wc': 2,
7             'planta': 2,
8             'plantas_edificio_catastro': 3,
9             'calidad_catastro': 5,
10            'antiguedad': 10,

```

```

11         'terraza': False,
12         'ascensor': True,
13         'aire_acondicionado': False,
14         'parking': False,
15         'orientacion_norte': True,
16         'orientacion_sur': False,
17         'orientacion_este': False,
18         'orientacion_oeste': False,
19         'trastero': False,
20         'armario_empotrado': False,
21         'piscina': False,
22         'portero': False,
23         'jardin': False,
24         'duplex': False,
25         'estudio': False,
26         'ultima_planta': False,
27         'estado': '2HANDGOOD',
28         'latitud': 39.470669,
29         'longitud': -0.372466,
30     }
31
32     def test_form_valid(self):
33         with patch('webapp.forms.obtener_barrio_desde_geojson') as
mock_barrio:
34             mock_barrio.return_value = 'BarrioTest'
35             form = ViviendaPrediccionForm(data=self.valid_data)
36             self.assertTrue(form.is_valid())
37
38     def test_banos_mayor_que_habitaciones(self):
39         data = self.valid_data.copy()
40         data['num_wc'] = 5
41         data['num_hab'] = 2
42         form = ViviendaPrediccionForm(data=data)
43         self.assertFalse(form.is_valid())
44         self.assertIn('num_wc', form.errors)
45
46     def test_planta_baja_no_ultima(self):
47         data = self.valid_data.copy()
48         data['planta'] = 0
49         data['ultima_planta'] = True
50         form = ViviendaPrediccionForm(data=data)
51         self.assertFalse(form.is_valid())
52         self.assertIn('ultima_planta', form.errors)
53
54     def test_duplex_necesita_mas_de_una_planta(self):
55         data = self.valid_data.copy()
56         data['duplex'] = True
57         data['plantas_edicio_catastro'] = 1
58         form = ViviendaPrediccionForm(data=data)
59         self.assertFalse(form.is_valid())

```

```

60         self.assertIn('duplex', form.errors)
61
62         ...

```

Extracto de código 4.17: Pruebas unitarias de la aplicación

- **Pruebas de integración:** este tipo de pruebas comprueban el comportamiento de las vistas relacionadas con la predicción de la vivienda, simulando peticiones HTTP (GET y POST) y comprobando que las respuestas de las vistas, el uso de las plantillas y el manejo de la sesión es el correcto. Para realizar estas pruebas se utilizan *mocks* que simulan dependencias externas para aislar la lógica de la vista. El bloque de código 4.18 expone algunas de las pruebas implementadas para este tipo:

```

1 class PrediccionViviendaViewTests(TestCase):
2     def setUp(self):
3         self.valid_data = {
4             'metros_construidos': 80,
5             'num_hab': 3,
6             'num_wc': 2,
7             'planta': 2,
8             'plantas_edificio_catastro': 3,
9             'calidad_catastro': 5,
10            'antiguedad': 10,
11            'terraza': False,
12            'ascensor': True,
13            'aire_acondicionado': False,
14            'parking': False,
15            'orientacion_norte': True,
16            'orientacion_sur': False,
17            'orientacion_este': False,
18            'orientacion_oeste': False,
19            'trastero': False,
20            'armario_empotrado': False,
21            'piscina': False,
22            'portero': False,
23            'jardin': False,
24            'duplex': False,
25            'estudio': False,
26            'ultima_planta': False,
27            'estado': '2HANDGOOD',
28            'latitud': 39.470669,
29            'longitud': -0.372466,
30        }
31
32    def test_prediccion_vivienda_get(self):
33        response = self.client.get(reverse('prediccion_vivienda'))
34        self.assertEqual(response.status_code, 200)
35        self.assertTemplateUsed(response, 'prediccion_vivienda.html')

```

```

36         self.assertIn('form', response.context)
37
38     @patch('webapp.forms.obtener_barrio_desde_geojson')
39     @patch('webapp.views.procesar_viviendas_barrio')
40     @patch('webapp.views.calcular_distancias')
41     @patch('webapp.views.procesar_nueva_vivienda_formulario')
42     @patch('webapp.views.predecir_precio_m2_vivienda')
43     @patch('webapp.views.obtener_vecinos_vivienda')
44     def test_prediccion_vivienda_post_valido(
45         self, mock_vecinos, mock_pred, mock_proc_nueva, mock_dist,
46         mock_proc_barrio, mock_barrio
47     ):
48         mock_barrio.return_value = MagicMock(id=1, __str__=lambda self: "
49         BarrioTest")
50         mock_proc_barrio.return_value = ([{}], MagicMock(), MagicMock())
51         mock_dist.return_value = (0.5, 1.2, 0.8)
52         mock_proc_nueva.return_value = {}
53         mock_pred.return_value = 2000
54         mock_vecinos.return_value = [{'latitud': 39.47, 'longitud': -0.37,
55         'precio_m2': 2100}]
56         response = self.client.post(reverse('prediccion_vivienda'), data=
57         self.valid_data)
58         self.assertEqual(response.status_code, 302)
59         self.assertIn(reverse('resultado_prediccion'), response.url)
60
61     def test_resultado_prediccion_redirect_si_no_hay_resultado(self):
62         response = self.client.get(reverse('resultado_prediccion'))
63         self.assertEqual(response.status_code, 302)
64         self.assertIn(reverse('prediccion_vivienda'), response.url)
65
66     def test_resultado_prediccion_ok(self):
67         session = self.client.session
68         session['resultado_prediccion'] = {
69             'datos': self.valid_data,
70             'precio_predicho': 100000,
71             'precio_m2_predicho': 2000,
72             'distancia_blasco': 0.8,
73             'distancia_metro': 0.5,
74             'distancia_centro': 1.2,
75             'vecinos': [{'latitud': 39.47, 'longitud': -0.37, 'precio_m2':
76             2100}],
77         }
78         session.save()
79         response = self.client.get(reverse('resultado_prediccion'))
80         self.assertEqual(response.status_code, 200)
81         self.assertTemplateUsed(response, 'resultado_prediccion.html')
82         self.assertIn('precio_predicho', response.context)

```

Extracto de código 4.18: Pruebas de integración de la aplicación

En total se han implementado 14 pruebas de funcionalidad, de las cuales 11 pertenecen a la clase `ViviendaPrediccionFormTests` y 4 a `PrediccionViviendaViewTests`, cubriendo en gran parte la funcionalidad y las vistas principales del código.

5. Cierre

Este capítulo final tiene como objetivo reflexionar sobre el trabajo realizado a lo largo del proyecto, así como ofrecer una visión crítica y constructiva de los resultados obtenidos.

Para ello en primer lugar se presentarán una serie de propuestas de mejora a largo plazo para el proyecto que podrían aplicarse en futuras iteraciones o vertientes del trabajo realizado, siguiendo con las lecciones aprendidas durante el desarrollo, como fruto de la experiencia acumulada durante el mismo, así como retos enfrentados y soluciones aportadas.

Finalmente se recogerán las conclusiones del proyecto destacando los logros alcanzados, el cumplimiento de los planes y el impacto del trabajo realizado.

5.1. Propuestas de mejora

A lo largo del desarrollo del proyecto han surgido nuevas necesidades de optimización que, por limitaciones de tiempo, no han podido abordarse en el trabajo. Es por ello que en esta sección se recogen las propuestas de mejora que podrían aplicarse de cara a mejorar investigación y aumentar la eficiencia, escalabilidad y usabilidad de la aplicación. Se plantean las siguientes mejoras en el proyecto:

- **Añadir nuevas características de la vivienda o el barrio:** aun habiendo obtenido como resultado con tan solo una iteración un buen modelo tras aplicar KDD, bien es sabido que siempre es posible encontrar nuevas variantes trabajando con nuevos atributos de la vivienda que aporten valor al contexto del problema. Durante la investigación se encontró que además de la vecindad, otro de los puntos fuertes del precio de la vivienda era los servicios que ofrece un barrio, es decir, que contenga zonas comerciales, colegios, o parques, entre otros. Por ello se propone buscar o crear nuevos datos para así tratar de mejorar la predicción.
- **Uso de grafos distintos:** de la misma forma, generar grafos con nuevas relaciones puede resultar interesante en la búsqueda de perfeccionar aun más la predicción. Algunas de las propuestas que se pueden explorar pueden ser:
 - **Grafo de uso compartido de infraestructuras o servicios públicos:** se forman aristas entre viviendas que estén conectadas a la misma red eléctrica, zona escolar, centro de salud o estaciones de transporte.
 - **Grafo de reputación online:** se establecen aristas entre las viviendas en función de si tienen valoraciones similares en plataformas inmobiliarias, foros vecinales o redes sociales.

- **Grafo de comportamiento del mercado inmobiliario:** relaciona viviendas en función de si los precios de estas han evolucionado de forma similar en el tiempo, aunque estas estén lejos o sean distintas, capturando así las tendencias del mercado y barrios especulativos.
- **Uso de modelos distintos:** ligado con las entradas anteriores, probar con otros modelos durante la etapa KDD puede ser también una buena propuesta de mejora, ya que puede dar lugar a obtener modelos que devuelvan una mejor evaluación para el contexto del problema. Alguno de los modelos que se podrían investigar son los siguientes:
 - **GraphSAGE (Sample and Aggregate):** permite entrenamiento inductivo, lo que puede ayudar a generalizar bien a nodos nuevos que se integren en el grafo.
 - **GIN (Graph Isomorphism Network):** tienen más capacidad expresiva que el modelo de convolución usado capturando con más precisión patrones estructurales del grafo, lo cual resulta útil si las relaciones entre las viviendas resultan complejas.
 - **Temporal GNN:** modela la dinámica del grafo a lo largo del tiempo que relacionado con la predicción de precios, ayuda a capturar la evolución del mercado inmobiliario y cómo las relaciones cambian a lo largo del tiempo.
- **Mejora de la aplicación:** la aplicación resultante ofrece simplemente la funcionalidad principal, que es predecir viviendas y observar sus resultados, lo que lo hace funcional pero a la vez básica. Alguno de los incrementos más interesantes podrían ser:
 - **Registro de usuarios:** permitir a los usuarios interesados en la predicción registrarse en la aplicación, tratando de hacerlo lo más sencillo posible y estableciendo distintas formas de iniciar sesión.
 - **Registro de predicciones:** esto es, registrar en la base de datos todas las predicciones realizadas asociadas a los usuarios, para así mostrarles un historial a los mismos y puedan ver el recorrido de predicciones que realizan en la aplicación.
 - **Timeline de predicciones:** consistiría en añadir una página que se actualiza en función de las predicciones que realizan los usuarios en la aplicación, decidiendo estos si deciden hacer la predicción pública o no. Anonimatizando la localización la publicación de las predicciones podría dar juego a una especie de foro de debate sobre precios, donde otros usuarios pueden valorar las predicciones publicadas y dar criterio al precio devuelto por el modelo.

Estas propuestas son solo un conjunto de las mejoras más interesantes consideradas por el equipo de trabajo, pero existen multitud de formas de añadir un valor incremental al proyecto realizado.

5.2. Lecciones aprendidas

A medida que avanzaba el proyecto el equipo de trabajo se ha enfrentado a diversos retos técnicos, organizativos y metodológicos que han supuesto una valiosa fuente de aprendizaje durante el camino.

Es por este motivo que la sección se encarga de recoger las principales lecciones adquiridas durante el proceso, con la intención de identificar aciertos, reconocer errores y destacar aspectos que podrían abordarse de forma más eficiente de cara futuros proyectos similares.

La autorreflexión de estas experiencias contribuye no solo al crecimiento profesional sino también a la mejora continua del enfoque de trabajo y de la toma de decisiones.

Con esto, se procede a detallar las lecciones más importantes obtenidas:

- **Planificación:** en términos generales la planificación del proyecto ha sido realmente efectiva, ya que se ha cumplido tras implementar los requisitos establecidos y seguir con rigurosidad los planes elaborados. Sin embargo, caben destacar los siguientes errores cometidos:
 - **Replanificación:** durante la planificación del proyecto se tuvieron que modificar los cronogramas debido a la falta de conocimiento del tiempo que podría costar la implementación de las tareas. Esto ha llevado a prolongar el tiempo de trabajo del proyecto y, en consecuencia, los costes del mismo. Este hecho invita a reflexionar de cara a futuros proyectos, donde se deberá tener más constancia del esfuerzo que requieren las tareas que, tras una primera toma de contacto con un proyecto de este calibre, servirá para mejorar los planes.
 - **Planificación inicial esencial pero no definitiva:** haber tenido un plan base para comenzar el proyecto ha resultado clave, sin embargo, es importante tener en mente que la flexibilidad en los proyectos es un hecho real, ya que los requisitos son cambiantes durante el proyecto. Por ello de cara a futuros proyectos se deberá tratar de mejorar la adaptabilidad frente a posibles cambios.
- **Implementación:** en cuanto a la implementación se hace reflexión de los siguientes puntos que han resultado ser claves:
 - **Prototipos sencillos:** tratar de abarcar todo en un mismo momento, ha llevado a tener errores durante largos periodos de tiempo y que estos sean difíciles de identificar y tratar. Es por este motivo que para futuros proyectos relacionados, se tendrá en cuenta realizar pequeños incrementos, de cara agilizar el trabajo durante el desarrollo.
 - **Constancia del cambio:** relacionado con lo anterior, el repositorio de GitHub posee commits que tienen cantidades enormes de código, mezclando además distintas funcionalidades en el mismo. Con esto, se quiere tomar consciencia a la hora de añadir un incremento al proyecto,

haciendo commits atómicos y claros, lo que ayudará a tener un control del cambio y mejorar la mantenibilidad del sistema.

- **Comunicación:** se destaca la siguiente lección:
 - **Constante y efectiva:** durante el desarrollo del proyecto, las reuniones con los colaboradores no han sido constantes, además de no estar guiadas. Por ello, se propone de cara al futuro mejorar la comunicación con los clientes, mejorando la periodicidad de las reuniones y haciendo que estas sean más efectivas con la creación de guiones previos que favorezcan el curso de la misma.

5.3. Conclusiones

El desarrollo de este proyecto ha permitido integrar de forma efectiva el marco de investigación teórica junto con la aplicación práctica a través del sistema implementado, generando un gran impacto en el área del conocimiento estudiado a lo largo del grado.

La realización de este trabajo ha permitido validar hipótesis, contrastar enfoques y demostrar la viabilidad de las soluciones propuestas para un problema real como es la predicción de la vivienda, que se encuentra en auge en los tiempos que corren.

Si bien se han alcanzado los objetivos principales del proyecto, también han surgido nuevas preguntas y oportunidades de mejora que abren puerta a futuras líneas de investigación y desarrollo por la predicción de precios de la vivienda. En este sentido, el proyecto no solo cierra con una solución implementada, sino con una mayor comprensión del problema y una base sólida sobre la cual seguir avanzando.

En términos generales, este proyecto representa una experiencia enriquecedora desde un punto de vista técnico, metodológico y personal, y formará parte de una contribución relevante como cierre de la era académica.

Bibliografía

- [1] Project Management Institute. *Guía de los fundamentos para la dirección de proyectos (Guía del PMBOK)*. Project Management Institute, 2014. ISBN 1628250097.
- [2] GitHub. Github: Where the world builds software. <https://github.com/>, 2025.
- [3] Microsoft. Excel online. <https://excel.cloud.microsoft/es-es/>, 2024.
- [4] Clockify. Clockify: Rastrear el tiempo de forma gratuita. <https://clockify.me/es/>, 2025.
- [5] Talent.com. Talent.com: Encuentra tu próximo empleo. <https://es.talent.com/>, 2025.
- [6] Selectra. Semáforo de precios de la luz - selectra. <https://selectra.es/energia/semaforo-selectra>, 2025.
- [7] Repsol. ¿cuánto consume un ordenador? <https://www.repsol.es/particulares/asesoramiento-consumo/cuanto-consume-ordenador/>, 2025.
- [8] Git SCM. Git. <https://git-scm.com/>.
- [9] Codacy. Codacy - automated code reviews and quality assurance. <https://www.codacy.com/>.
- [10] Overleaf. Overleaf - online latex editor. <https://www.overleaf.com/>.
- [11] Microsoft. Microsoft teams - versión gratuita. <https://teams.live.com/free>, .
- [12] Microsoft. Visual studio code. <https://code.visualstudio.com/>, .
- [13] Python Software Foundation. Python programming language. <https://www.python.org/>.
- [14] NumPy Developers. Numpy. <https://numpy.org/>.
- [15] Pandas Development Team. pandas - python data analysis library. <https://pandas.pydata.org/>.
- [16] GeoPandas Development Team. Getting started with geopandas. https://geopandas.org/en/stable/getting_started/introduction.html.
- [17] Shapely Developers. Shapely manual. <https://shapely.readthedocs.io/en/stable/manual.html>.
- [18] Matplotlib Development Team. Matplotlib: Visualization with python. <https://matplotlib.org/>.

- [19] Scikit-learn. Scikit-learn: Machine learning in python. <https://scikit-learn.org/stable/index.html>.
- [20] PyTorch. Pytorch. <https://pytorch.org/>.
- [21] Optuna. Optuna: A hyperparameter optimization framework. <https://optuna.org/>.
- [22] NetworkX. Networkx. <https://networkx.org/>.
- [23] Django Software Foundation. Django: The web framework for perfectionists with deadlines. <https://www.djangoproject.com/>.
- [24] SQLite. Sqlite home page. <https://sqlite.org/index.html>.
- [25] Markdown.es. Markdown.es - documentación y recursos sobre markdown. <https://markdown.es/>.
- [26] GitKraken. Git flow: A successful git branching model. <https://www.gitkraken.com/learn/git/git-flow>.
- [27] Awan Abid Ali. Introducción completa a las redes neuronales gráficas (gnn). *DataCamp*, 2024. URL <https://www.datacamp.com/es/tutorial/comprehensive-introduction-graph-neural-networks-gnns-tutorial>.
- [28] Emily Pearce Adam y Wiltchko Alexander B. Sanchez-Lengeling, Benjamin Reif. A gentle introduction to graph neural networks. *Distill*, 2021. URL <https://distill.pub/2021/gnn-intro/>.
- [29] KM2. Cómo la inteligencia artificial está revolucionando el mercado inmobiliario, 2025. URL <https://km2.es/inteligencia-artifical-mercado-inmobiliario/>.
- [30] IArtificial Blog. La ia de fotocasa predice qué ciudades de españa subirán o bajarán de precio en 2025, 2025. URL https://iartificial.blog/noticias/la-ia-de-fotocasa-predice-que-ciudades-de-espana-subiran-o-bajaran-de-precio-en-2025/#la_inteligencia_social_del_mercado_el_caso_real_que_sorprendio_a_los_desarrolladores.
- [31] Martín Hernández. Fases del kdd, jan 2024. URL <https://www.adn-mundo.com/fases-del-kdd/>.
- [32] Data Crew. Idealista18: Un conjunto de datos abierto, 2024. URL <https://www.idealista.com/labs/blog/?p=4207>.
- [33] Rafael Antonio Paez, Pelayo Arbués. idealista18, 2025. URL <https://github.com/paezha/idealista18>.
- [34] Fernando Lopez y Antonio Paez David Rey-Blanco, Pelayo Arbues. A geo-referenced micro-data set of real estate listings for spain's three largest cities. *Environment and Planning B: Urban Analytics and City Science*, 2024. doi: 10.1177/23998083241242844. URL <https://journals.sagepub.com/doi/full/10.1177/23998083241242844>.

- [35] Ingeniería Naranja. ¿qué influye en el valor de la vivienda cuando vas a comprar?, 2024. URL <https://www.ing.es/ennaranja/comprar-casa/que-influye-en-el-valor-de-la-vivienda-cuando-vas-a-comprar/>.
- [36] Idealista. El verdadero coste de tener terraza: los pisos con espacio exterior son un 36 URL [https://www.idealista.com/news/inmobiliario/vivienda/2017/06/09/746818-el-verdadero-coste-de-tener-terraza-los-pisos-con-espacio-exterior-son-un-](https://www.idealista.com/news/inmobiliario/vivienda/2017/06/09/746818-el-verdadero-coste-de-tener-terraza-los-pisos-con-espacio-exterior-son-un-36)
- [37] Anna Gaya. ¿cuánto influye el ascensor en el precio de la vivienda?, 2016. URL <https://www.api.cat/noticias/cuanto-influye-el-ascensor-en-el-precio-de-la-vivienda/>.
- [38] 20minutos. Estas son las ciudades españolas con más (y menos) viviendas con aire acondicionado, 2020. URL <https://www.20minutos.es/noticia/4326224/0/ciudades-espanolas-mas-menos-viviendas-aire-acondicionado/>.
- [39] Anaïs López. Una plaza de garaje incrementa el valor de un piso un 7,5 URL <https://www.fotocasa.es/fotocasa-life/compraventa/una-plaza-de-garaje-incrementa-el-valor-de-un-piso-en-espana/>.
- [40] BBVA. ¿cómo influye la orientación de la casa en el precio?, 2022. URL <https://www.bbva.es/finanzas-vistazo/ef/hipotecas/orientacion-de-la-casa-influye-precio.html>.
- [41] Raúl Alonso. Los 27 factores que suman o restan valor en la tasación de tu vivienda, 2021. URL <https://www.idealista.com/news/inmobiliario/vivienda/2021/02/23/789199-los-27-factores-que-suman-o-restan-valor-en-la-tasacion-de-tu-vivienda>.
- [42] Tinsa. ¿qué factores influyen en el valor de una vivienda?, 2025. URL <https://www.tinsa.es/blog/tasaciones/que-factores-influyen-en-el-valor-de-una-vivienda/>.
- [43] Intereconomía. La proximidad a un colegio o parada de metro aumenta el precio de la vivienda, 2016. URL <https://www.intereconomia.com/noticia/empresas/la-proximidad-colegio-parada-metro-aumenta-precio-la-vivienda-20161110-1504/>.
- [44] Portal de Datos Abiertos de Valencia. Valencia barris/barrios - datos abiertos, 2023. URL <https://tdata.dlsi.ua.es/dataset/336451140282-valencia-barris-barrios/resource/619d466f-18d0-4abe-a388-ebcf48da683d>.
- [45] Virgilio Oliva Alonso. tfg-idealista. <https://github.com/violialo/tfg-idealista>, 2024. URL <https://github.com/violialo/tfg-idealista>. Repositorio en GitHub.

A. Anexos

A.1. Actas de reuniones

La sección tiene como objetivo recoger por medio de tablas las reuniones realizadas a lo largo del recorrido de vida del proyecto.

En total se han registrado siete reuniones, cuyos detalles se recogen en las tablas expuestas a continuación:

Acta de Reunión	
Fecha	18 de septiembre de 2024
Hora	16:00 - 16:30
Lugar	Sala virtual de Microsoft Teams
Asistentes	Manuel J. Jiménez, Andreea M. Oprescu, V. Oliva
Temas tratados	■ Pautas iniciales del trabajo a realizar ■ Toma de contacto con la página del trabajo ■ Dudas sobre el trabajo a realizar
Tareas asignadas	No aplica.

Tabla A.2: Registro de reunión del 18 de septiembre de 2024

Acta de Reunión	
Fecha	21 de noviembre de 2024
Hora	11:00 - 11:30
Lugar	Sala virtual de Microsoft Teams
Asistentes	Manuel J. Jiménez, Andreea M. Oprescu, V. Oliva
Temas tratados	■ Tecnologías a utilizar ■ Ideas principales del trabajo ■ Recursos ofrecidos
Tareas asignadas	■ Definir requisitos iniciales ■ Realizar formación del curso Python

Tabla A.4: Registro de reunión del 21 de noviembre de 2024

Acta de Reunión	
Fecha	30 de enero de 2025
Hora	10:00 - 10:30
Lugar	Escuela Técnica Superior de Ingeniería Informática - E0.41
Asistentes	Manuel J. Jiménez, Andreea M. Oprescu, V. Oliva
Temas tratados	■ Etapa de planificación del trabajo ■ Dudas a resolver
Tareas asignadas	■ Definir una planificación básica para comenzar con el proyecto ■ Establecer hitos del proyecto

Tabla A.6: Registro de reunión del 30 de enero de 2025

Acta de Reunión	
Fecha	27 de marzo de 2025
Hora	12:30 - 13:30
Lugar	Escuela Técnica Superior de Ingeniería Informática - F1.82
Asistentes	Manuel J. Jiménez, Andreea M. Oprescu, V. Oliva
Temas tratados	■ Entrega planificación completa y replanificación ■ Cambio de tecnologías
Tareas asignadas	■ Comenzar a investigar grafos y redes neuronales de grafos

Tabla A.8: Registro de reunión del 27 de marzo de 2025

Acta de Reunión	
Fecha	15 de mayo de 2025
Hora	12:30 - 13:30
Lugar	Sala virtual de Microsoft Teams
Asistentes	Manuel J. Jiménez, Andreea M. Oprescu, V. Oliva
Temas tratados	■ Etapa de selección del KDD
Tareas asignadas	■ Comenzar con la etapa de preprocesamiento y transformación de los datos

Tabla A.10: Registro de reunión del 15 de mayo de 2025

Acta de Reunión	
Fecha	27 de mayo de 2025
Hora	16:00 - 16:45
Lugar	Sala virtual de Microsoft Teams
Asistentes	Manuel J. Jiménez, Andreea M. Oprescu, V. Oliva
Temas tratados	■ Código implementado para los grafos ■ Problema de visualización y representación de grafos ■ Revisión primer incremento de la memoria
Tareas asignadas	■ Entrenar buscando hiperparámetros por Optuna ■ Dividir el entrenamiento por barrios, no como todo en conjunto ■ Añadir condición para evitar nodos aislados ■ Añadir un mapa interactivo para la interfaz a realizar

Tabla A.12: Registro de reunión del 27 de mayo de 2025

Acta de Reunión	
Fecha	11 de junio de 2025
Hora	16:00 - 17:00
Lugar	Sala virtual de Microsoft Teams
Asistentes	Manuel J. Jiménez, Andreea M. Oprescu, V. Oliva
Temas tratados	■ Interfaz implementada para la predicción ■ Propuesta de incremento de la aplicación ■ Sugerencias de mejora
Tareas asignadas	■ Eliminar el volver a entrenar para la interfaz web ■ Elaborar el resto de capítulos de la memoria

Tabla A.14: Registro de reunión del 11 de junio de 2025

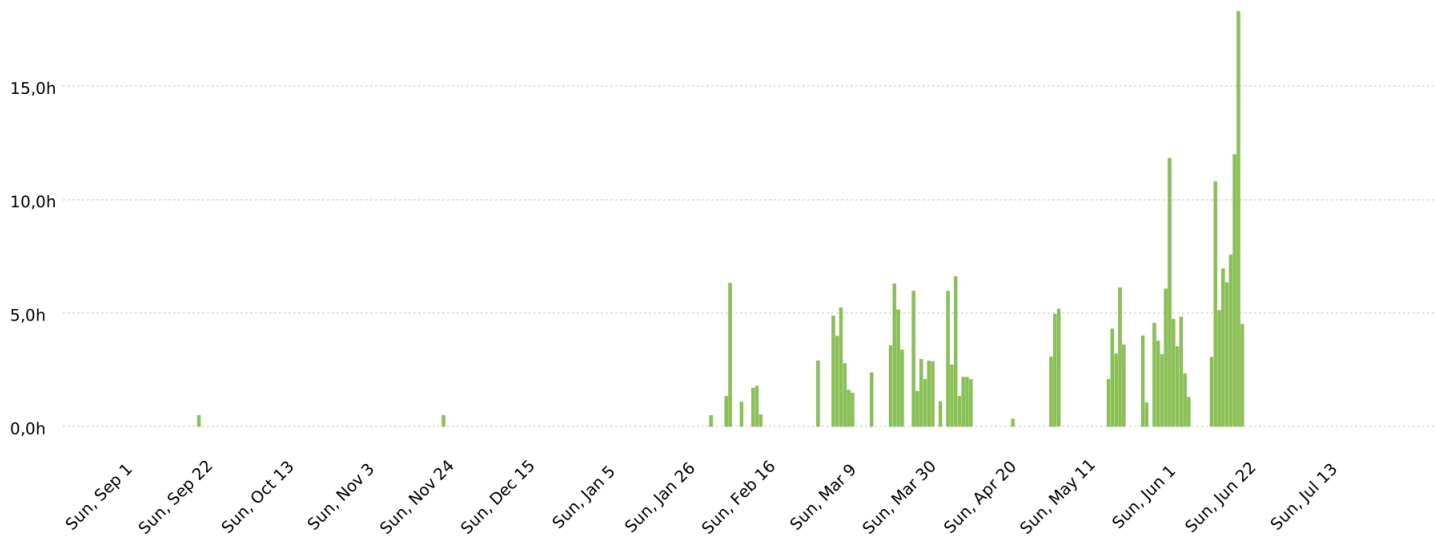
A.2. Informe de Clockify

Summary report

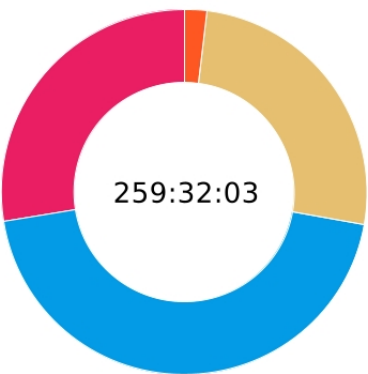
01/09/2024 - 20/07/2025



Total: 259:32:03 Billable: 259:32:03 Amount: 0,00 USD

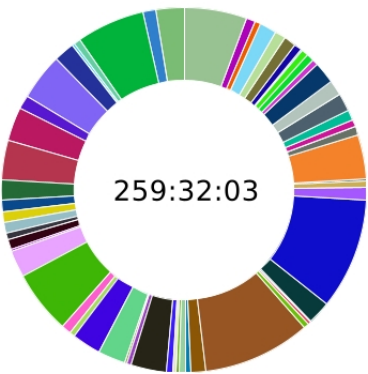


Project



Cierre	71:27:55	27,54%
Implementación	115:53:30	44,65%
Planificación	66:55:38	25,79%
Reuniones	05:15:00	2,02%

Description



Memoria: Redactar capítulo Planificación	06:26:00	2,48%
Iteración 1: Revisar resultados	03:00:00	1,16%
Memoria: Revisión memoria	15:32:03	5,99%
Gestión de los Riesgos: Planificar control y seguimiento riesgos	01:34:00	0,60%
Memoria: Redactar capítulo Introducción	00:45:06	0,29%
Manuales: Revisar manuales	04:31:00	1,74%
Memoria: Redactar capítulo Cierre	10:53:34	4,20%

● Gestión de la Calidad: Establecer técnicas y procesos	03:04:43	1,19%
● Iteración 3: Realizar interfaz	07:46:19	2,99%
● Iteración 3: Visualizar resultados	08:56:07	3,44%
● Estudio previo: Realizar cursos introductorios Python	04:42:00	1,81%
● Gestión de la Calidad: Planificar aseguramiento de la Calidad	02:38:00	1,01%
● Gestión del Tiempo: Identificar, secuenciar y establecer duración de las tareas	02:23:00	0,92%
● Iteración 2: Revisar resultados	02:34:00	0,99%
● Gestión del Alcance: Definir enunciado del alcance	01:32:07	0,59%
● Configuración del entorno: Instalar tecnologías	02:11:00	0,84%
● Gestión de los Costes: Planificar control y seguimiento costes	00:33:08	0,21%
● Estudio previo: Realizar ejemplo práctico GNN	06:02:00	2,32%
● Gestión del Tiempo: Crear cronograma	14:35:29	5,62%
● Estudio previo: Revisar datos a usar	02:11:00	0,84%
● Gestión de los Riesgos: Identificar riesgos	01:18:11	0,50%
● Iteración 2: Procesar datos	06:41:56	2,58%
● Gestión de los Costes: Identificar costes	06:18:54	2,43%
● Iteración 2: Validar modelos	00:24:21	0,16%
● Iteración 1: Validar modelos	01:02:51	0,40%
● Manuales: Redactar manuales	08:06:00	3,12%
● Gestión de la Comunicación: Establecer técnicas y herramientas	01:29:34	0,58%
● Gestión del Alcance: Crear EDT/WBS	00:51:32	0,33%
● Configuración del entorno: Preparar GitHub y Projects	00:41:25	0,27%
● Gestión de Interesados: Registrar interesados	01:26:45	0,56%
● Metodología: Establecer metodologías de desarrollo	01:07:49	0,44%
● Gestión de los Costes: Crear presupuesto	03:23:00	1,30%
● Memoria: Redactar capítulo Ejecución	24:38:24	9,49%
● Estudio previo: Revisar tecnologías a usar	01:21:00	0,52%

● Memoria: Redactar capítulo Anexos	00:35:48	0,23%
● Metodología: Registrar tecnologías	00:13:36	0,09%
● Reunión TFG	05:15:00	2,02%
● Iteración 1: Procesar datos	25:36:27	9,87%
● Configuración del entorno: Configurar entorno de desarrollo	02:58:18	1,15%
● Equipo de proyecto: Identificar participantes del proyecto	01:21:32	0,52%
● Gestión del Tiempo: Planificar control y seguimiento del cronograma	00:27:42	0,18%
● Iteración 3: Consultar precio vivienda	10:09:59	3,92%
● Iteración 3: Registrar y almacenar usuarios	02:06:52	0,81%
● Gestión de Interesados: Clasificar interesados	01:30:00	0,58%
● Gestión de Interesados: Definir roles del sistema	02:17:24	0,88%
● Iteración 1: Entrenar modelos	04:03:51	1,57%
● Metodología: Crear Plan de Gestión de la Configuración	03:30:16	1,35%
● Gestión del Alcance: Registrar y clasificar requisitos	05:05:25	1,96%
● Gestión de los Riesgos: Establecer planes de contingencia	01:13:48	0,47%
● Iteración 1: Validar modelos	01:44:52	0,67%
● Gestión de la Calidad: Identificar herramientas	01:35:36	0,61%
● Equipo de proyecto: Definir roles de equipo	00:21:09	0,14%
● Gestión de los Costes: Establecer costes	01:43:00	0,66%
● Estudio previo: Leer artículo GNN	02:44:00	1,05%
● Iteración 1: Generar resultados	02:32:56	0,98%
● Gestión del Tiempo: Replanificación	03:47:58	1,46%
● Gestión del Tiempo: Identificar hitos	01:32:00	0,59%
● Iteración 2: Generar resultados	02:03:28	0,79%
● Iteración 2: Entrenar modelos	14:18:48	5,52%

Project / Description	Duration	Amount
Cierre	71:27:55	0,00 USD
Memoria: Redactar capitulo Planificación	06:26:00	0,00 USD
Memoria: Revisión memoria	15:32:03	0,00 USD
Memoria: Redactar capitulo Introducción	00:45:06	0,00 USD
Manuales: Revisar manuales	04:31:00	0,00 USD
Memoria: Redactar capitulo Cierre	10:53:34	0,00 USD
Manuales: Redactar manuales	08:06:00	0,00 USD
Memoria: Redactar capitulo Ejecución	24:38:24	0,00 USD
Memoria: Redactar capitulo Anexos	00:35:48	0,00 USD
Implementación	115:53:30	0,00 USD
Iteración 1: Revisar resultados	03:00:00	0,00 USD
Iteración 3: Realizar interfaz	07:46:19	0,00 USD
Iteración 3: Visualizar resultados	08:56:07	0,00 USD
Estudio previo: Realizar cursos introductorios Python	04:42:00	0,00 USD
Iteración 2: Revisar resultados	02:34:00	0,00 USD
Configuración del entorno: Instalar tecnologías	02:11:00	0,00 USD
Estudio previo: Realizar ejemplo práctico GNN	06:02:00	0,00 USD
Estudio previo: Revisar datos a usar	02:11:00	0,00 USD
Iteración 2: Procesar datos	06:41:56	0,00 USD
Iteración 2: Validar modelos	00:24:21	0,00 USD
Iteración 1: Validar modelos	01:02:51	0,00 USD
Configuración del entorno: Preparar GitHub y Projects	00:41:25	0,00 USD
Estudio previo: Revisar tecnologías a usar	01:21:00	0,00 USD
Iteración 1: Procesar datos	25:36:27	0,00 USD

Configuración del entorno: Configurar entorno de desarrollo	02:58:18	0,00 USD
Iteración 3: Consultar precio vivienda	10:09:59	0,00 USD
Iteración 3: Registrar y almacenar usuarios	02:06:52	0,00 USD
Iteración 1: Entrenar modelos	04:03:51	0,00 USD
Iteración 1: Validar modelos	01:44:52	0,00 USD
Estudio previo: Leer artículo GNN	02:44:00	0,00 USD
Iteración 1: Generar resultados	02:32:56	0,00 USD
Iteración 2: Generar resultados	02:03:28	0,00 USD
Iteración 2: Entrenar modelos	14:18:48	0,00 USD
Planificación	66:55:38	0,00 USD
Gestión de los Riesgos: Planificar control y seguimiento riesgos	01:34:00	0,00 USD
Gestión de la Calidad: Establecer técnicas y procesos	03:04:43	0,00 USD
Gestión de la Calidad: Planificar aseguramiento de la Calidad	02:38:00	0,00 USD
Gestión del Tiempo: Identificar, secuenciar y establecer duración de las tareas	02:23:00	0,00 USD
Gestión del Alcance: Definir enunciado del alcance	01:32:07	0,00 USD
Gestión de los Costes: Planificar control y seguimiento costes	00:33:08	0,00 USD
Gestión del Tiempo: Crear cronograma	14:35:29	0,00 USD
Gestión de los Riesgos: Identificar riesgos	01:18:11	0,00 USD
Gestión de los Costes: Identificar costes	06:18:54	0,00 USD
Gestión de la Comunicación: Establecer técnicas y herramientas	01:29:34	0,00 USD
Gestión del Alcance: Crear EDT/WBS	00:51:32	0,00 USD
Gestión de Interesados: Registrar interesados	01:26:45	0,00 USD
Metodología: Establecer metodologías de desarrollo	01:07:49	0,00 USD
Gestión de los Costes: Crear presupuesto	03:23:00	0,00 USD
Metodología: Registrar tecnologías	00:13:36	0,00 USD

Equipo de proyecto: Identificar participantes del proyecto	01:21:32	0,00 USD
Gestión del Tiempo: Planificar control y seguimiento del cronograma	00:27:42	0,00 USD
Gestión de Interesados: Clasificar interesados	01:30:00	0,00 USD
Gestión de Interesados: Definir roles del sistema	02:17:24	0,00 USD
Metodología: Crear Plan de Gestión de la Configuración	03:30:16	0,00 USD
Gestión del Alcance: Registrar y clasificar requisitos	05:05:25	0,00 USD
Gestión de los Riesgos: Establecer planes de contingencia	01:13:48	0,00 USD
Gestión de la Calidad: Identificar herramientas	01:35:36	0,00 USD
Equipo de proyecto: Definir roles de equipo	00:21:09	0,00 USD
Gestión de los Costes: Establecer costes	01:43:00	0,00 USD
Gestión del Tiempo: Replanificación	03:47:58	0,00 USD
Gestión del Tiempo: Identificar hitos	01:32:00	0,00 USD
Reuniones	05:15:00	0,00 USD
Reunión TFG	05:15:00	0,00 USD